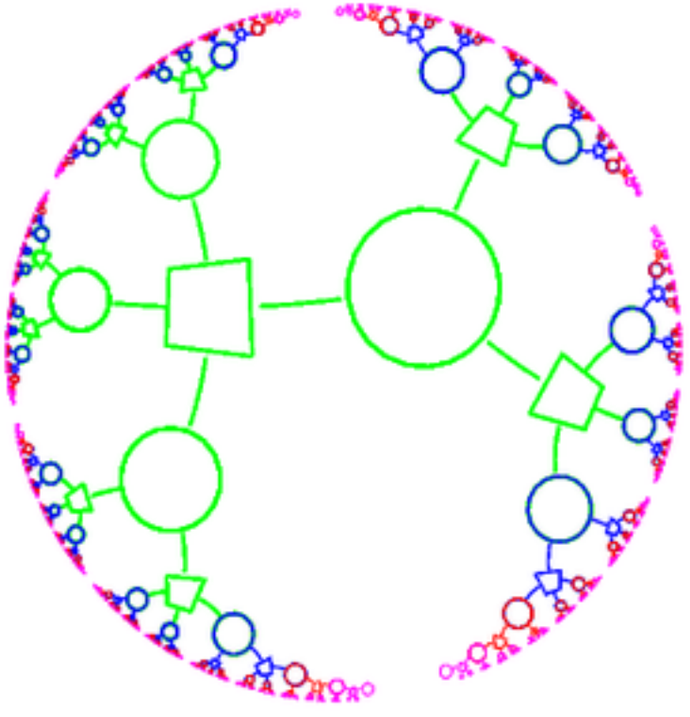
Neural networks I

- Motivations
- Feedforward networks
  - single neuron
  - "learning" for one neuron
  - learning as communication:
    the capacity of one neuron
  - multilayer networks
  - learning as inference



Information theory, pattern recognition, and neural networks

- Source coding (Data compression)
- Noisy-channel coding
- Inference + probabilistic methods
  - 9-10 Inference
  - 11 Clustering
  - 12 Monte Carlo methods
  - 13 Advanced Monte Carlo methods
  - 14 Variational methods
- Neural networks
  - 15 Introduction to feedforward neural networks
  - 16 Content-addressable memories
- State-of-the-art error-correcting codes

www.inference.phy.cam.ac.uk/itprnn/
www.inference.phy.cam.ac.uk/itila/

Reading

- Neural networks
  - Chs 38, 39, (& perhaps 41, 44), 42
- State-of-the-art error-correcting codes
  - Chs 47, (48, 49,) & 50

Brains

How to make a content-addressable memory?

Content-addressable memory

Shaken,

not...

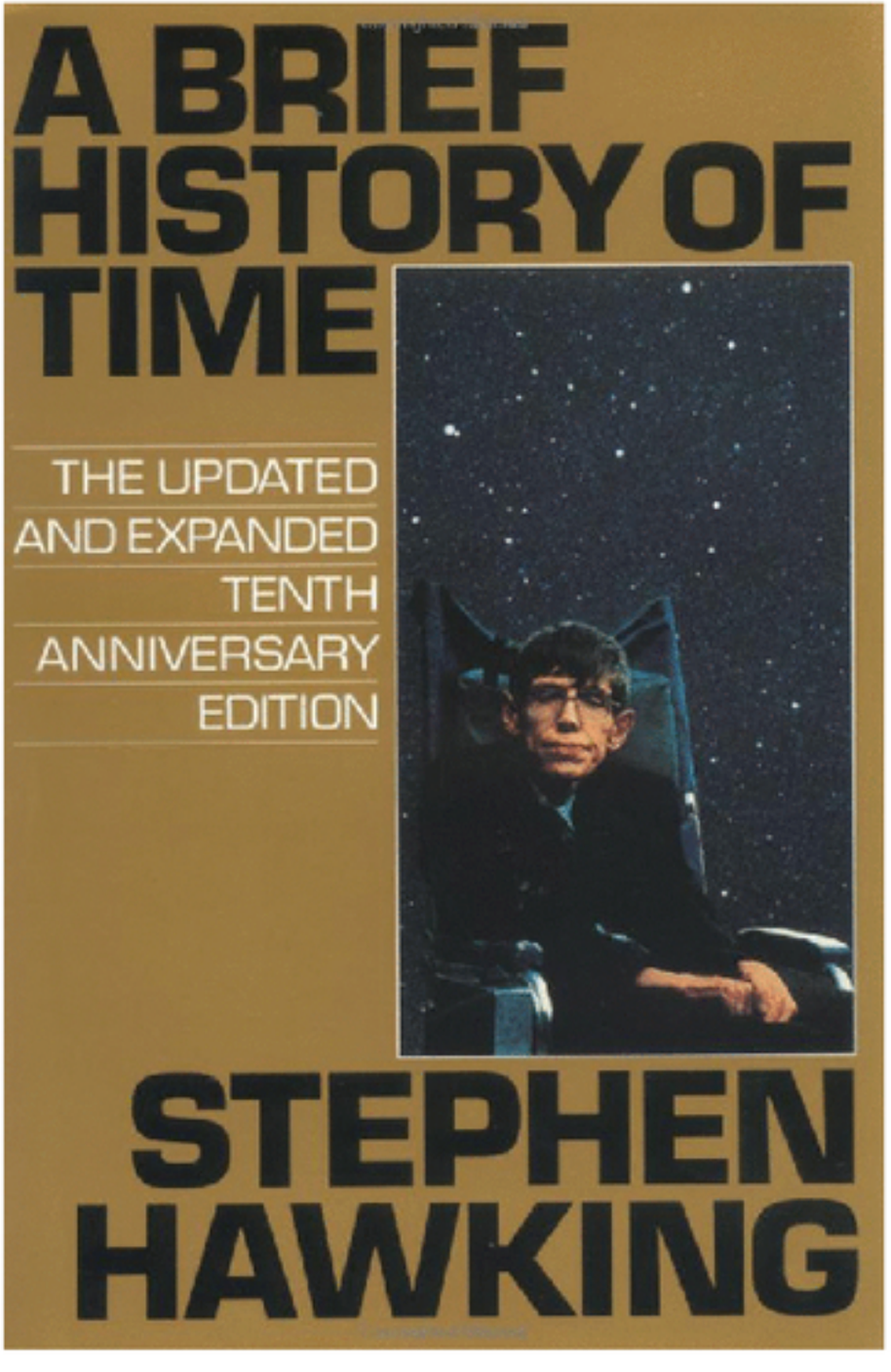
Content-addressable memory

Oscar-nominated actress called Natalie P******
Acted a senator in an amazingly good Lucas film

Content-addressable memory

An author

Content-addressable memory

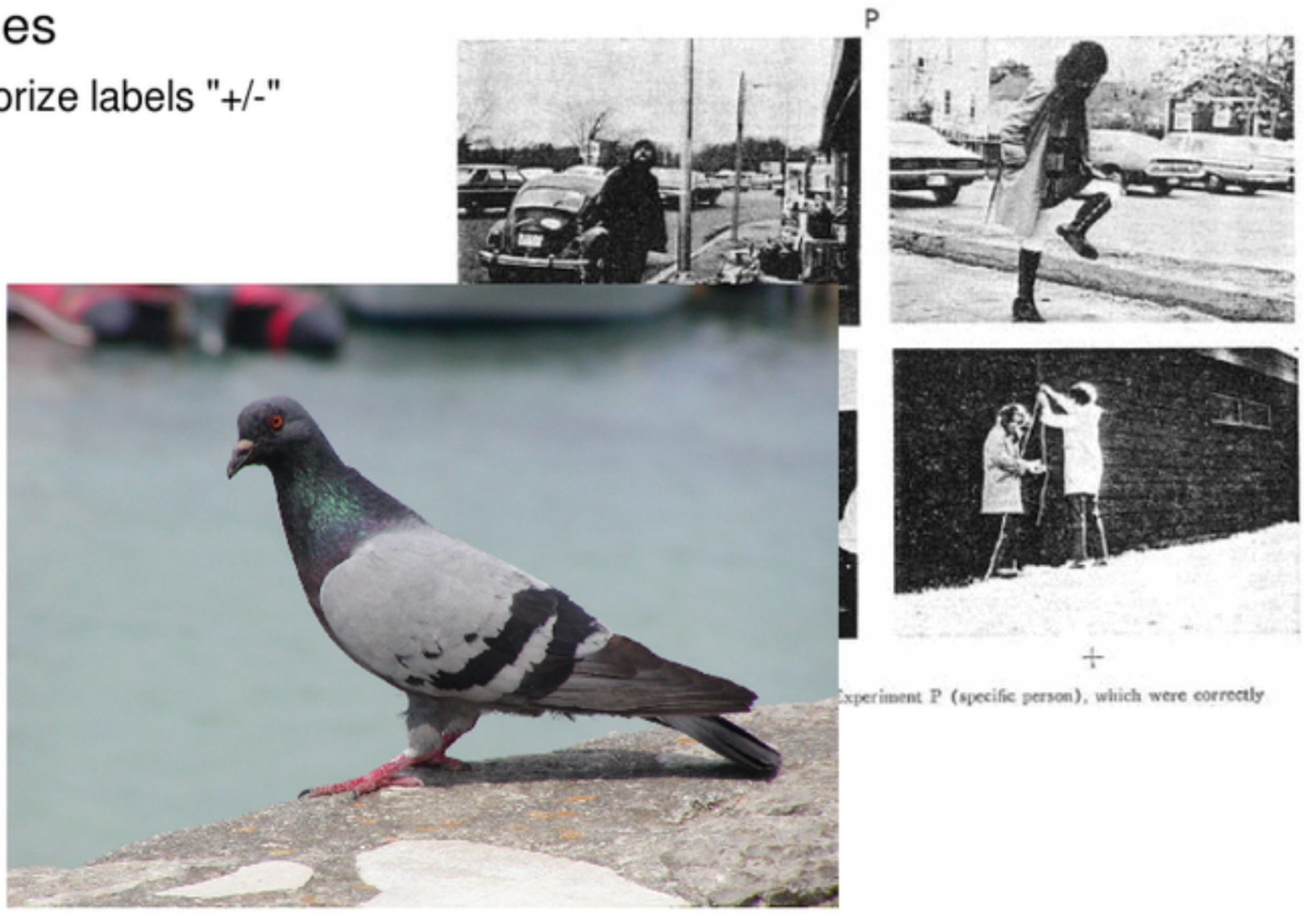


Content-addressable memory

- How to do it?
- How do brains do it?

Brains

- Shown 725 images
  - instructed to memorize labels "+/-"



Fersen, L. & Delius, J. D. (1989) Long-term retention of many visual patterns by pigeons. Ethology 82: 141-155.

What is the difference between

a pigeon and a supercomputer?

	Pigeon	Cray
Hardware	10^{11} neurons 10^{14} synapses	10^{10} bits of memory 1 Gig memory CPU $\simeq 10^6$ devices
Clock rate		1000 MHz
		$10^{15} - 10^{19}$ dance operations per second

Pigeon

Hardware

10^{11} neurons

10^{14} synapses

Clock rate

10ms clock time

10

Pigeon

10^{11} neurons

10^{14} synapses

10ms clock time

10^{13} operations per second

10^{16} operations per second

Cray

10^{10} bits of memory 1 Gig memory

CPU $\simeq 10^6$ devices

1000 MHz

$\rightarrow 10^{15} - 10^{19}$ device operations per second ✓

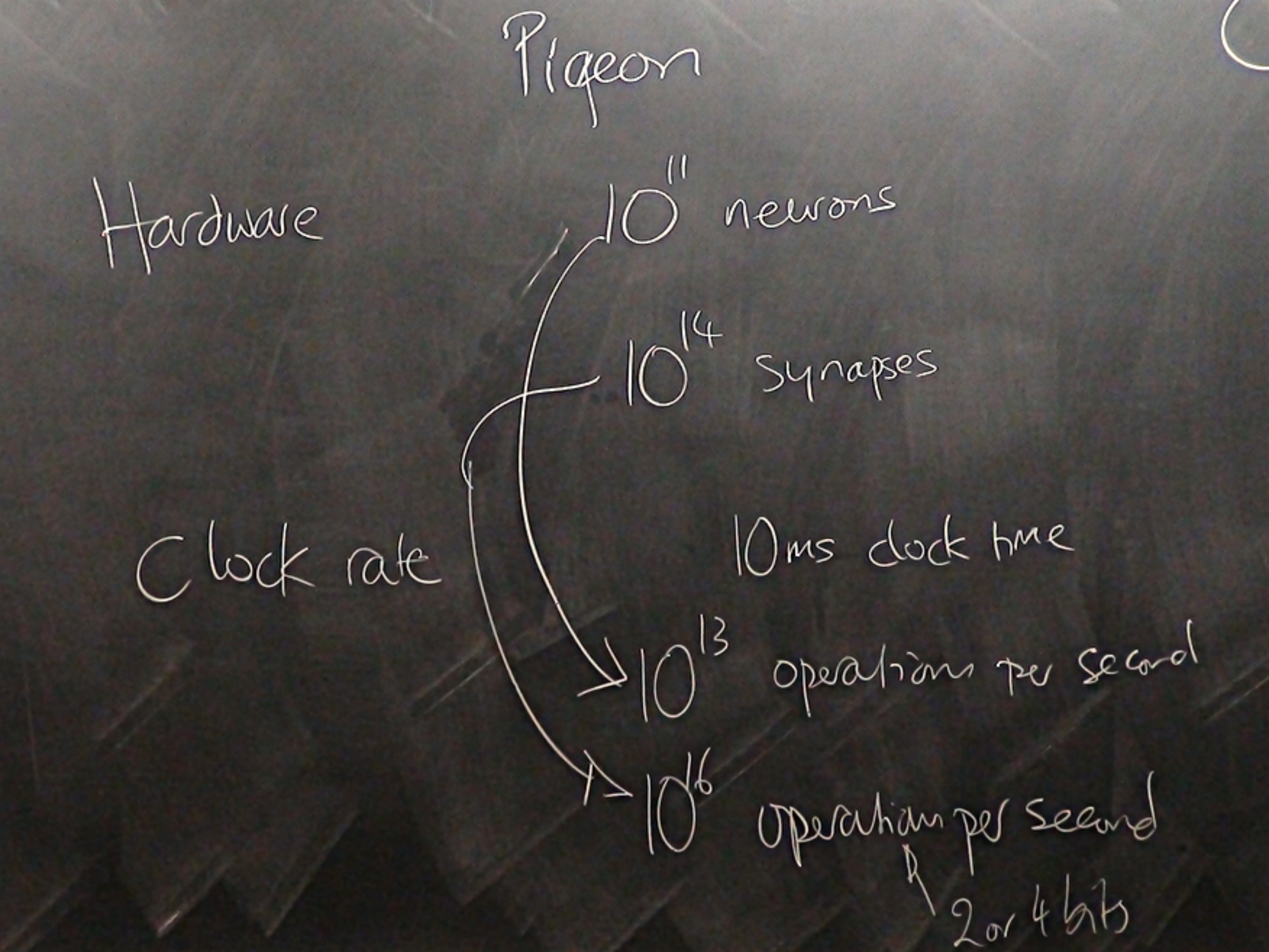
$\rightarrow 10^{16}$ flops / second ← (Today's top Supercomputer)

memory

e operations per second ✓

d ← (Today's top Supercomputer)

32 bit × 32 bit ?


Pigeon
Hardware
10^{11} neurons
10^{14} synapses
Clock rate
10ms clock time
10^{13} operations per second
10^{16} operations per second
2 or 4 bits

Cray

10^{10} bits of memory 1 Gig memory

CPU $\simeq 10^6$ devices

1000 MHz

$\rightarrow 10^{15} - 10^{19}$ device operations per second ✓

$\rightarrow 10^{16}$ flops / second ← (Today's top Supercomputer)

32 bit

Pigeon

Parallel

High connectivity
(~1000)

Distributed

New memories stored in same hardware

Gray

Serial

~3 or 10

CPU

New memories in new virgin hardware

Pigeon

Crow

Robust

Not

Idea maybe the style of computation is the key

Parallel Distributed Processing

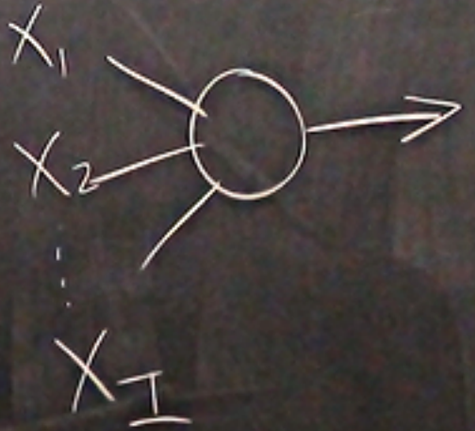
Neural Networks, Part I:

Feedforward Networks

Chapters 39, 41, 44

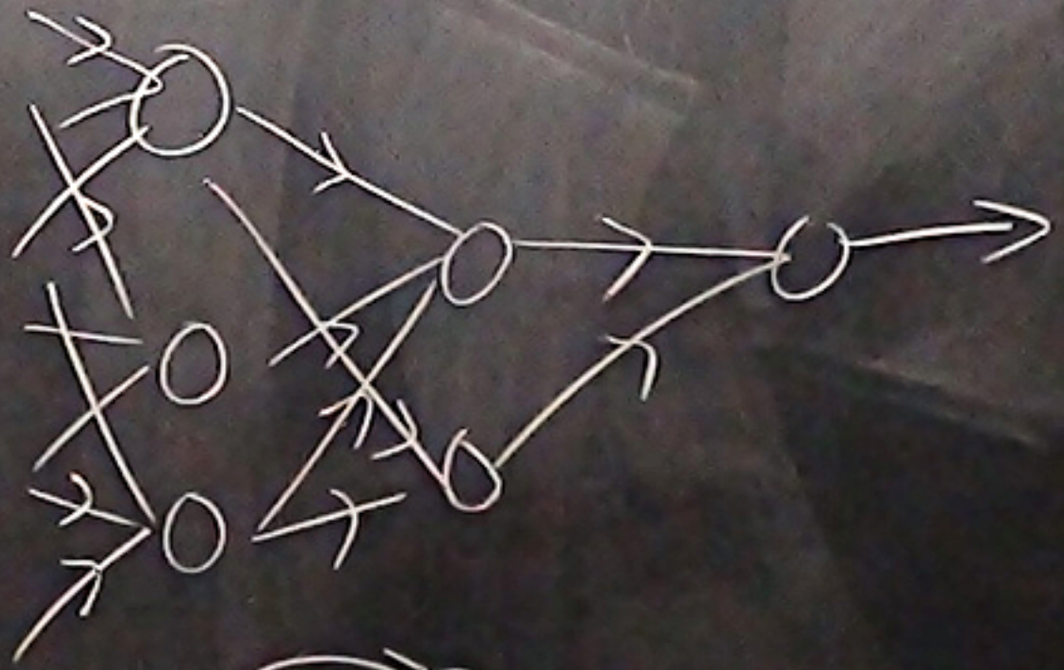
Parallel Distributed Processing

Single neuron

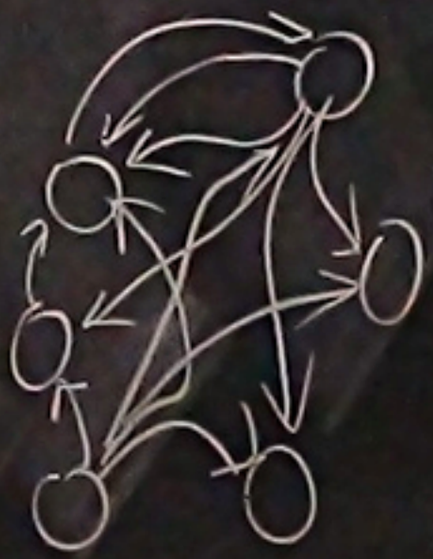


feedforward

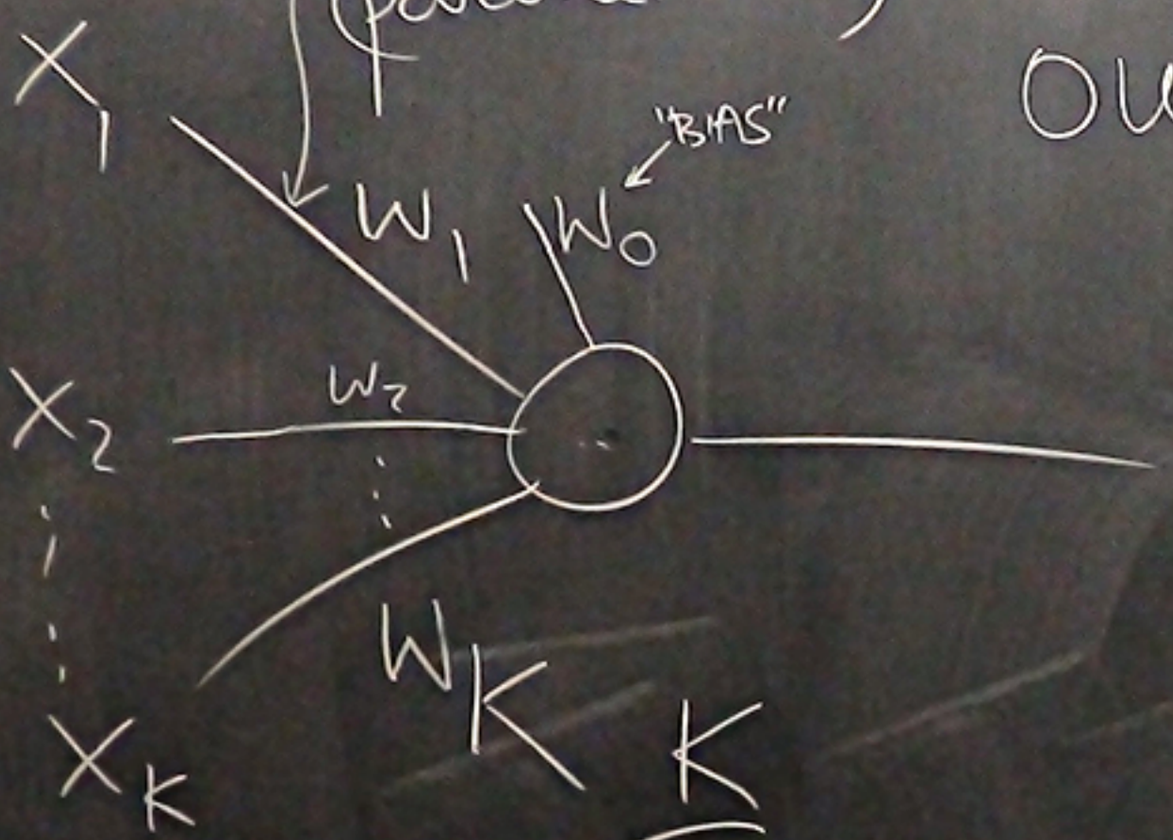
feedforward



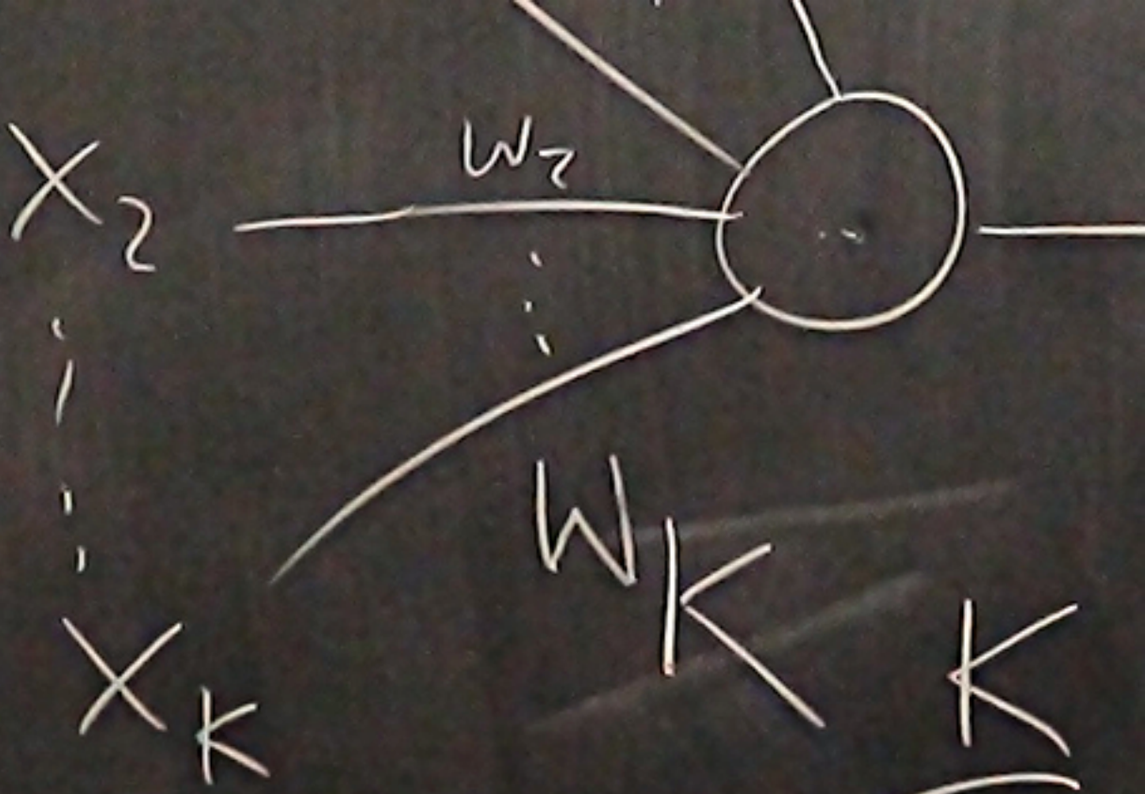
feedback



inputs "weights" (parameters)



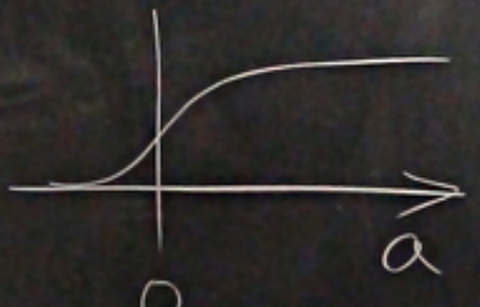
$$a = \sum_{k=1}^{K} w_k x_k + w_0$$



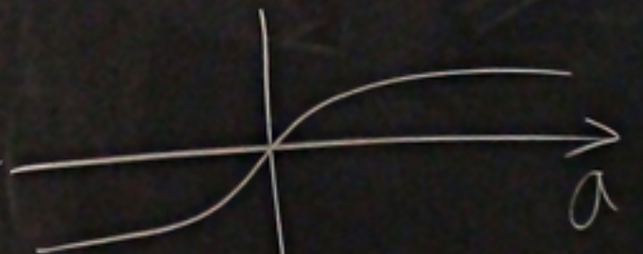
activation $a \equiv \sum_{k=1}^{K}$

output activity $y = f(a)$

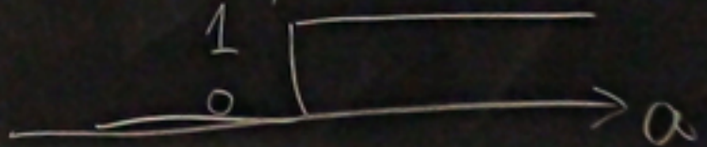
eg
- $f(a) = \frac{1}{1+e^{-a}}$



- $f(a) = \tanh(a)$

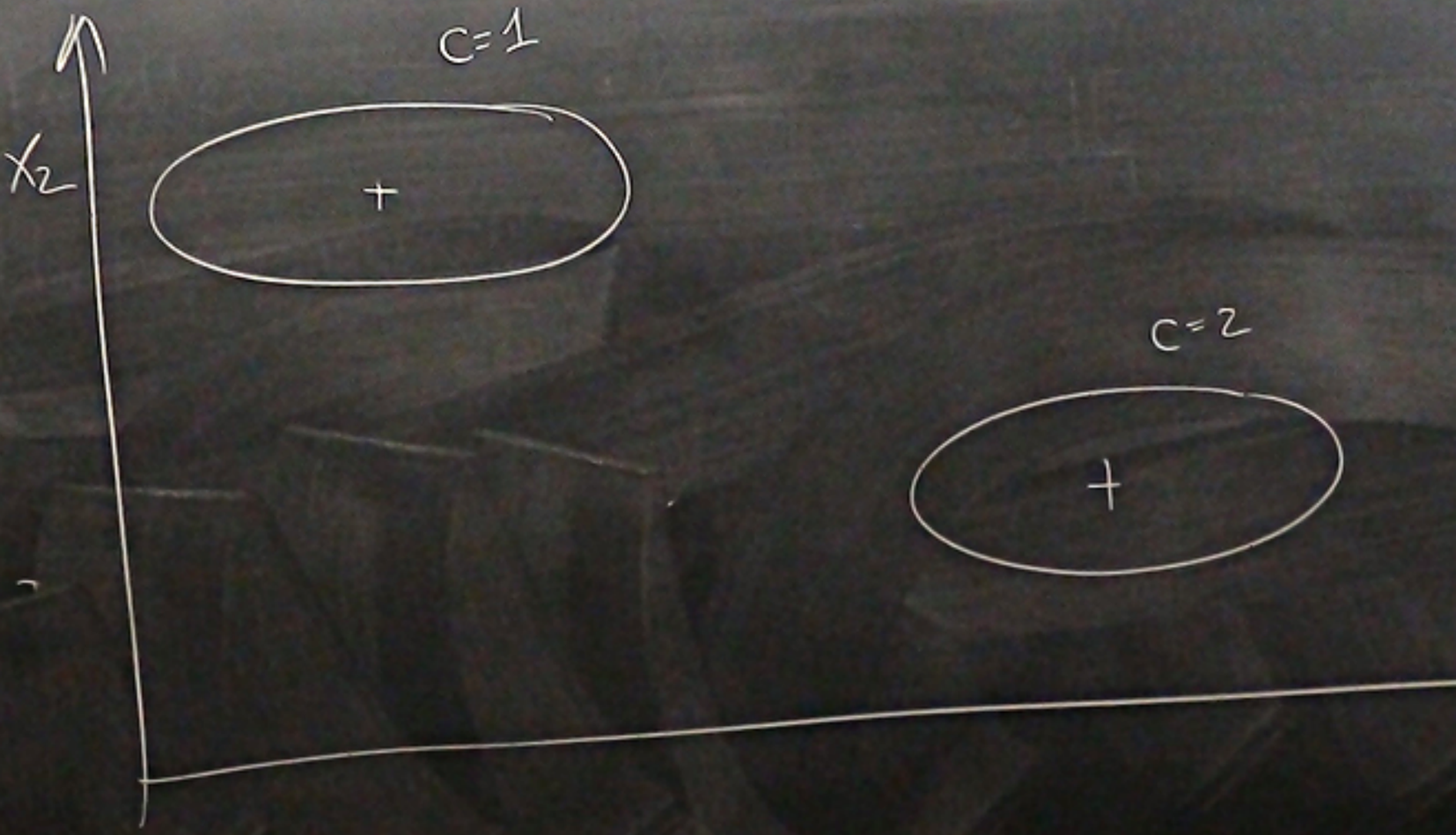


- step fn

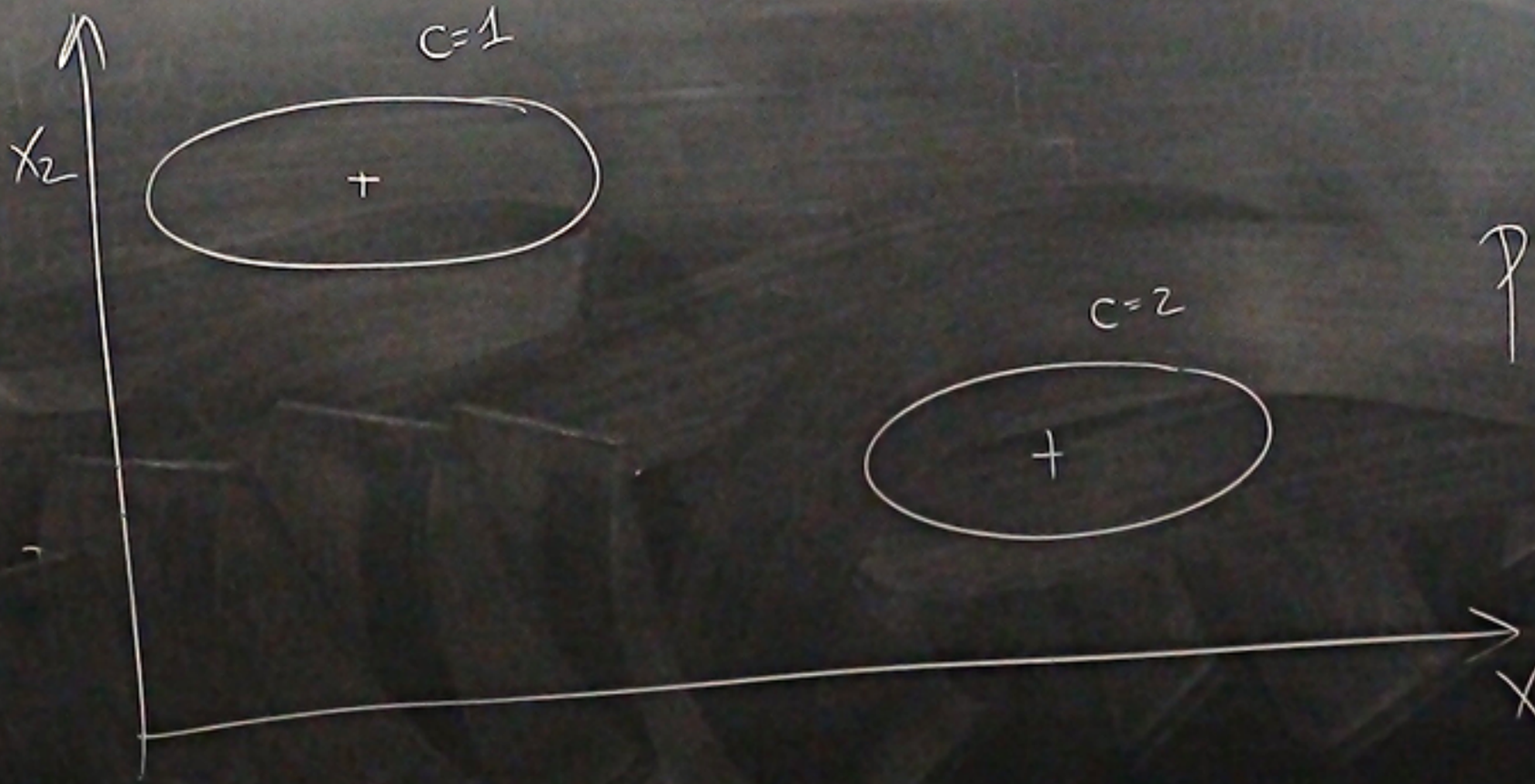


Familiar! 1) VFE minimizer for spin system

2)



VFE minimum for spin system



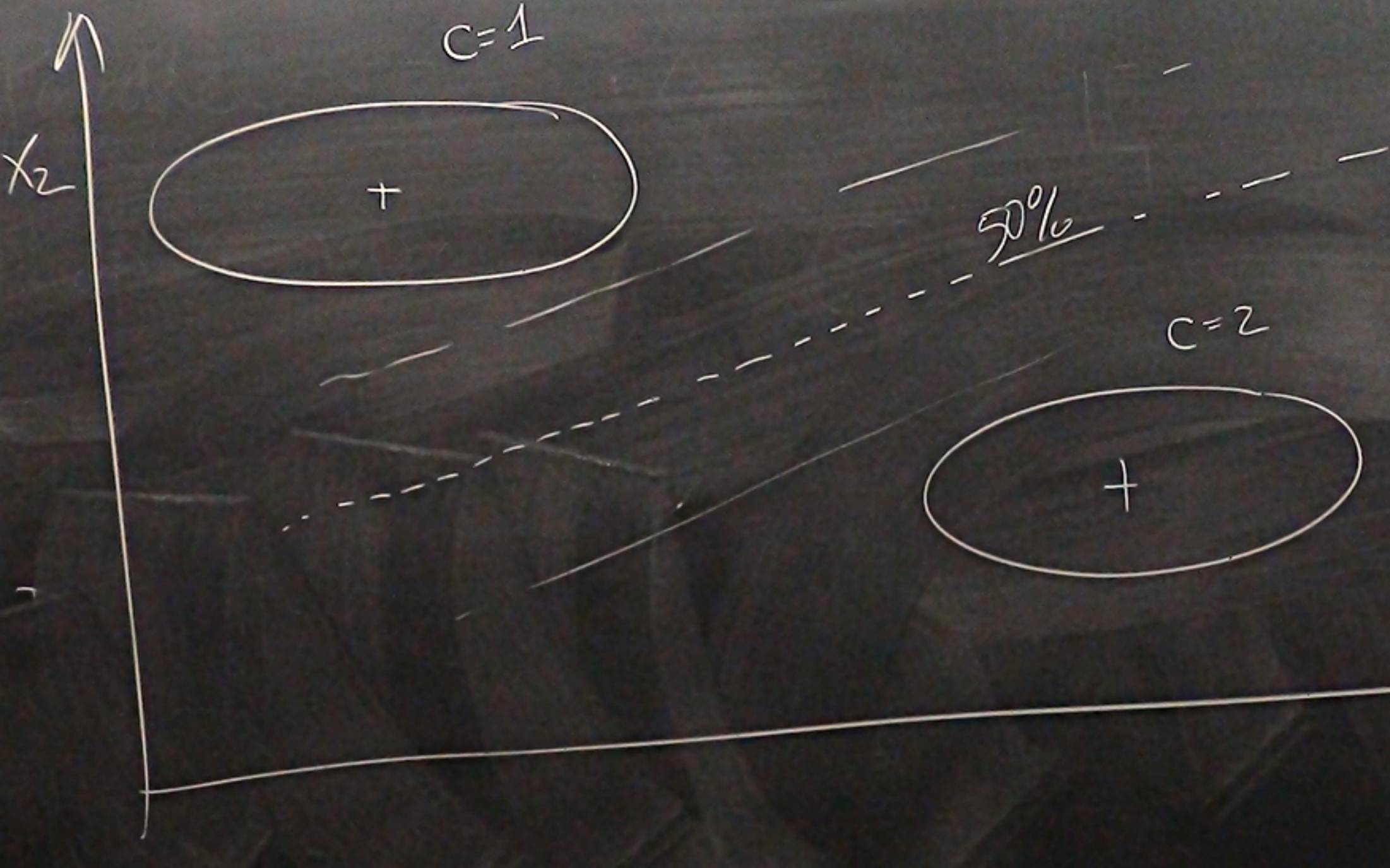
$$P(C=1 \mid \underline{x})$$

$$p(c=1 \mid \underline{x}) = f(a)$$

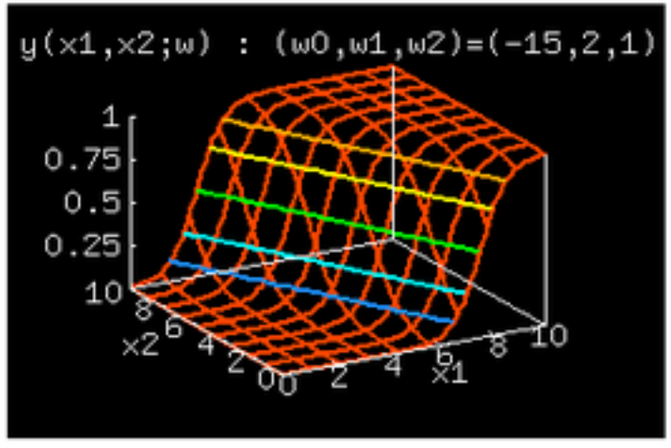
$$\frac{1}{1+e^{-a}}$$

$$a = \sum w_k x_k + w_0$$

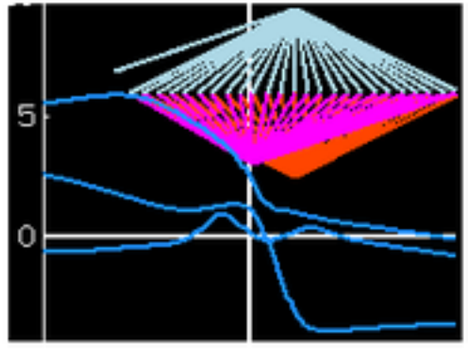
VFE minimizer for spin system



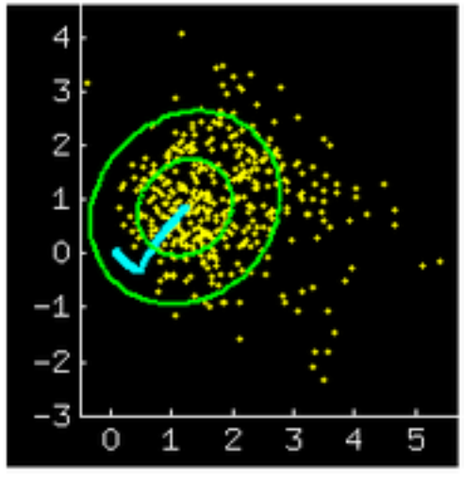
- Single neuron as a binary classifier



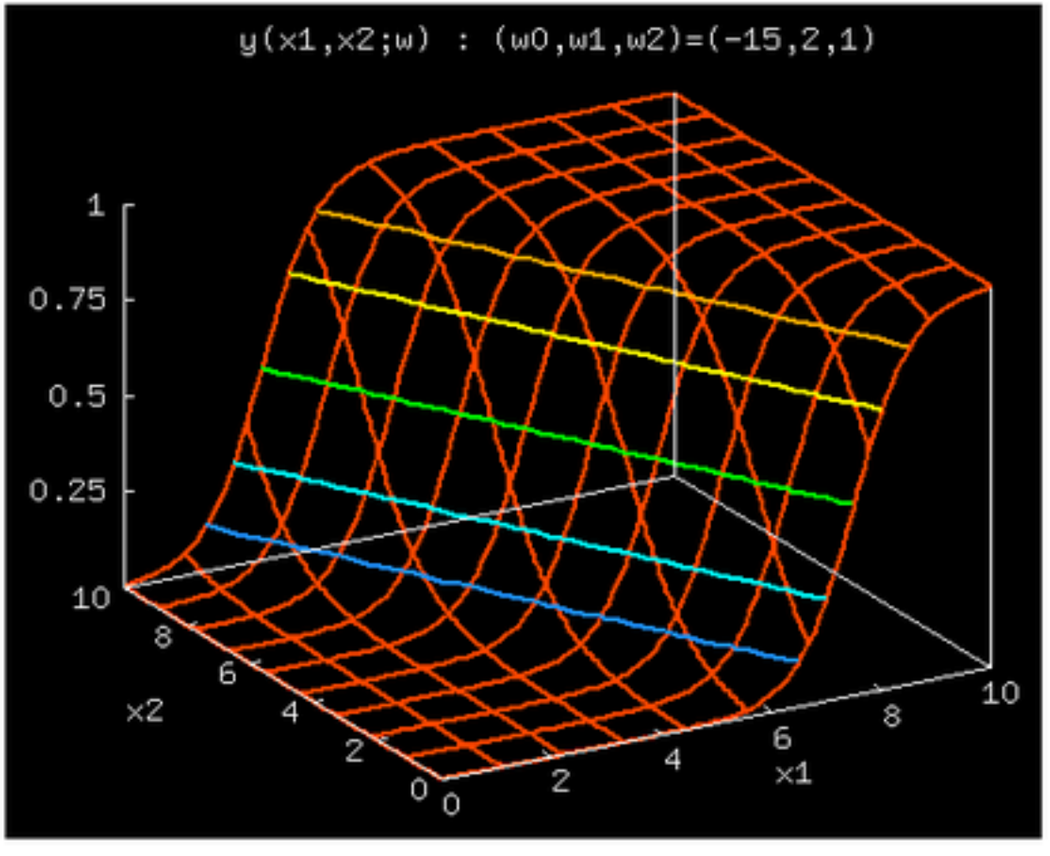
- Multi-layer network for regression



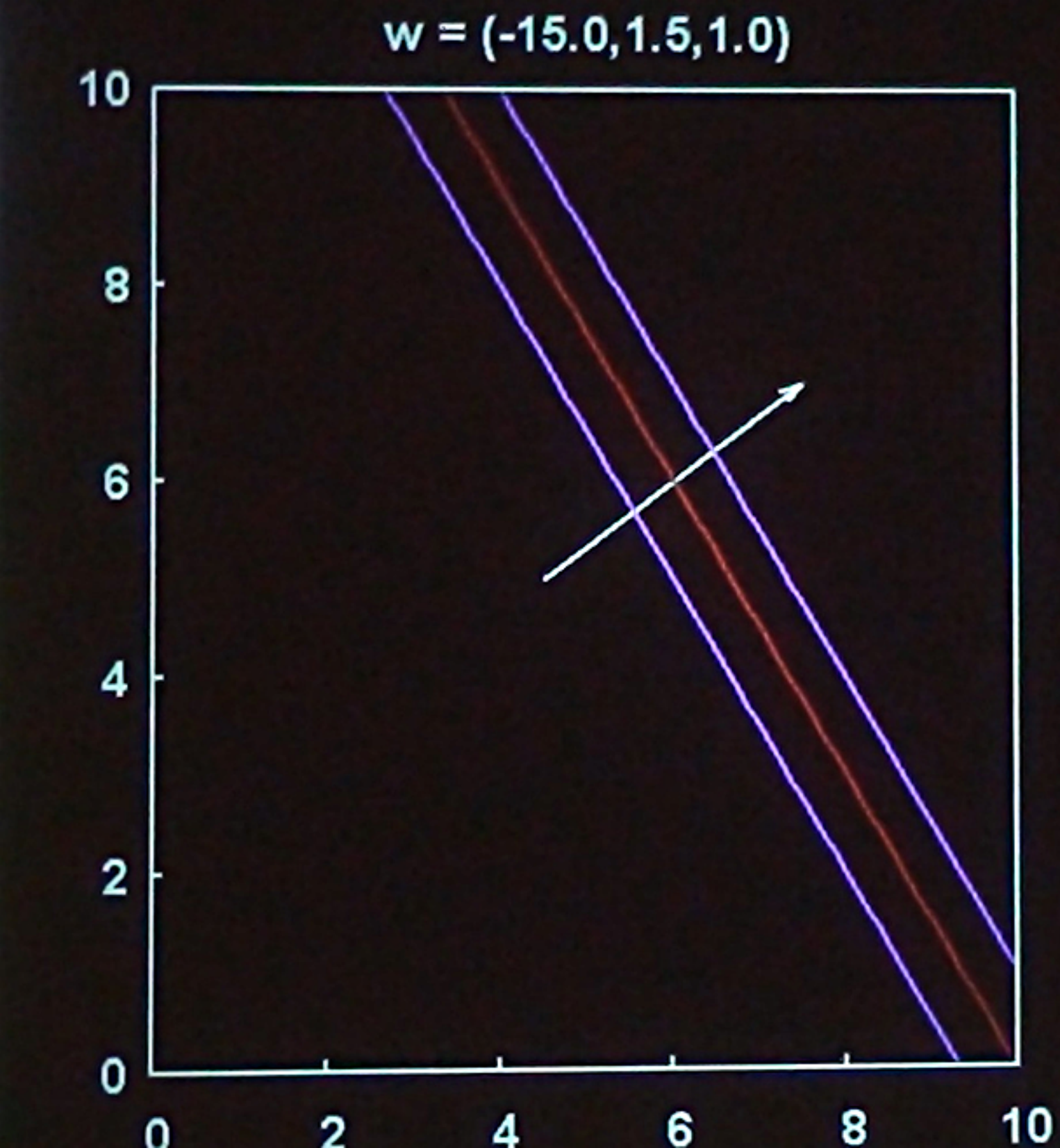
- Probabilistic interpretation of learning



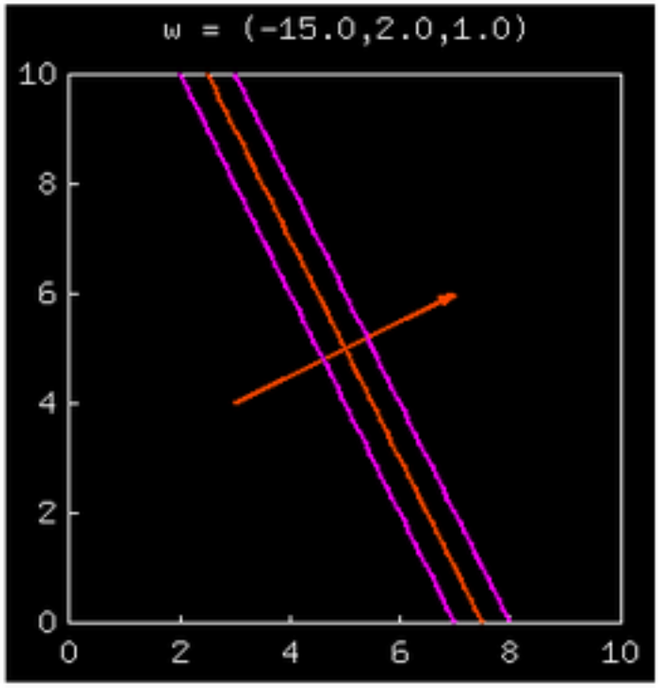
Learning for a Single Neuron

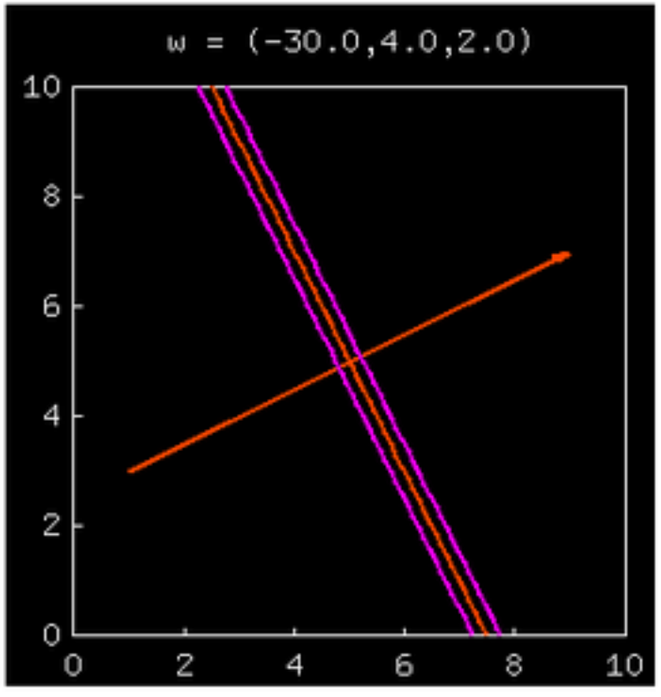


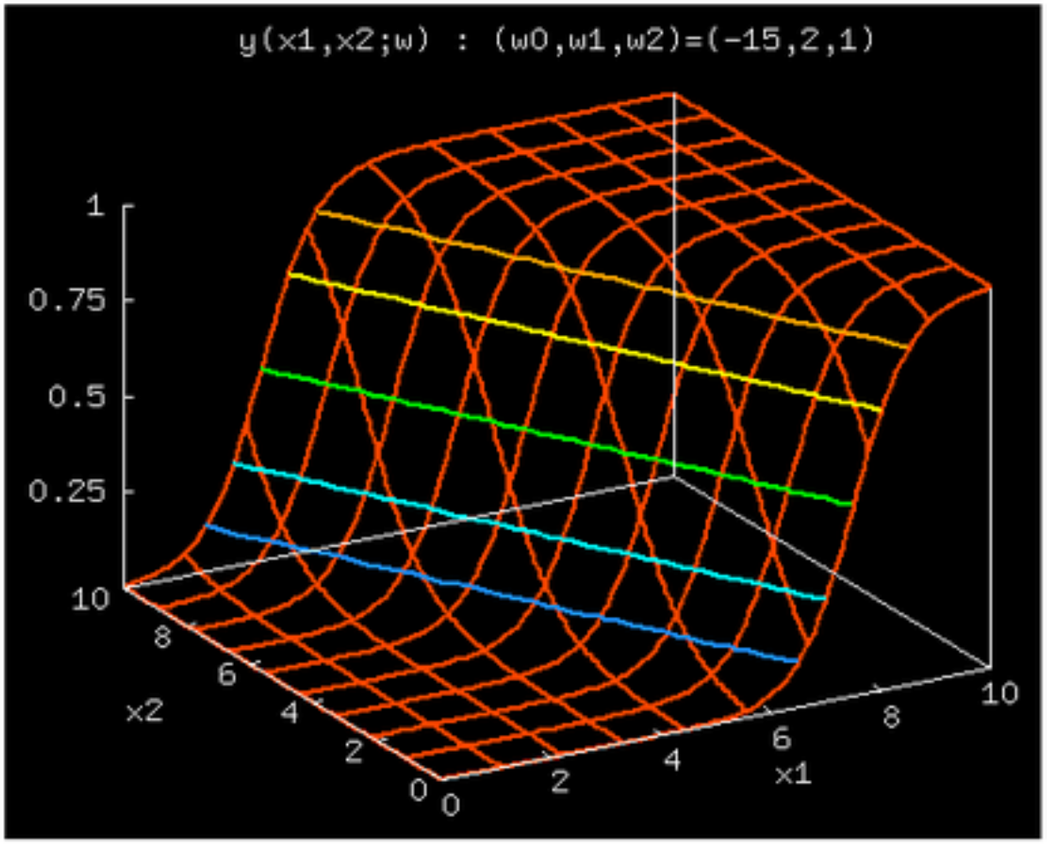
cd itp/neuron
gnuplot
load 'LDEMO'


w = (-15.0,1.5,1.0)
10
8
6
4
2
0
0
2
4
6
8
10

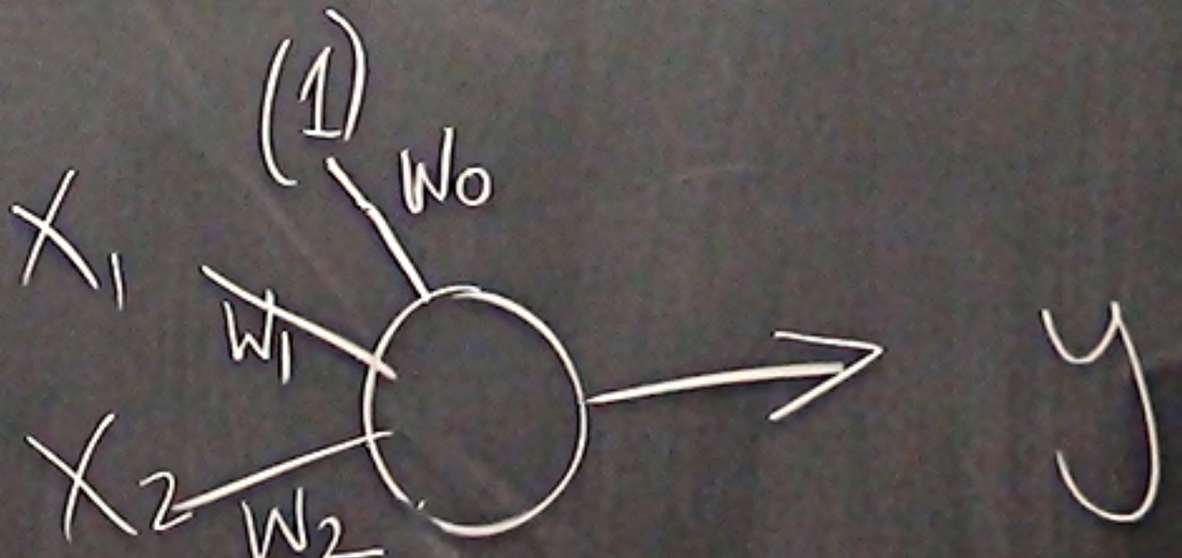
Q: What happens if all three weights are doubled?

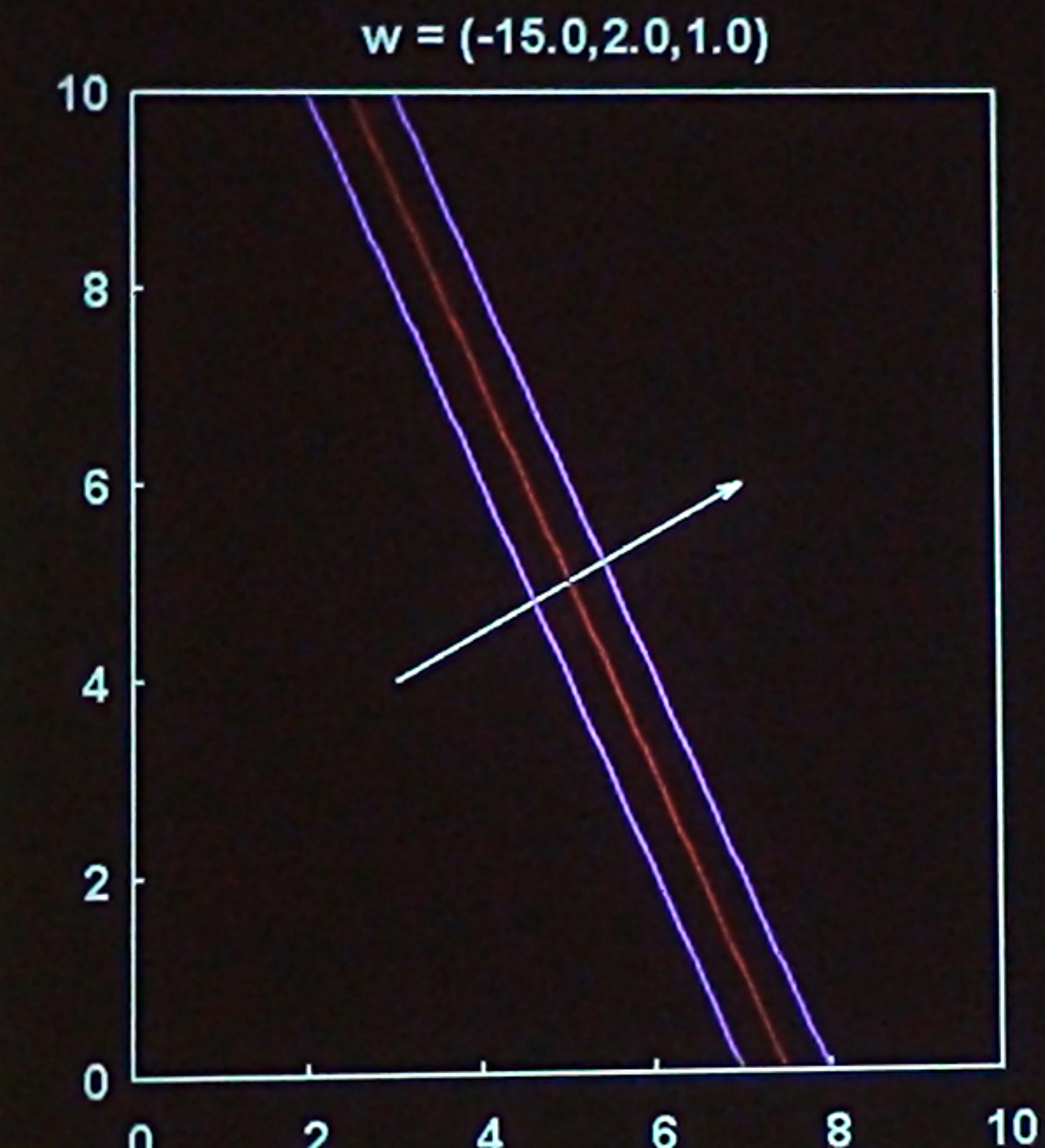






Single neuron




w = (-15.0,2.0,1.0)
10
8
6
4
2
0
0
2
4
6
8
10

"Learning"

Data $\left\{ \underline{x}^{(n)}, t^{(n)} \right\}_{n=1}^{N}$

input

target

"Learning"

Data $\{ \underline{x}^{(n)}, t^{(n)} \}_{n=1}^{N}$

input $\underline{x}^{(n)}$ ↓ target $t^{(n)}$ ↓

adjusting parameters (weights) s.t. $y(x^{(n)}; \underline{w})$

input target

$\{\underline{x}^{(n)}, t^{(n)}\}_{n=1}^{N}$

$y(x^{(n)}; \underline{w})$ is "close" to $t^{(n)}$, for all n.

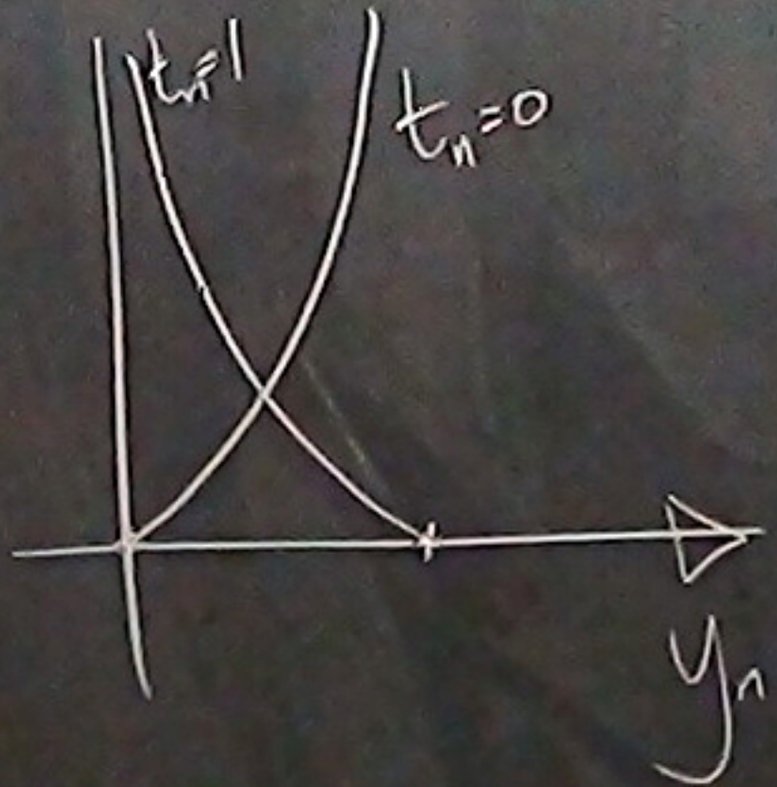
$$G(\underline{w}) = -\sum_{1}^{N} t_n \log y_n(x^{(n)}; w)$$

$$+ (1-t_n) \log (1-y_n)$$

$$\uparrow$$

$$y_n \equiv y(\underline{x}^{(n)}; \underline{w})$$

$$-\sum_{1}^{N} t_n \log y_n(x^{(n)}; w)$$



$$+ (1-t_n) \log (1-y_n)$$

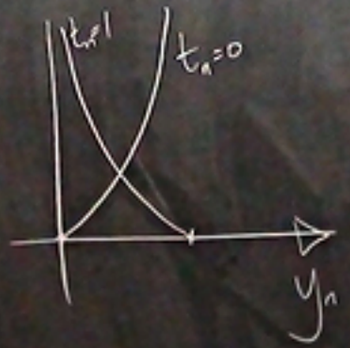
$$\uparrow$$

$$y_n \equiv y(\underline{x}^{(n)}; \underline{w})$$

G is the information of the data

$$G(\underline{w}) = -\sum_{1}^{N} t_n \log y_n(x^{(n)}; \underline{w}) + (1-t_n) \log (1-y_n)$$

$y_n \equiv y(\underline{x}^{(n)}; \underline{w})$



G is the information of the data

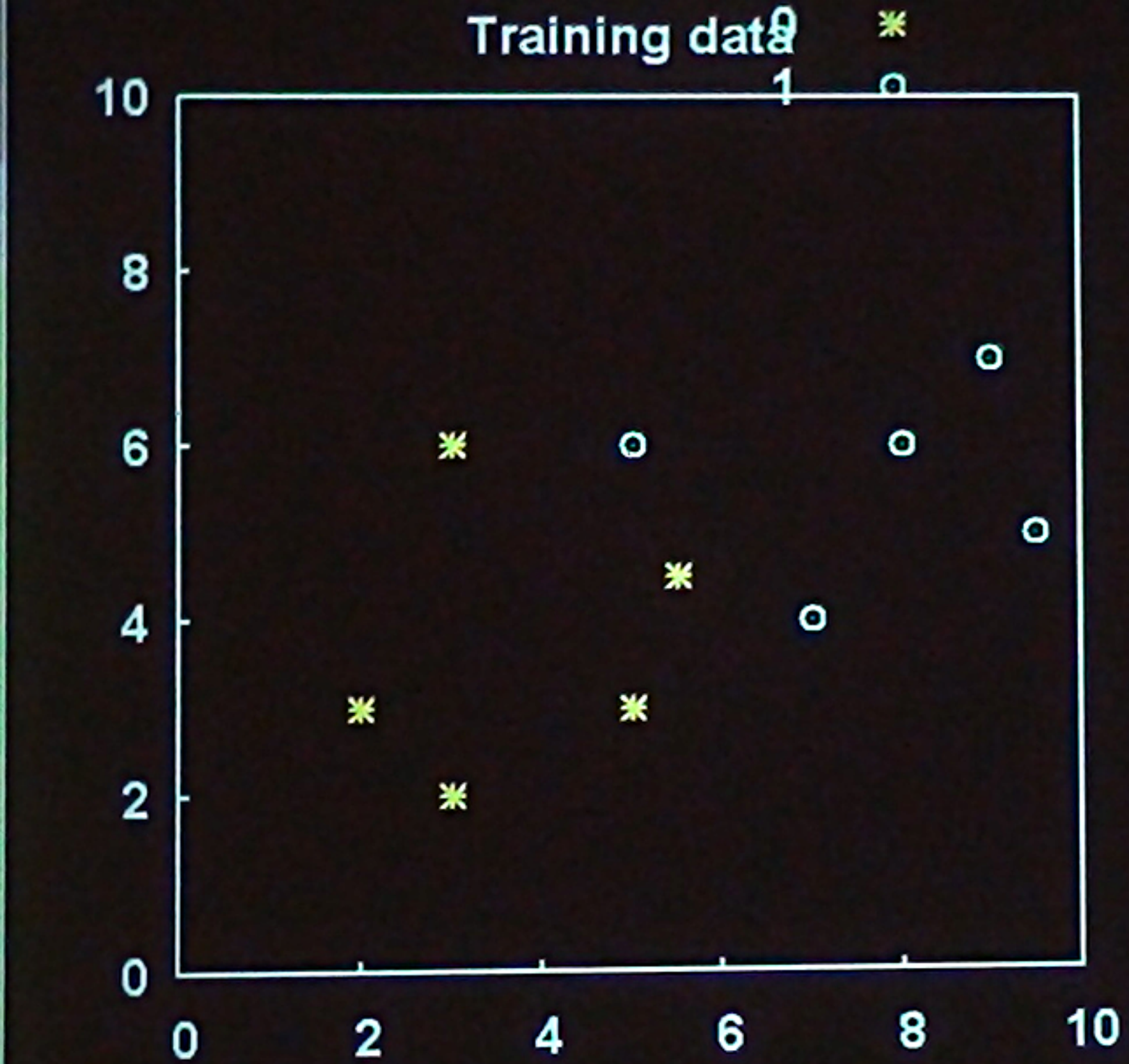
$$+ (1-t_n) \log (1-y_n)$$

$$y_n \equiv y(\underline{x}^{(n)}; \underline{w})$$

G is { the information content of (the data) $\{t^{(n)}\}_{n=1}^{N}$
the description length of the data }

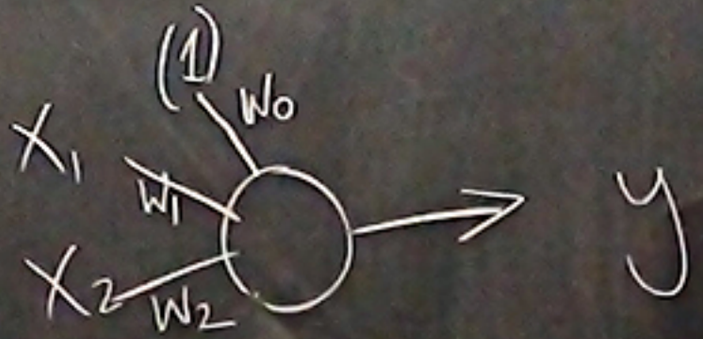
assuming

y the correct probability for t


Training data
0
1
10
8
6
4
2
0
0
2
4
6
8
10

Gradient of G $\quad g = \frac{\partial G}{\partial w}$ Gradient descent

Single neuron



Gradient descent

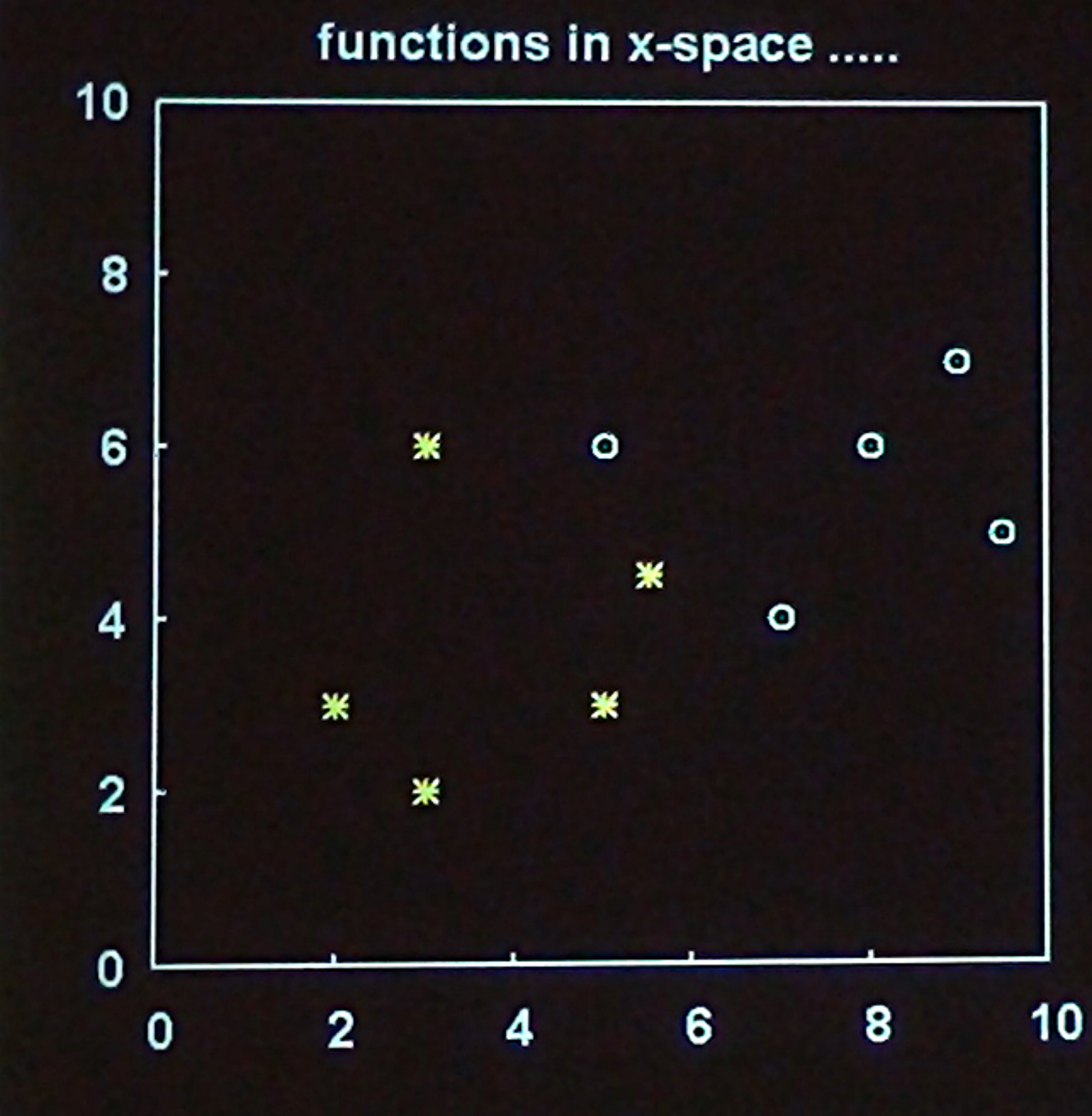
$$\Delta \underline{w} = -\eta \nabla f$$

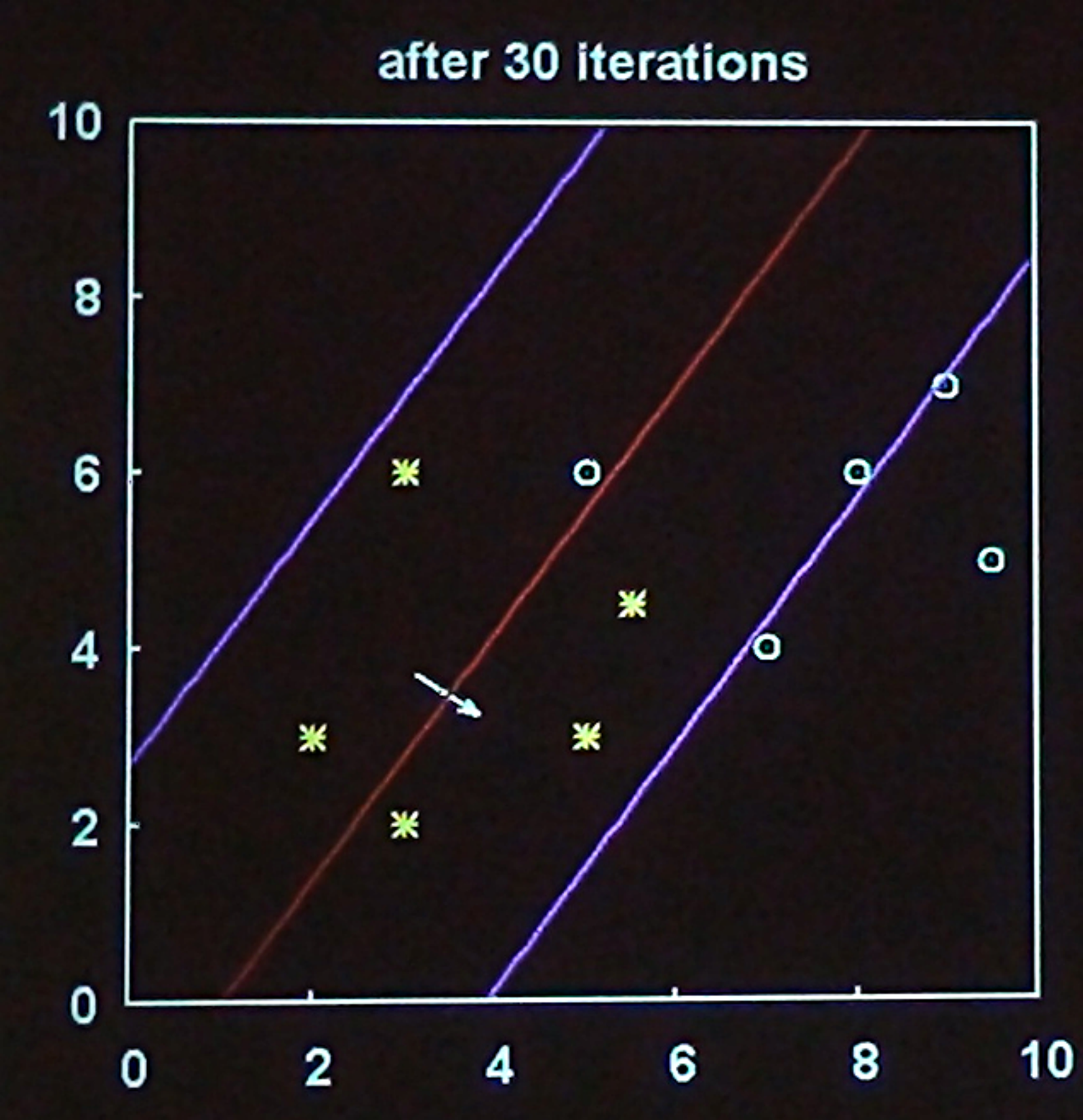
"learning rate"

"learning rate"

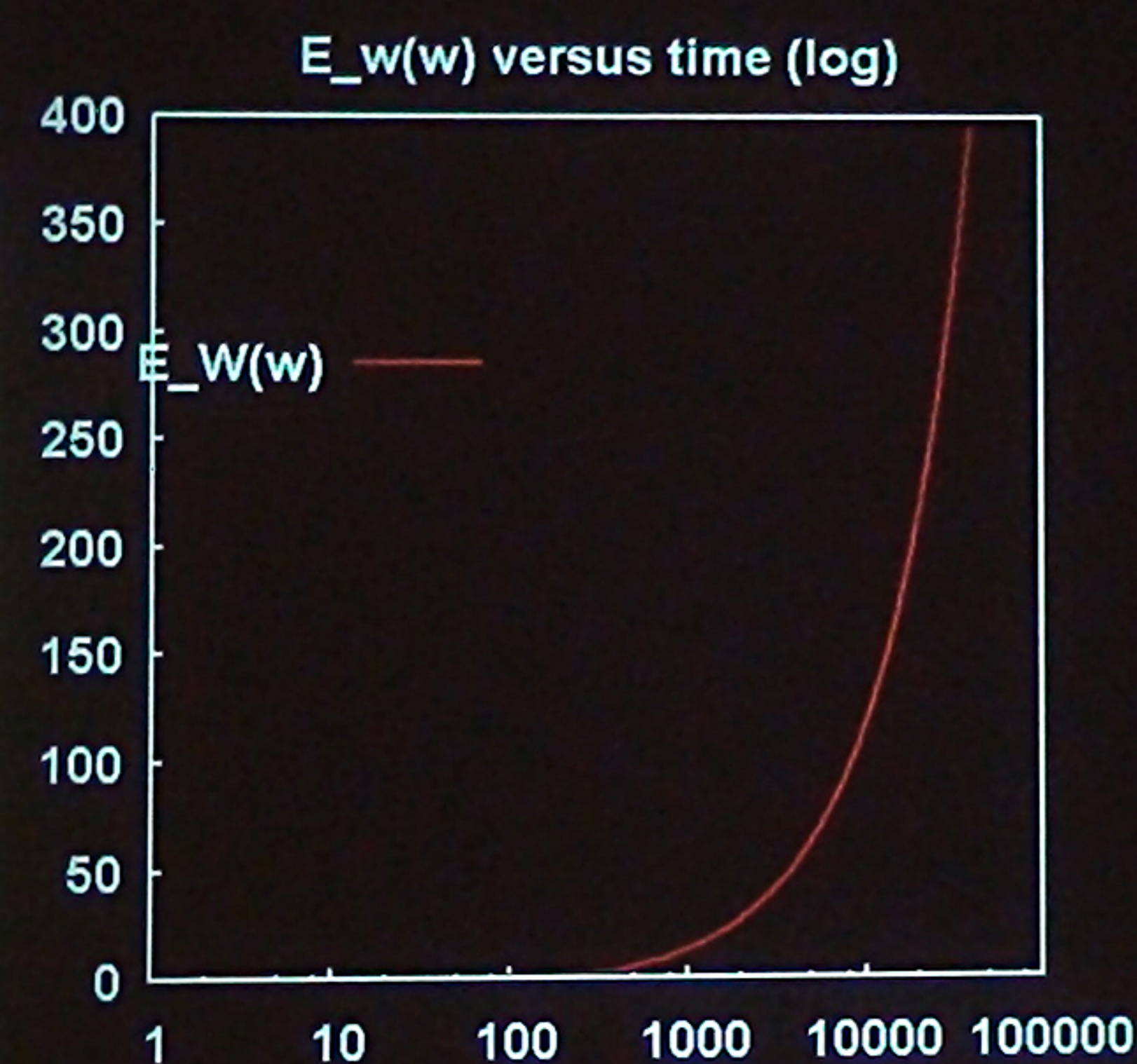
$$g = \sum_n \underbrace{(t_n - y_n)}_{\text{error}} x^{(n)}$$

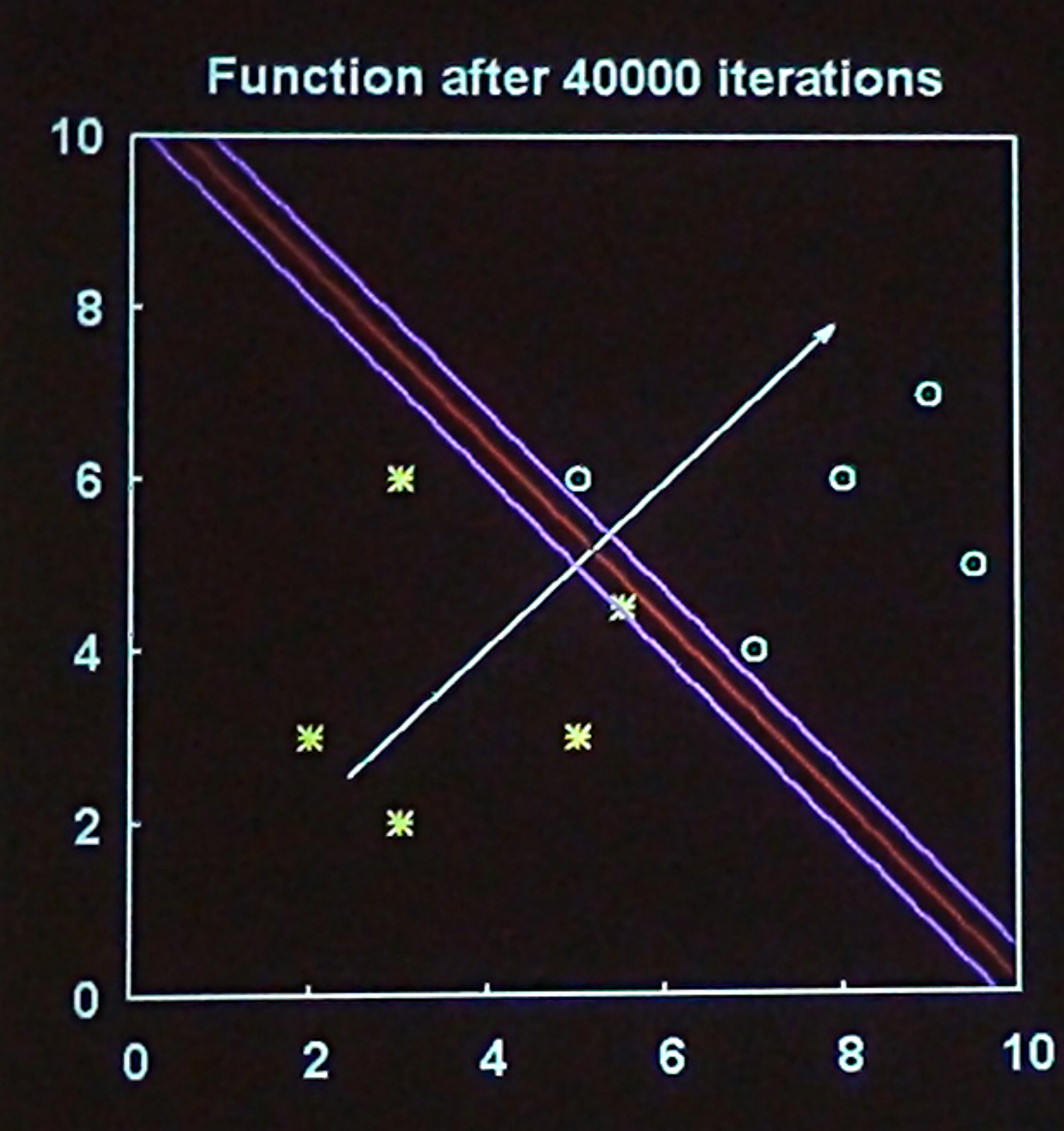
differentiation $\longleftrightarrow$ "backpropagation"


functions in x-space .....
10
8
6
4
2
0
0
2
4
6
8
10


after 30 iterations
10
8
6
4
2
0
0
2
4
6
8
10

after 10000 iterations
10
8
6
4
2
0
0
2
4
6
8
10

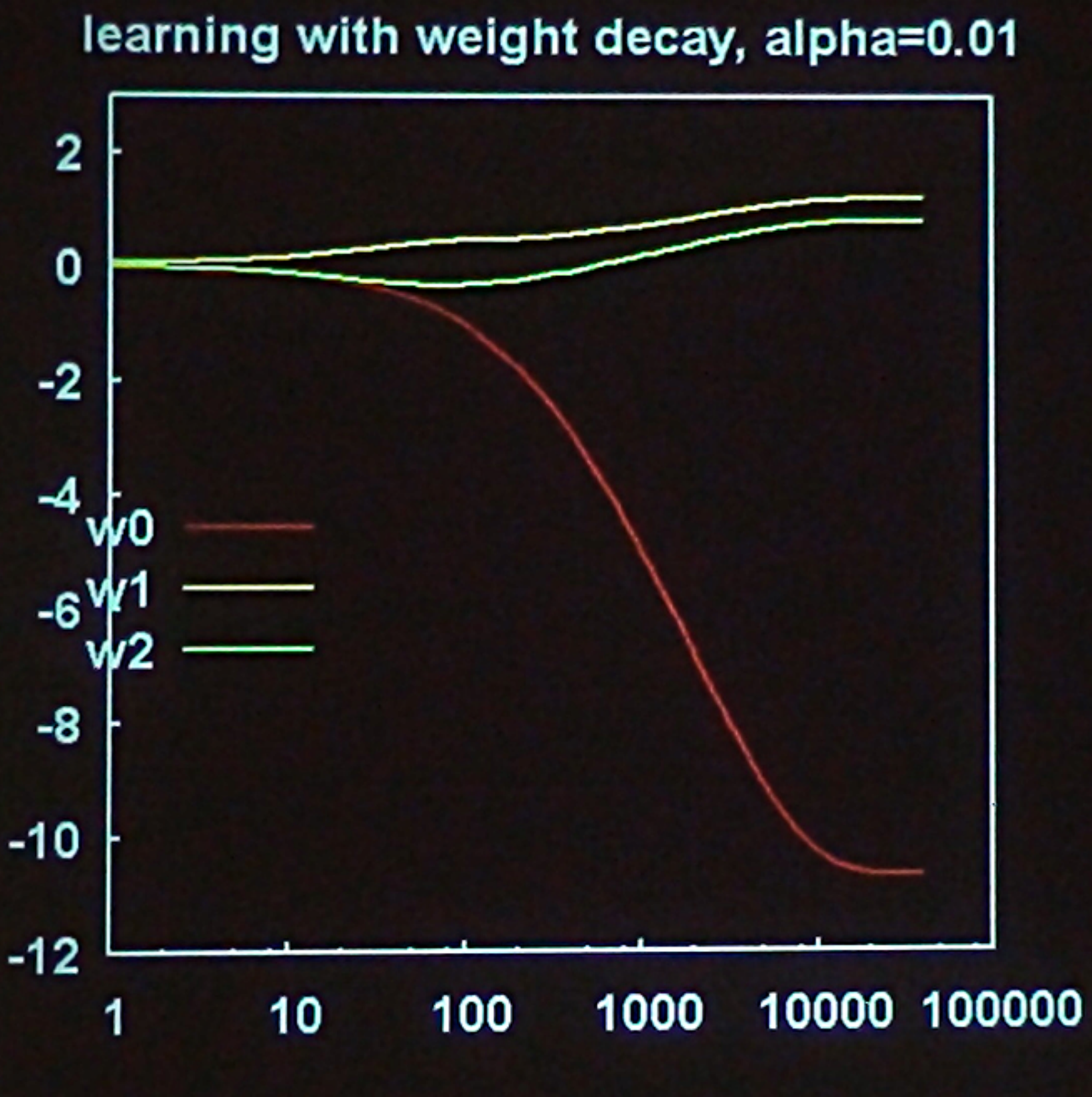

E_w(w) versus time (log)
E_W(w)
400
350
300
250
200
150
100
50
0
1
10
100
1000
10000
100000

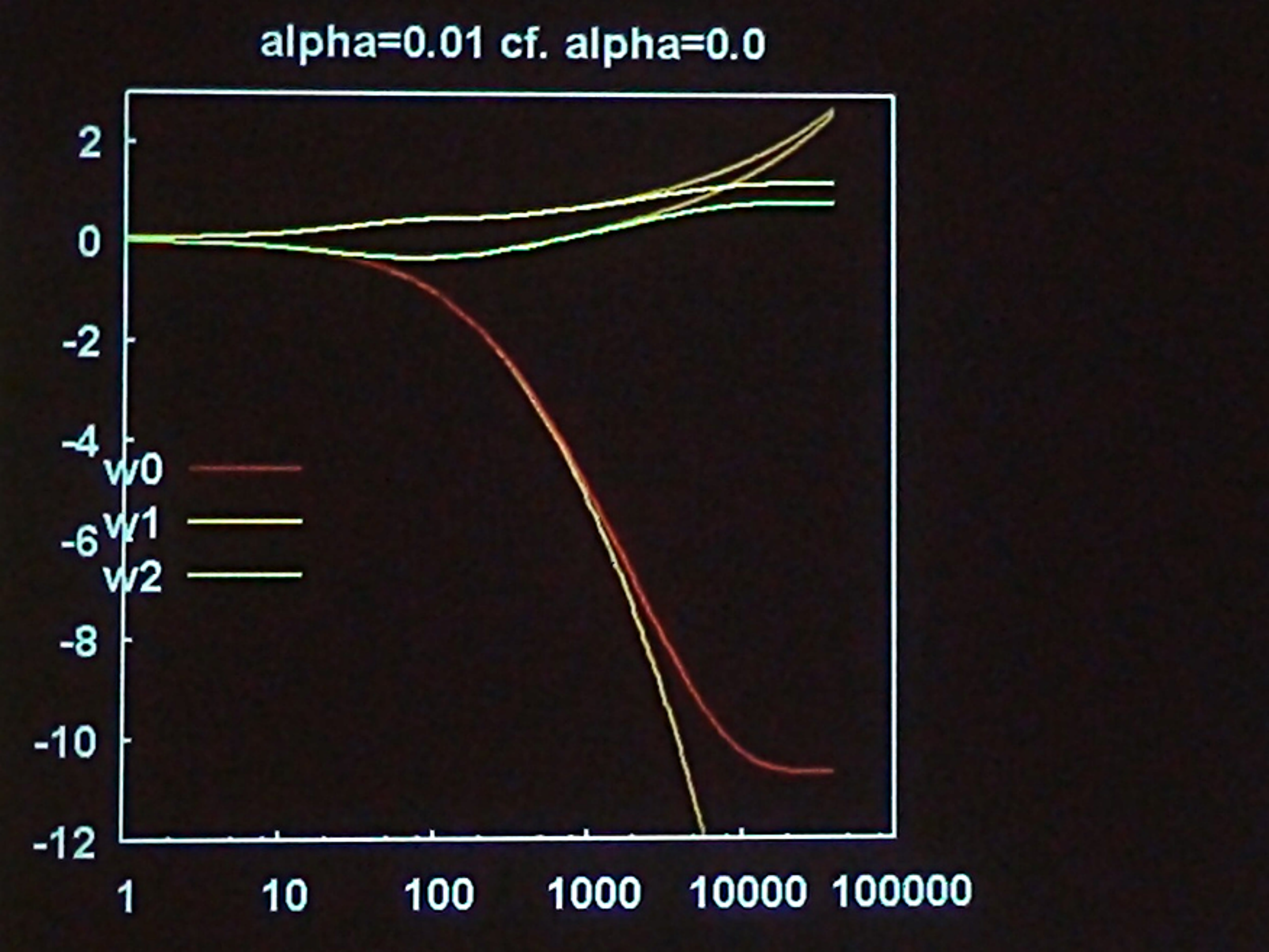

Function after 40000 iterations
10
8
6
4
2
0
0
2
4
6
8
10

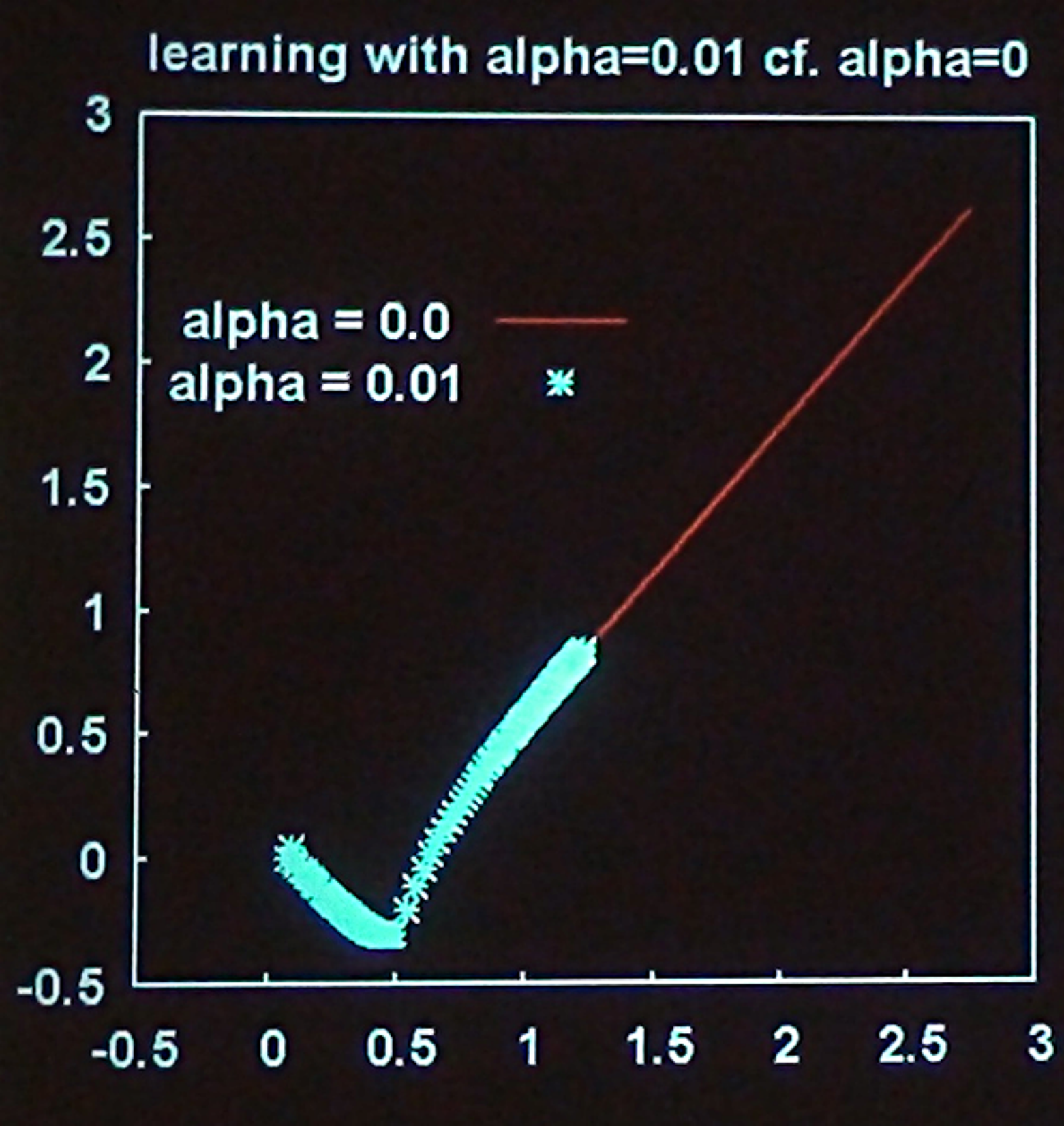
"Learning" with regularization
or "weight decay"

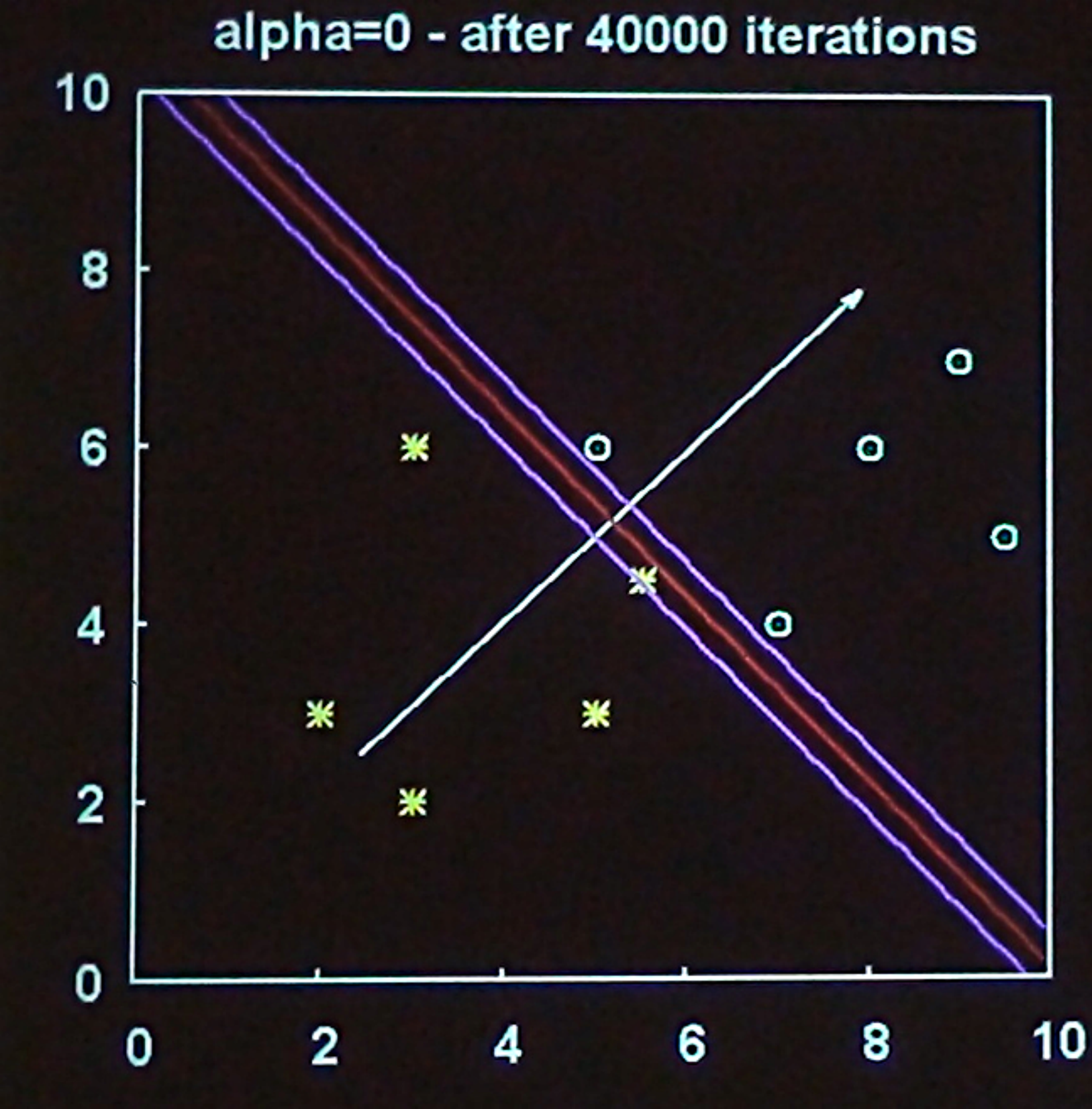
$$M(\underline{w}) = C(\underline{w}) + \alpha E_w(\underline{w})$$

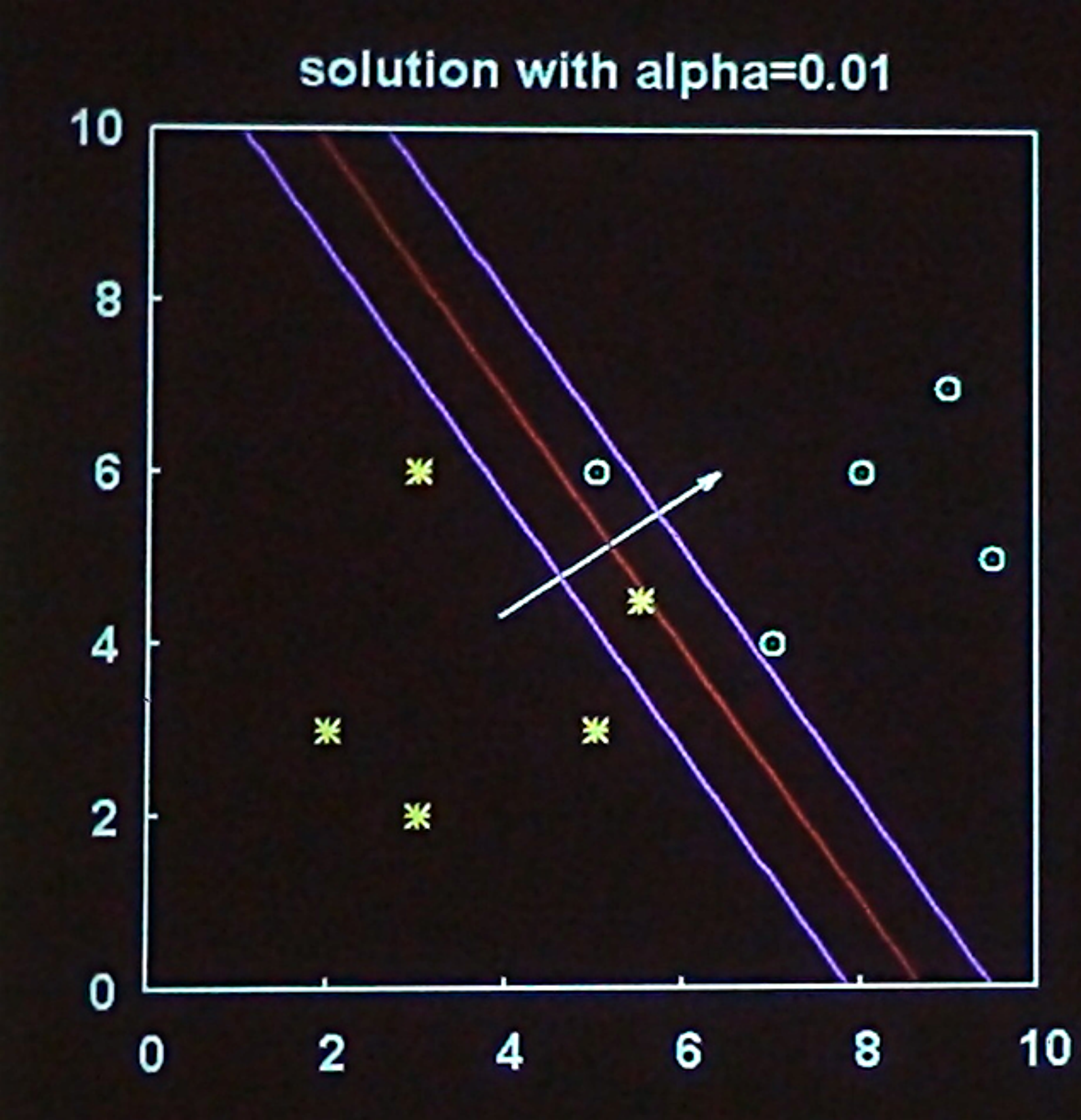
$$E_w(w) = \frac{1}{2} \sum_{k=1}^{K} w_k^2$$


learning with weight decay, alpha=0.01
2
0
-2
-4
-6
-8
-10
-12
w0
w1
w2
1
10
100
1000
10000
100000


alpha=0.01 cf. alpha=0.0
2
0
-2
-4
-6
-8
-10
-12
w0
w1
w2
1
10
100
1000
10000
100000

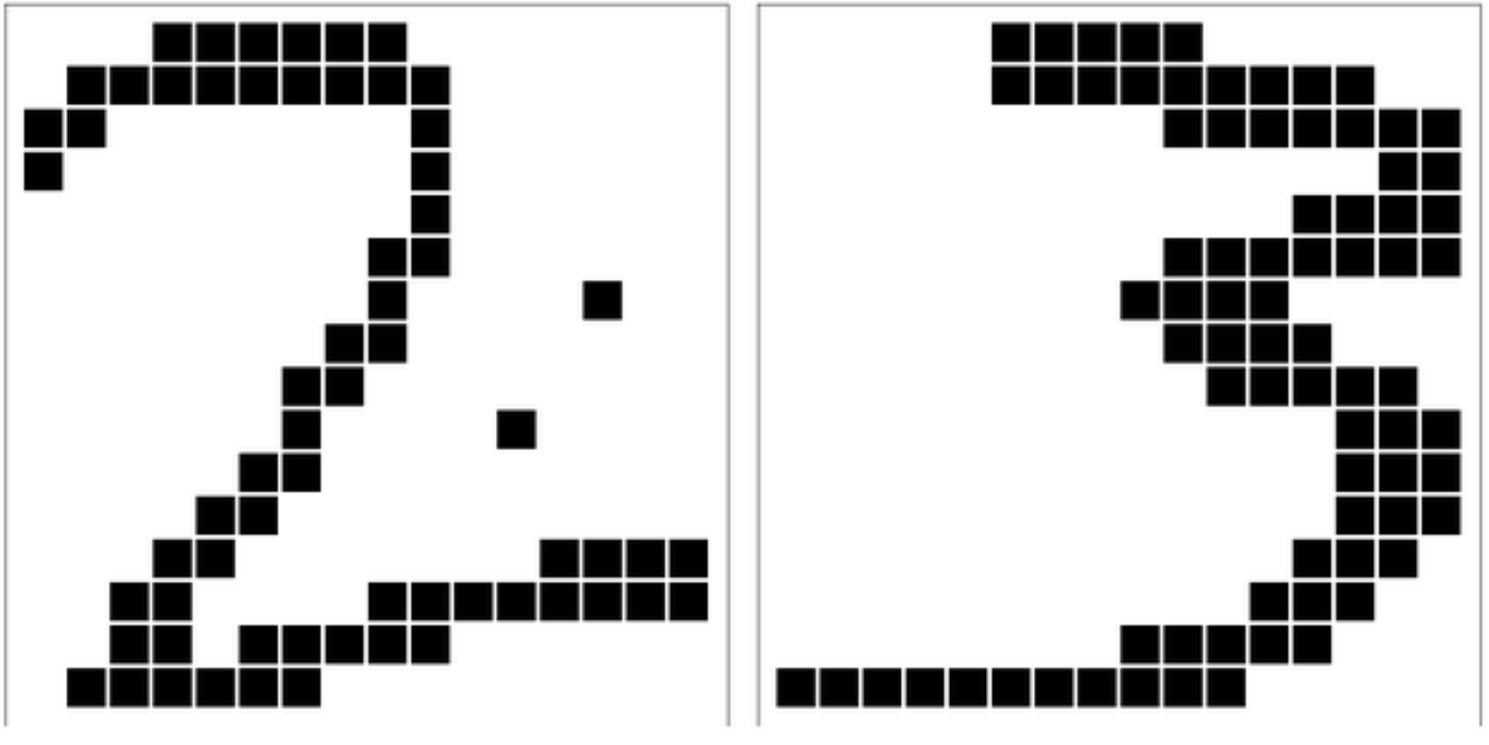

learning with alpha=0.01 cf. alpha=0
alpha = 0.0
alpha = 0.01
3
2.5
2
1.5
1
0.5
0
-0.5
-0.5
0
0.5
1
1.5
2
2.5
3


alpha=0 - after 40000 iterations
10
8
6
4
2
0
0
2
4
6
8
10


solution with alpha=0.01
10
8
6
4
2
0
0
2
4
6
8
10

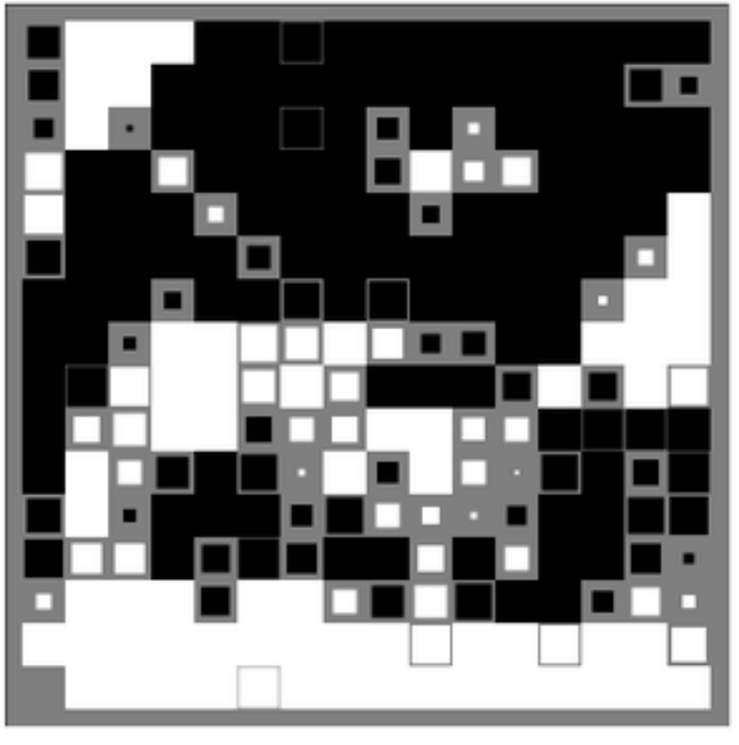
Single Neuron for digit classification

Trained to distinguish 2s from 3s



Single Neuron for digit classification

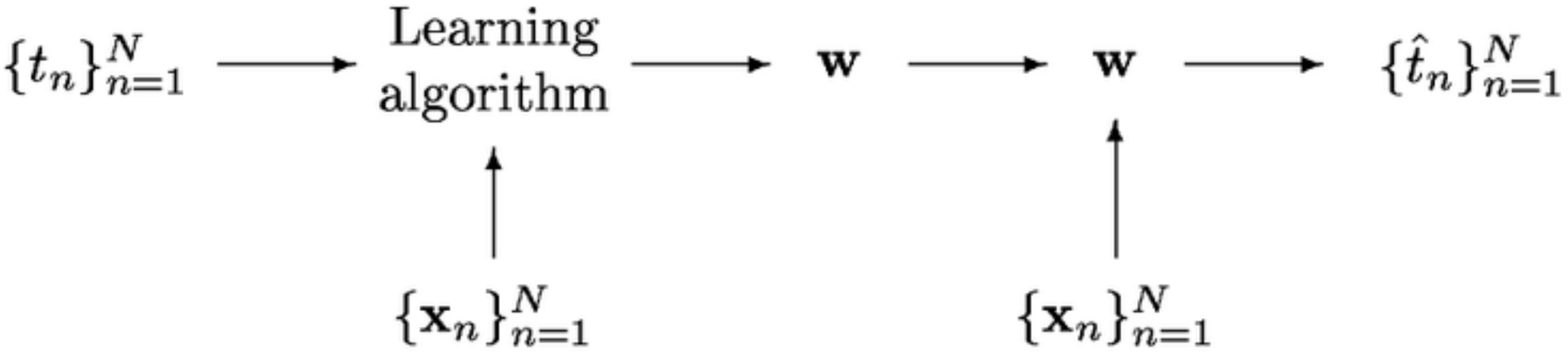
Trained to distinguish 2s from 3s



Error rate: 10%

Capacity of a neuron

Learning as communication



- How many bits can a neuron learn?

> A single neuron with K inputs can almost certainly memorize up to $N = 2K$ random binary labels. (Chapter 40)

So the capacity of a neuron is two bits per connection

"Learning" with regularization or "weight decay"

$$M(\underline{w}) = C(\underline{w}) + \alpha E_W(\underline{w})$$

$$E_W(\underline{w}) = \frac{1}{2} \sum_{k=1}^{K} w_k^2$$

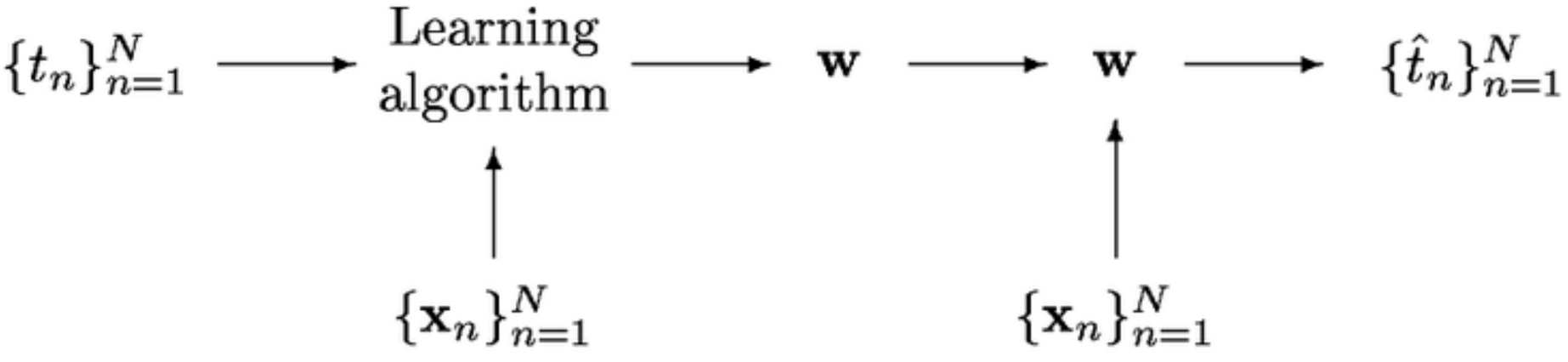
$E_W(\underline{w})$: regularizer

α: regularization constant

"weight decay rate"

Capacity of a neuron

Learning as communication

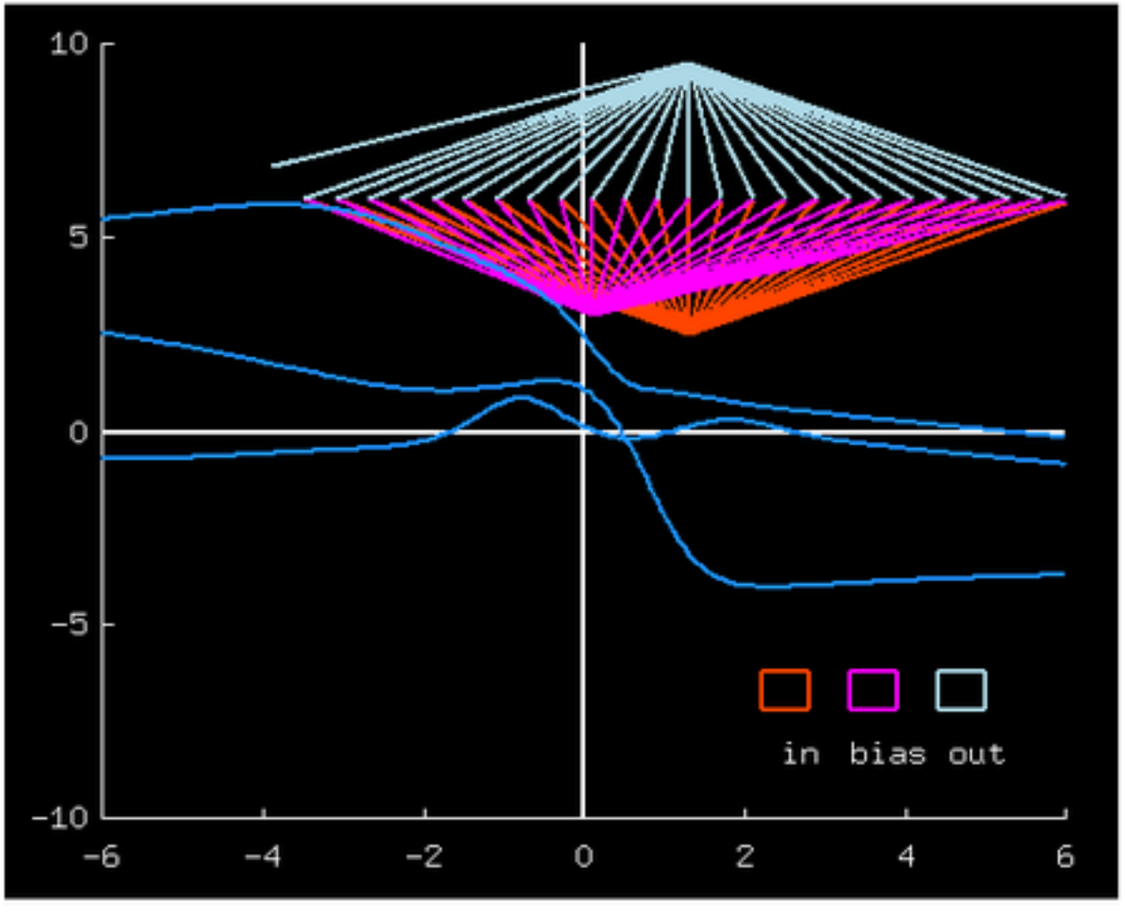


- How many bits can a neuron learn?

> A single neuron with K inputs can almost certainly memorize up to $N = 2K$ random binary labels. (Chapter 40)

So the capacity of a neuron is two bits per connection

Multi-layer network for regression



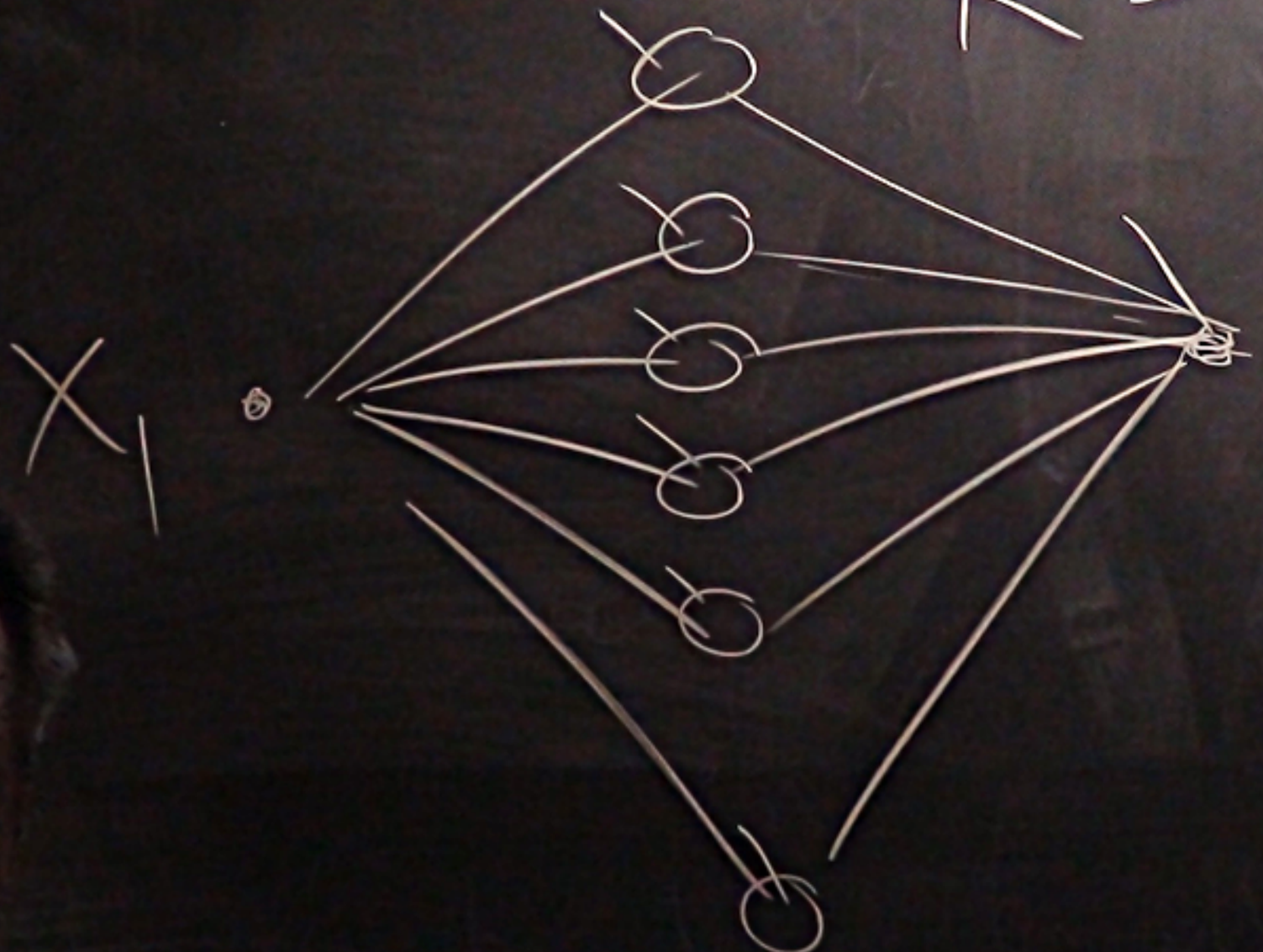
cd itp/neuron/net

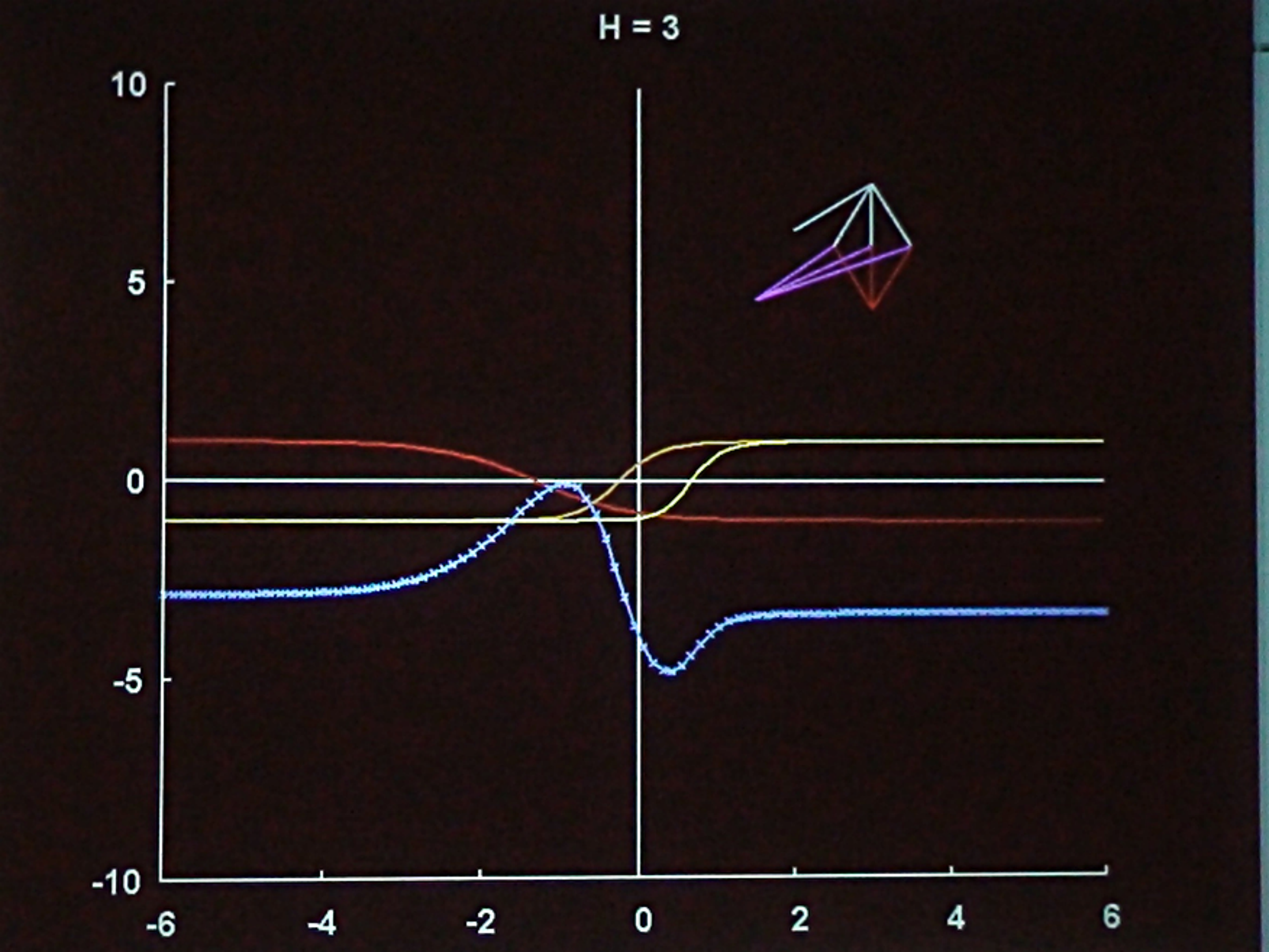
octave

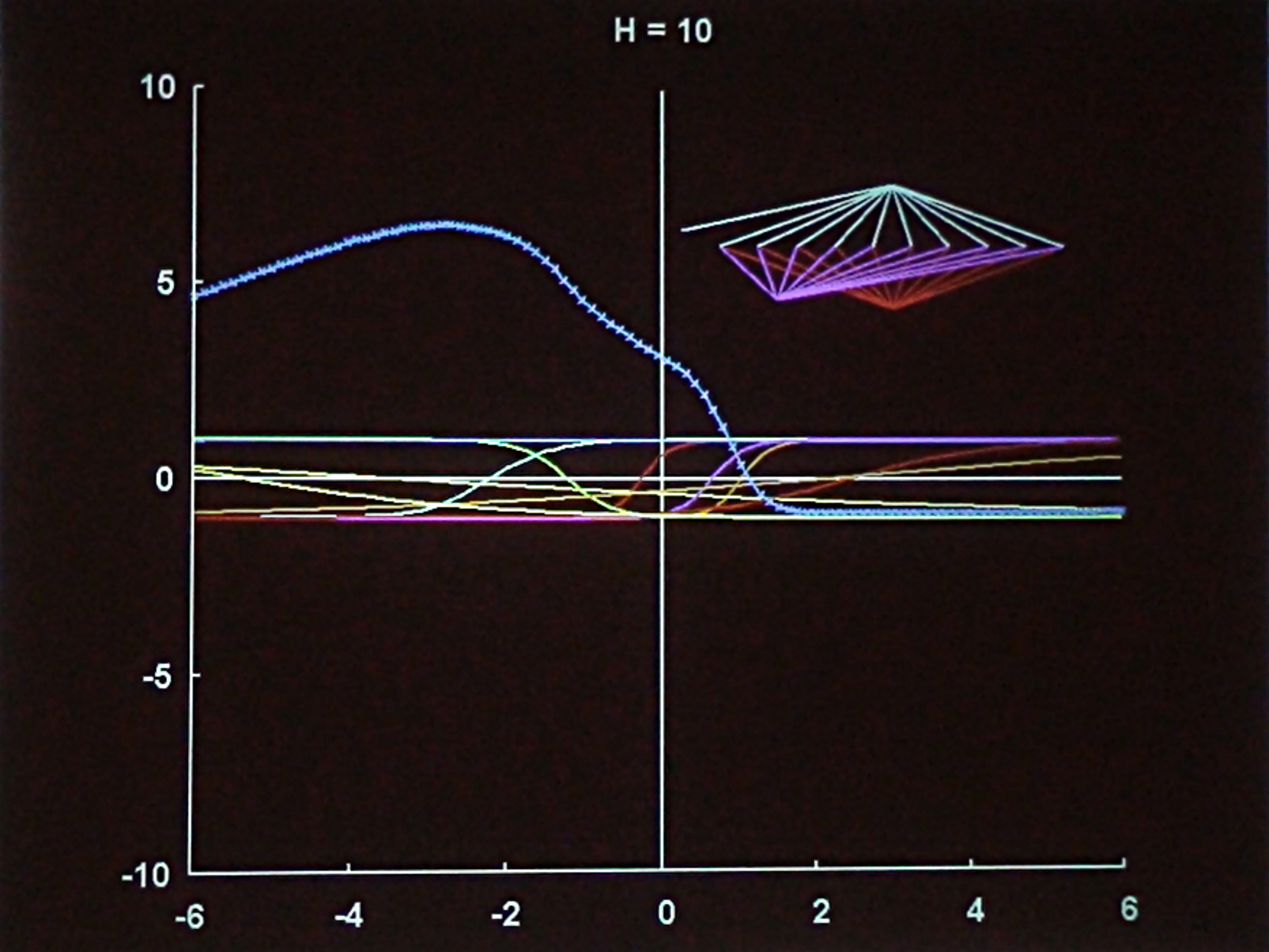
DEMO

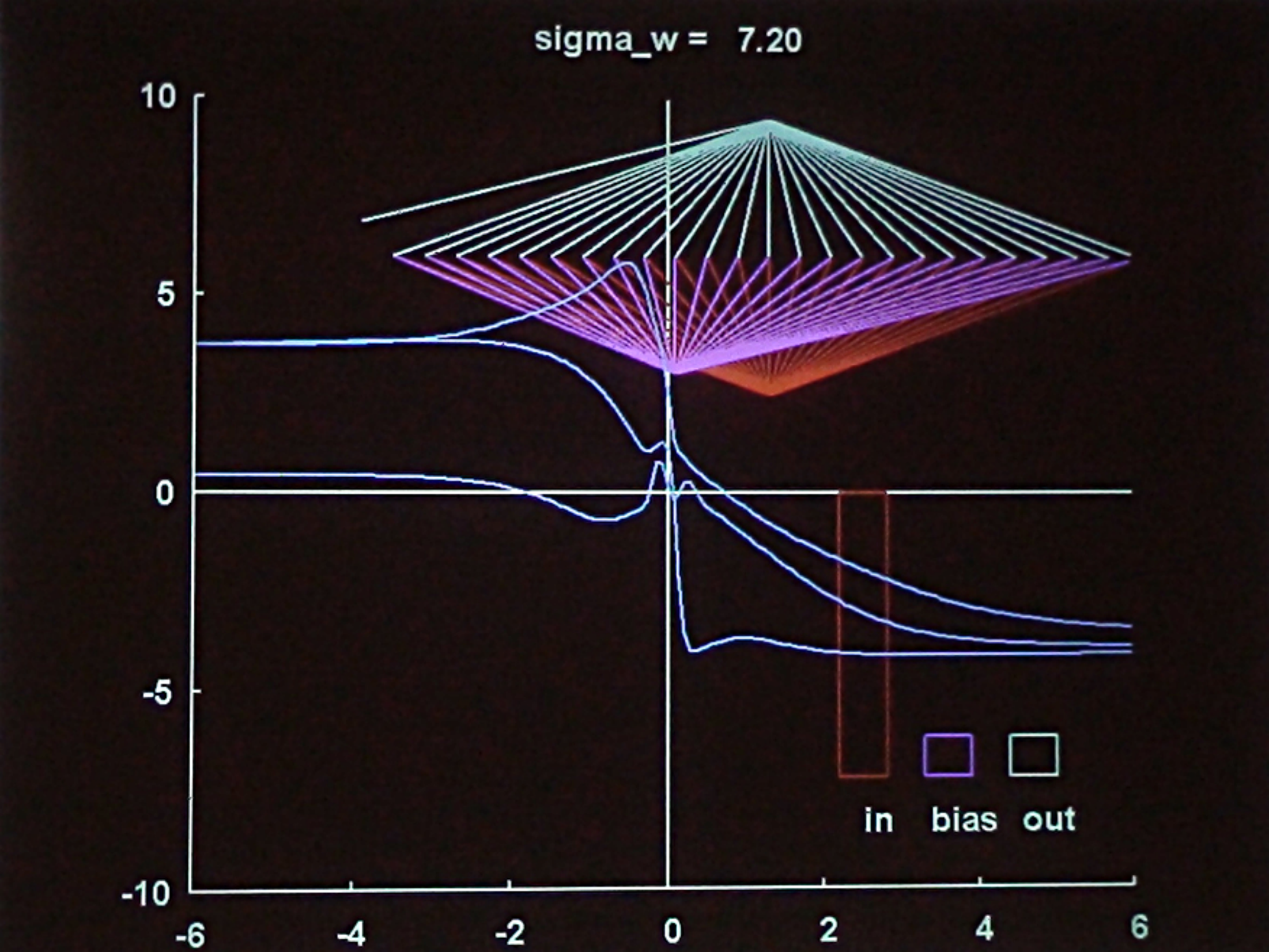
H

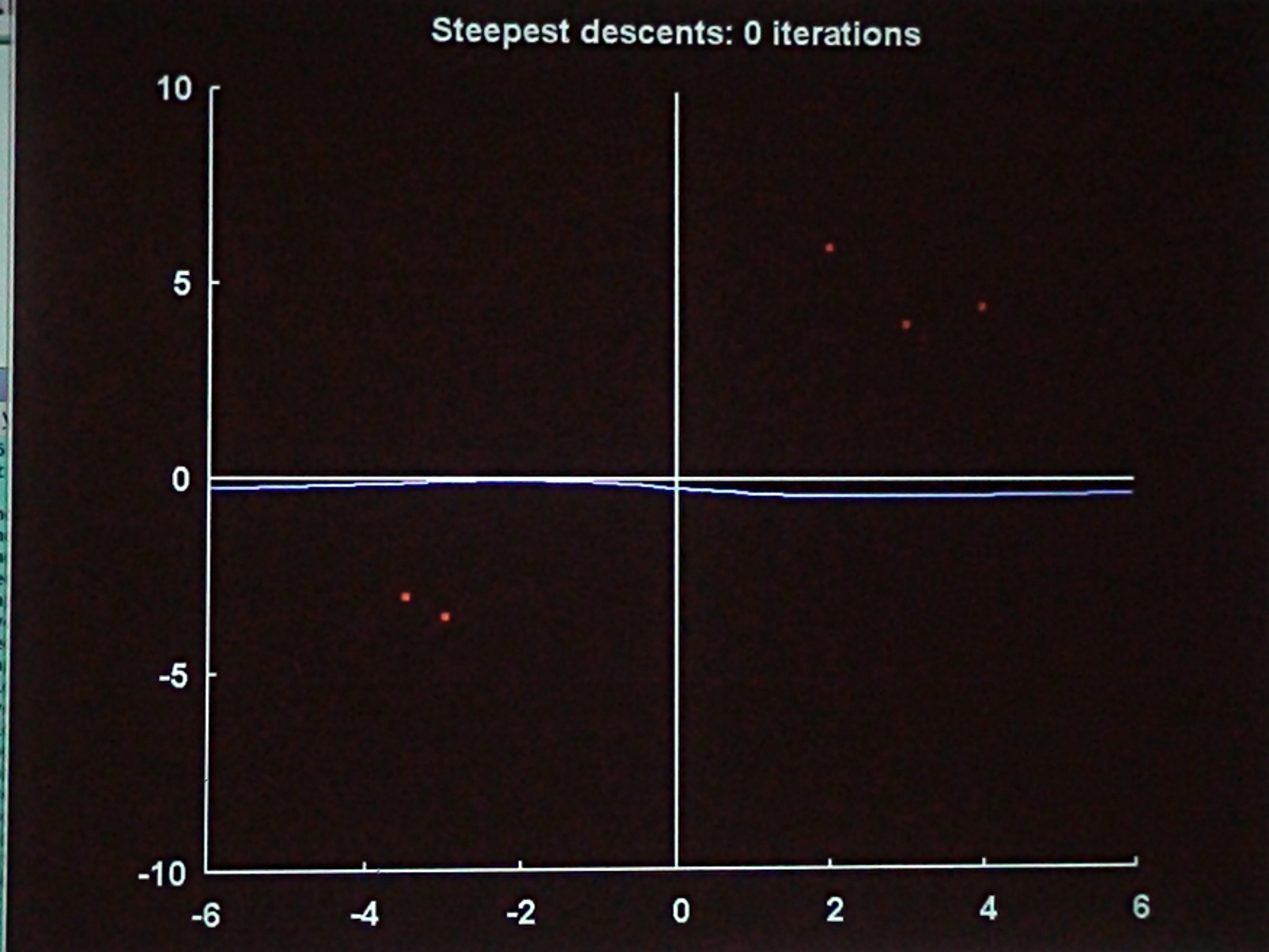
$K \simeq 3H + 1$

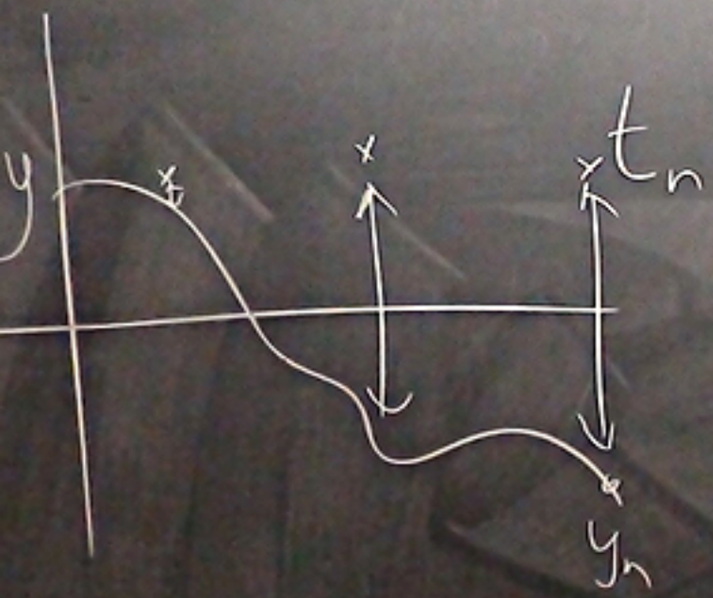



H = 3
10
5
0
-5
-10
-6
-4
-2
0
2
4
6


H = 10
10
5
0
-5
-10
-6
-4
-2
0
2
4
6

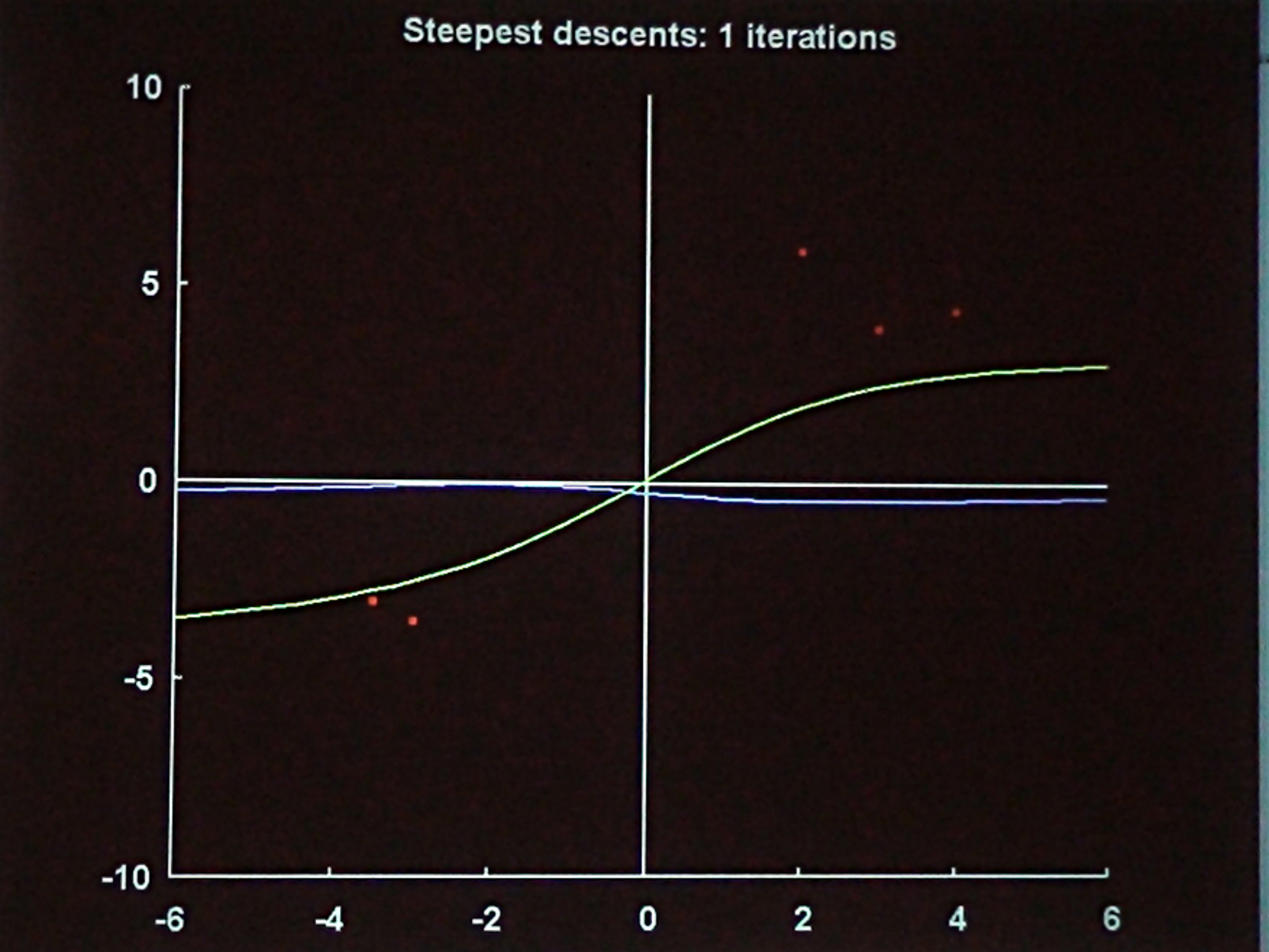

sigma_w = 7.20
10
5
0
-5
-10
-6
-4
-2
0
2
4
6
in
bias
out

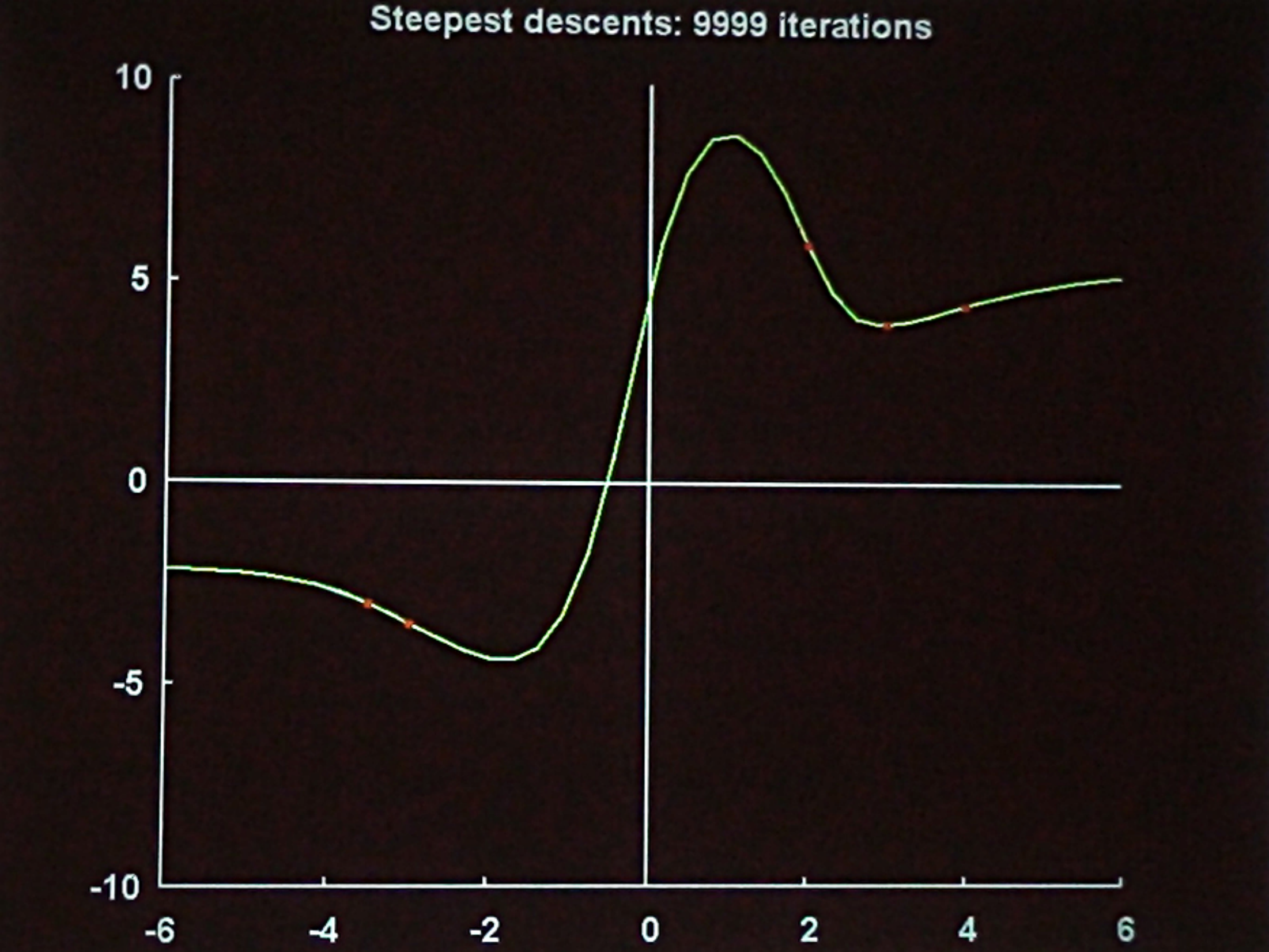

Steepest descents: 0 iterations
10
5
0
-5
-10
-6
-4
-2
0
2
4
6



$$E_D(\underline{w}) = \sum \frac{1}{2}(t_n - y_n(\underline{w}))^2$$

$$\underline{g} = \frac{\partial E_D}{\partial \underline{w}}$$

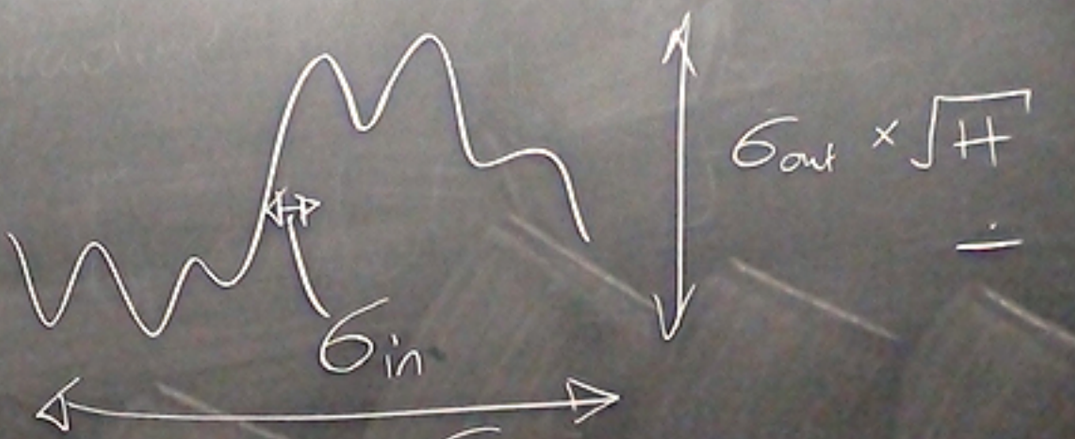

Steepest descents: 1 iterations
10
5
0
-5
-10
-6
-4
-2
0
2
4
6


Steepest descents: 9999 iterations
10
5
0
-5
-10
-6
-4
-2
0
2
4
6

Regularizer

$$\alpha E_W$$

$$\sum_{c=1}^{3} \alpha_c E_W^{(c)}$$

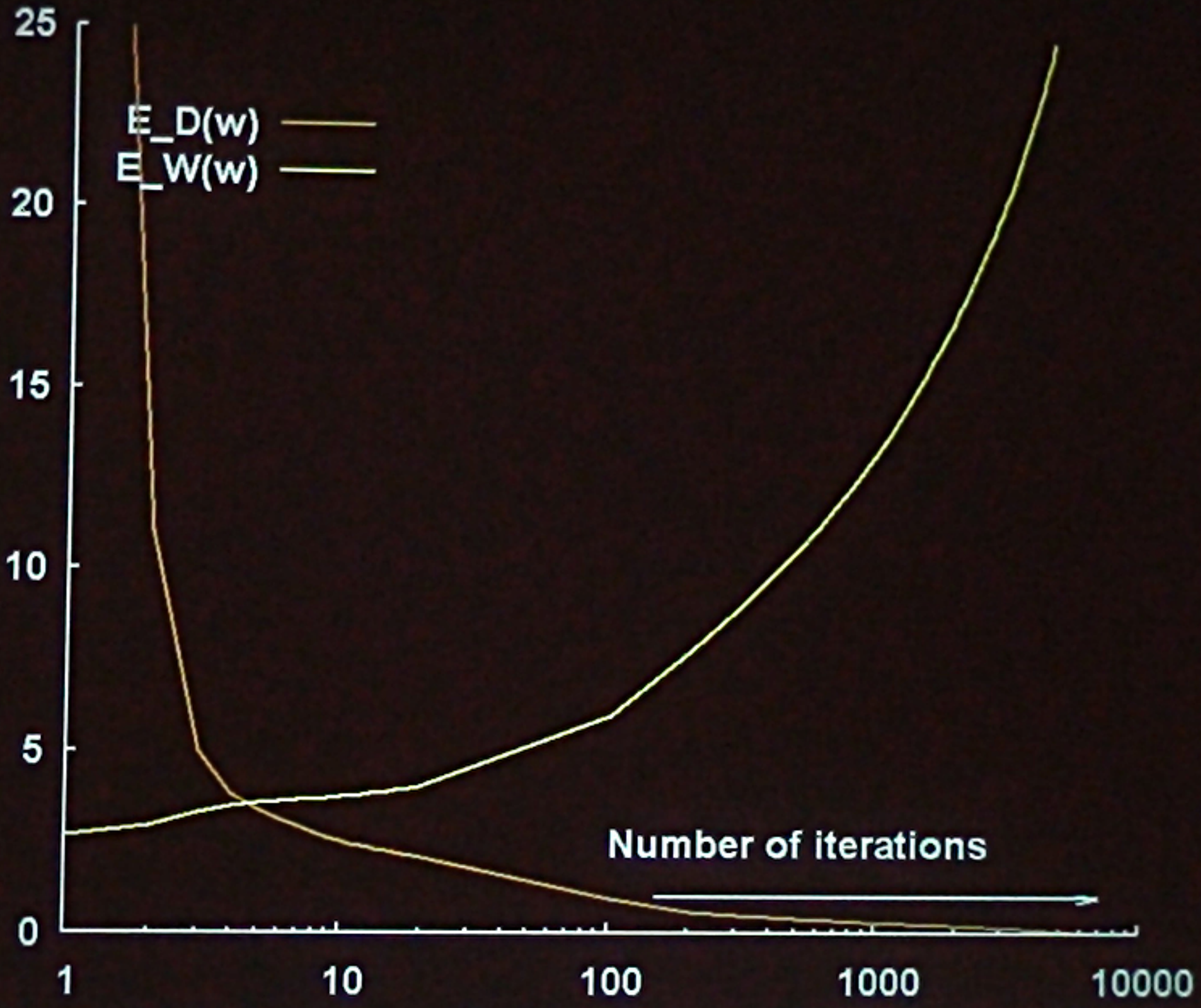


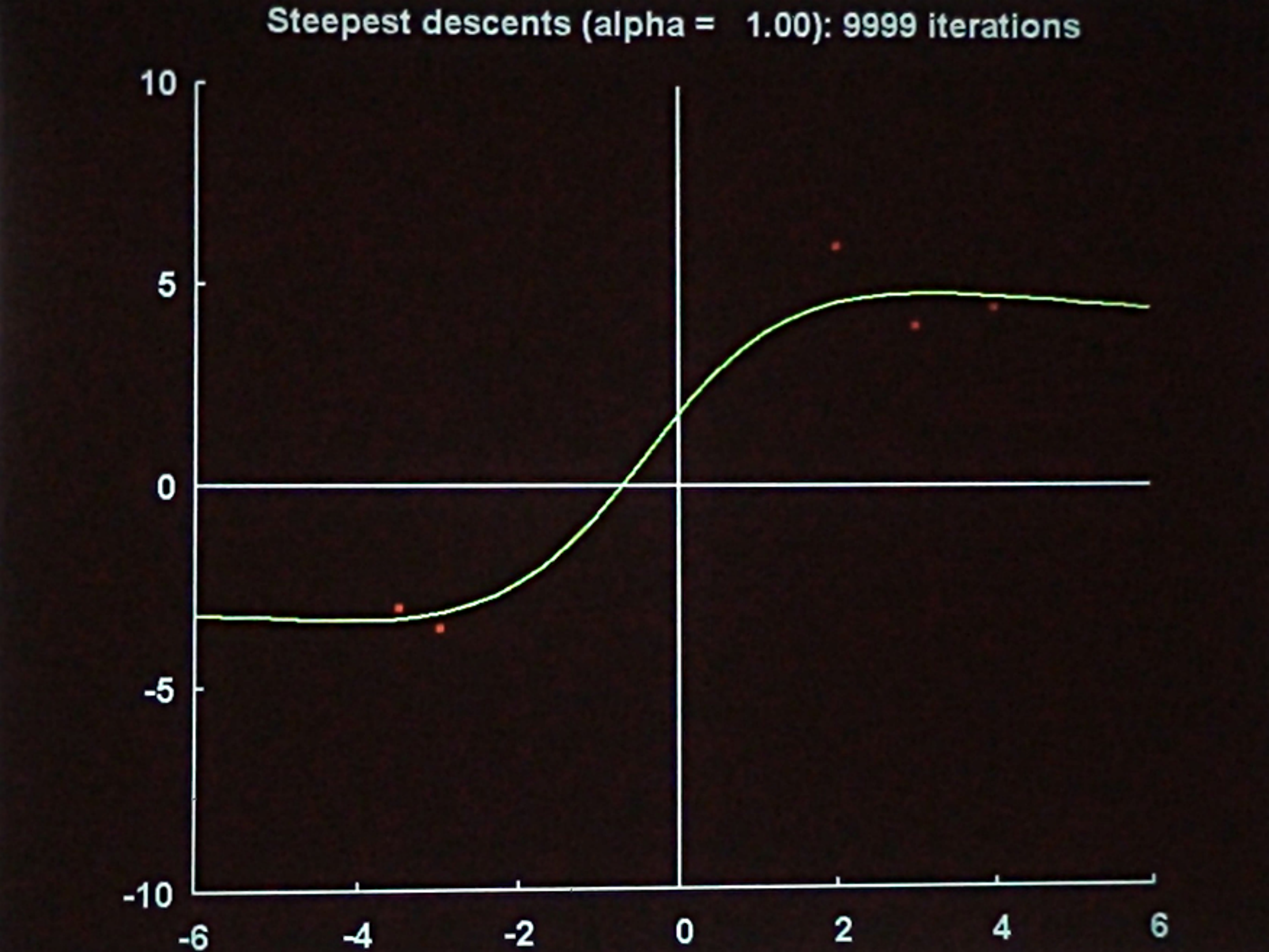
$$\sim \frac{\sigma_{bias}}{\sigma_{in}}$$

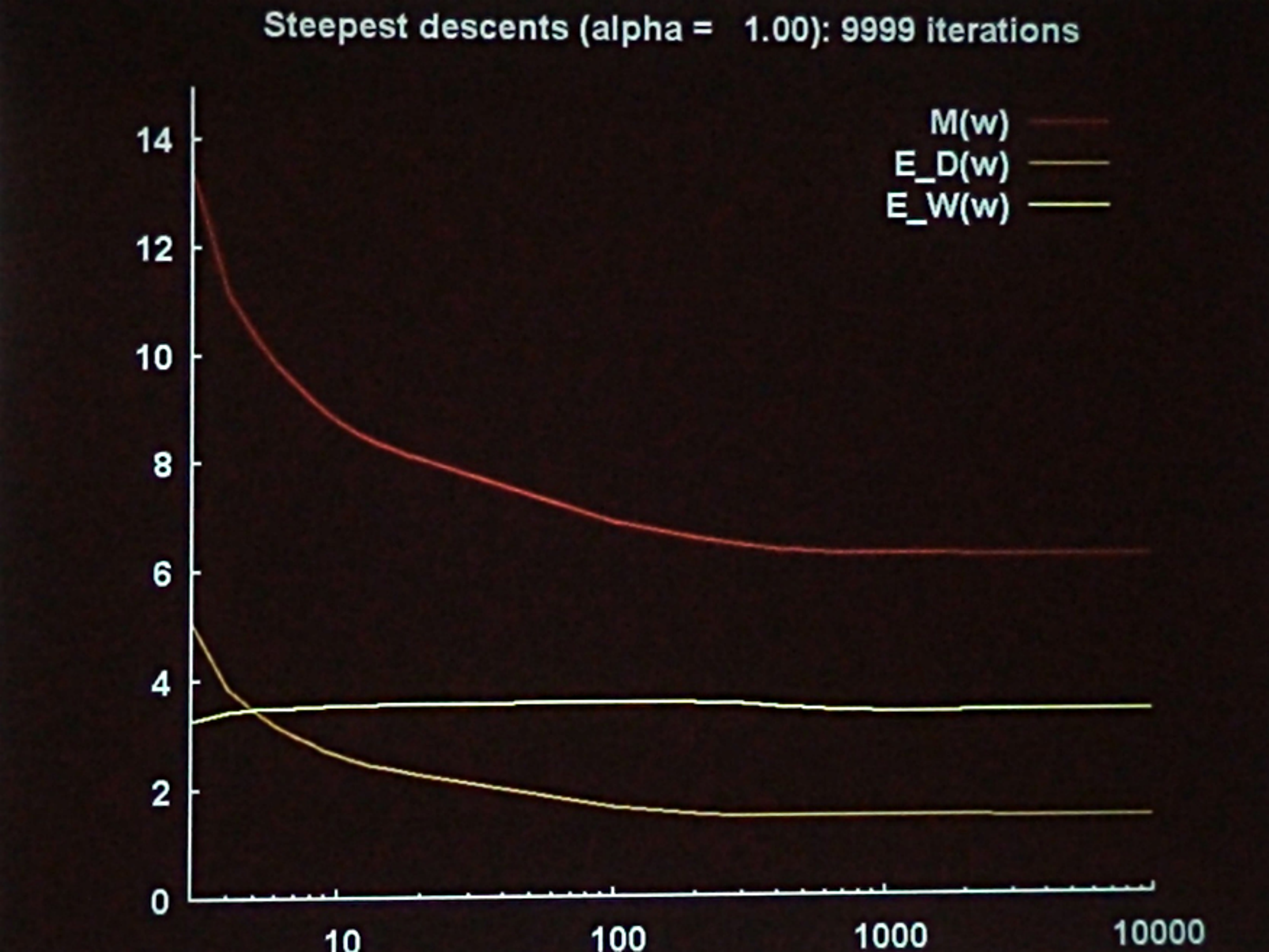
$$E_W^{(1)} = \frac{1}{2} \sum_{\text{biases}} W_k^2$$

$$E_W^{(2)} = \frac{1}{2} \sum_{\text{inputs}} W_k^2$$

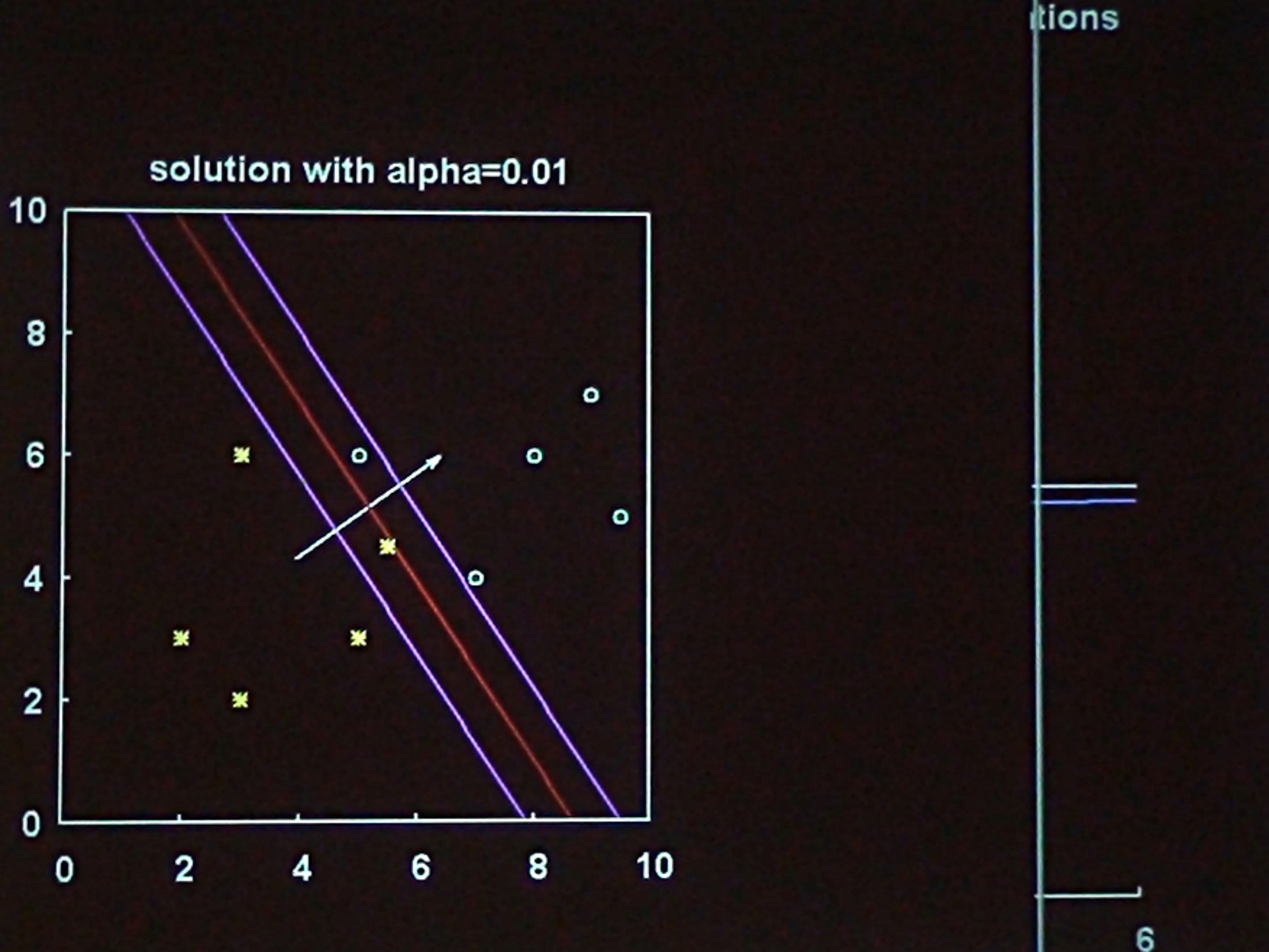
$$E_W^{(3)} = \frac{1}{2} \sum_{\text{outputs}} W_k^2$$

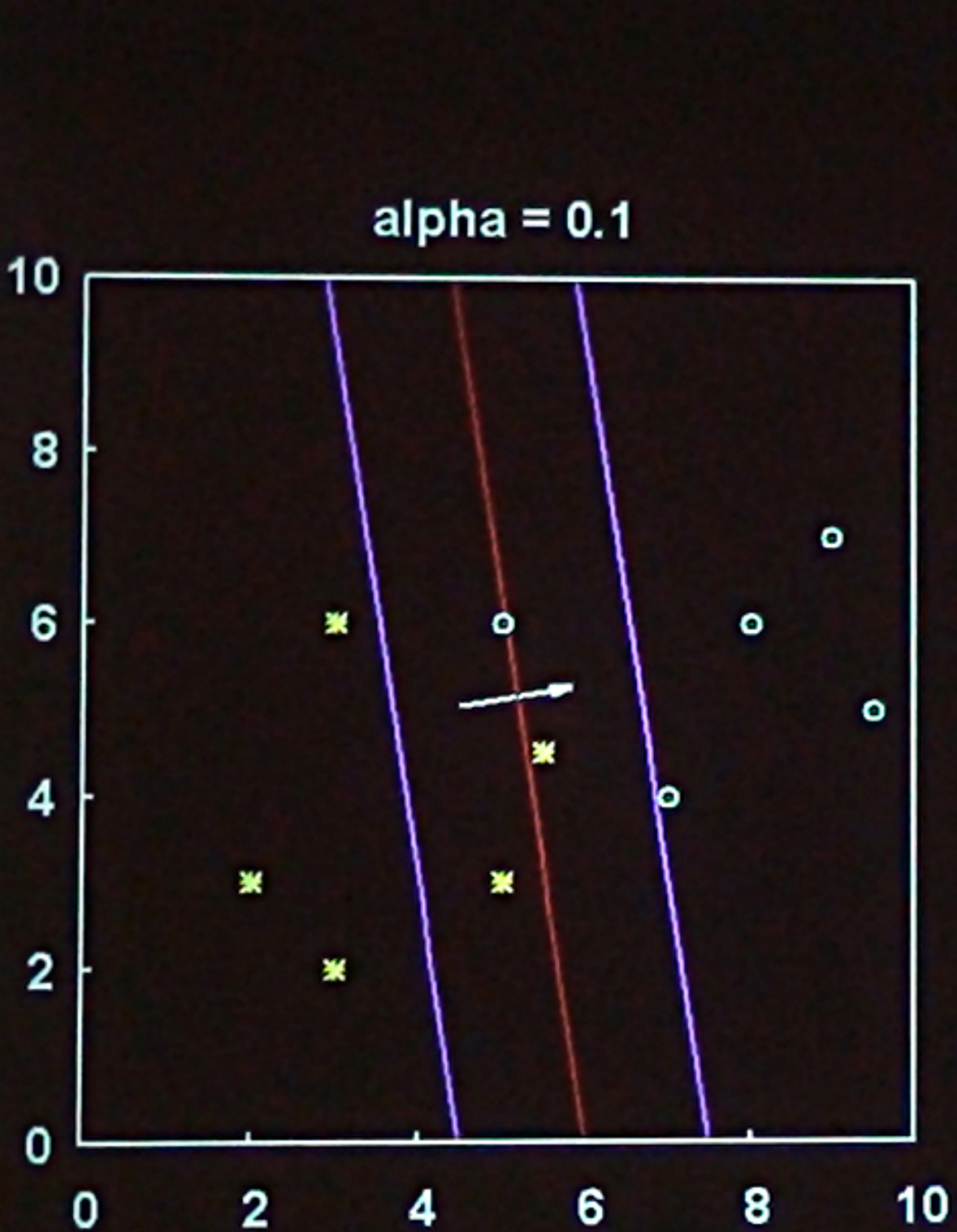

25
20
15
10
5
0
E_D(w)
E_W(w)
Number of iterations
1
10
100
1000
10000

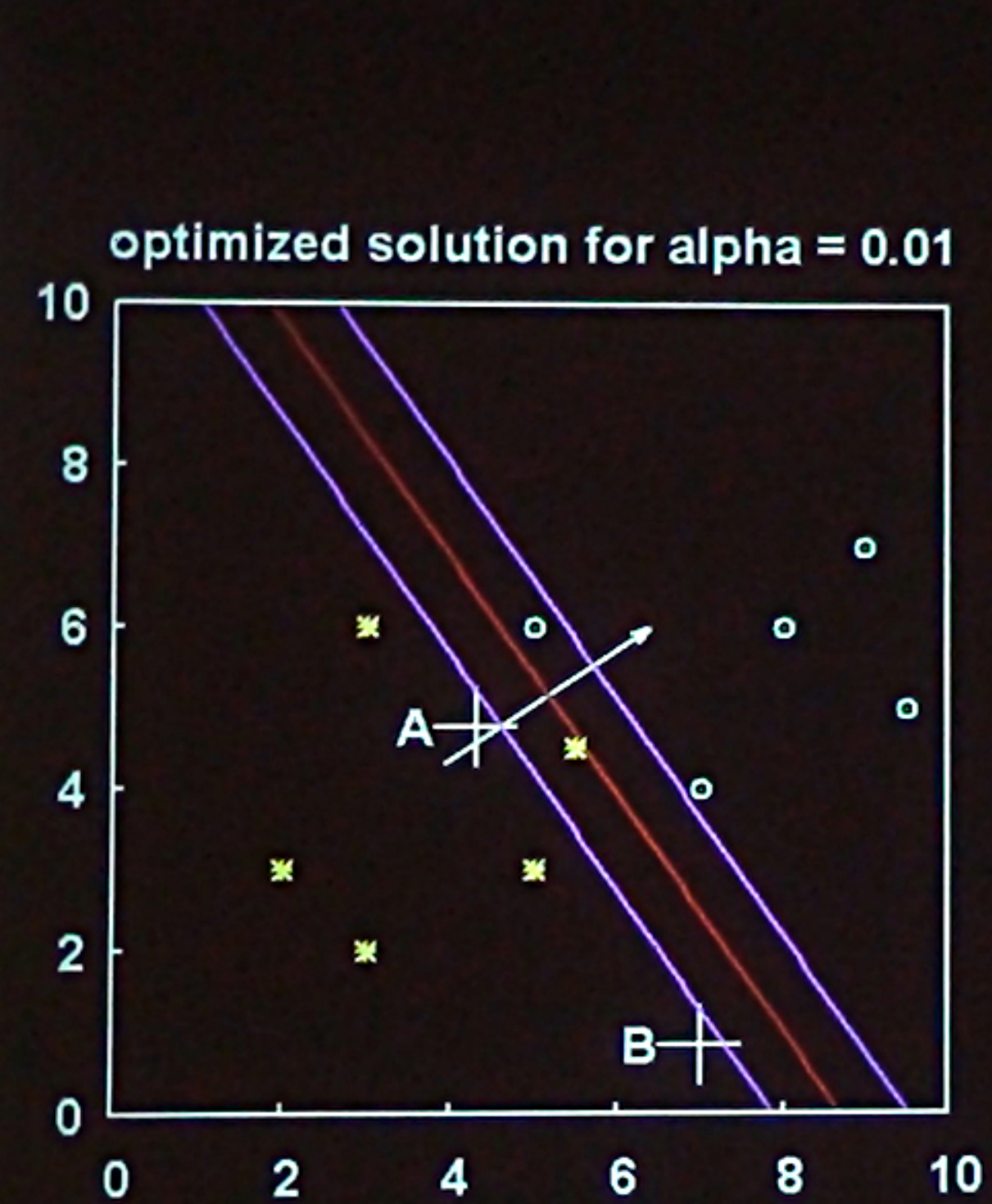

Steepest descents (alpha = 1.00): 9999 iterations
10
5
0
-5
-10
-6
-4
-2
0
2
4
6


Steepest descents (alpha = 1.00): 9999 iterations
M(w)
E_D(w)
E_W(w)
14
12
10
8
6
4
2
0
10
100
1000
10000

$$M = \beta E_D(w) + \sum_{c=1}^{C} \alpha_c E_w^{(c)}(w)$$


solution with alpha=0.01
10
8
6
4
2
0
0
2
4
6
8
10
tions
6


alpha = 0.1
10
8
6
4
2
0
0
2
4
6
8
10


optimized solution for alpha = 0.01
10
8
6
4
2
0
0
2
4
6
8
10
A
B

"Learning" with regularization
or "weight decay"

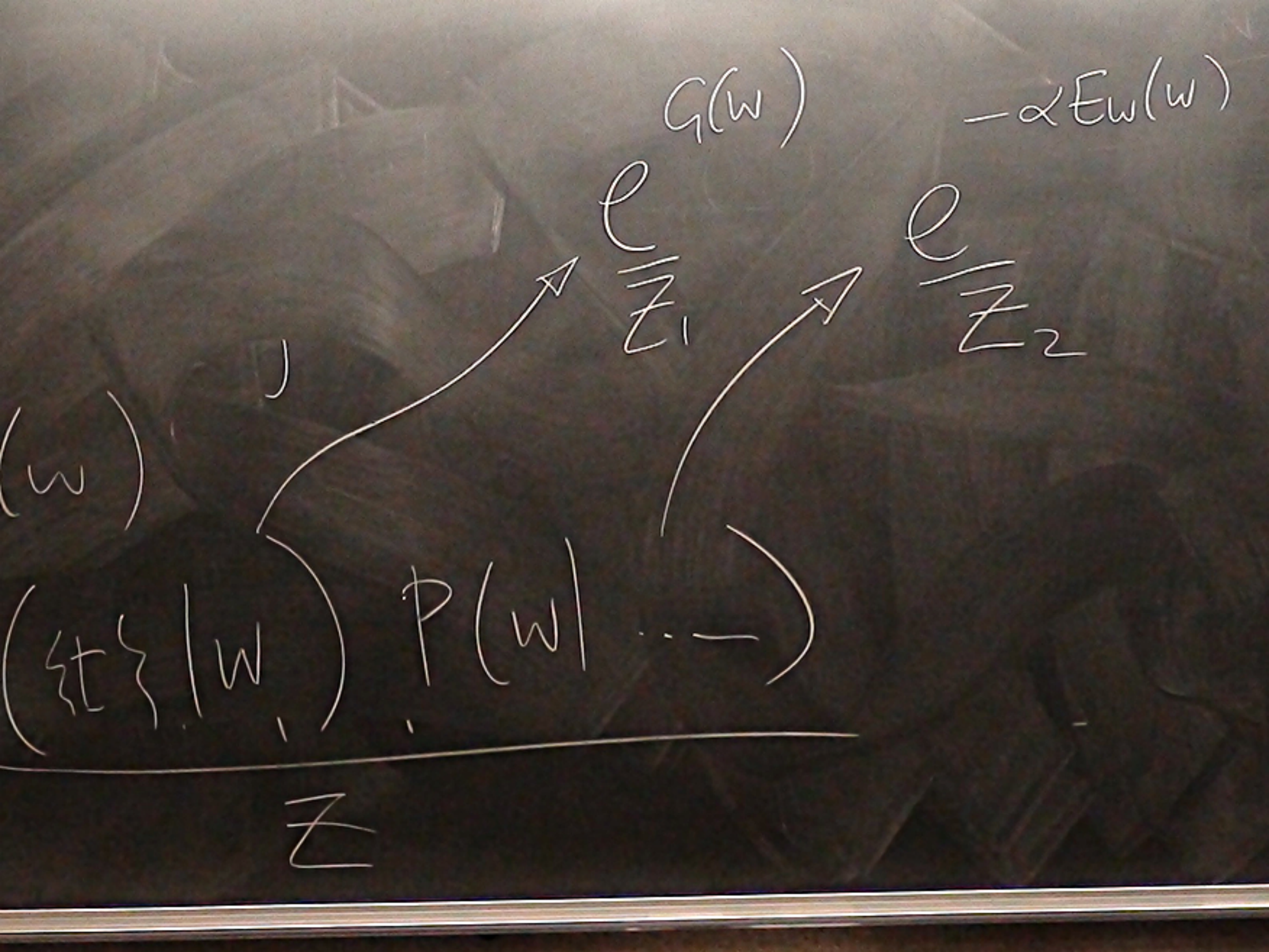
Learning as Inference

regularization

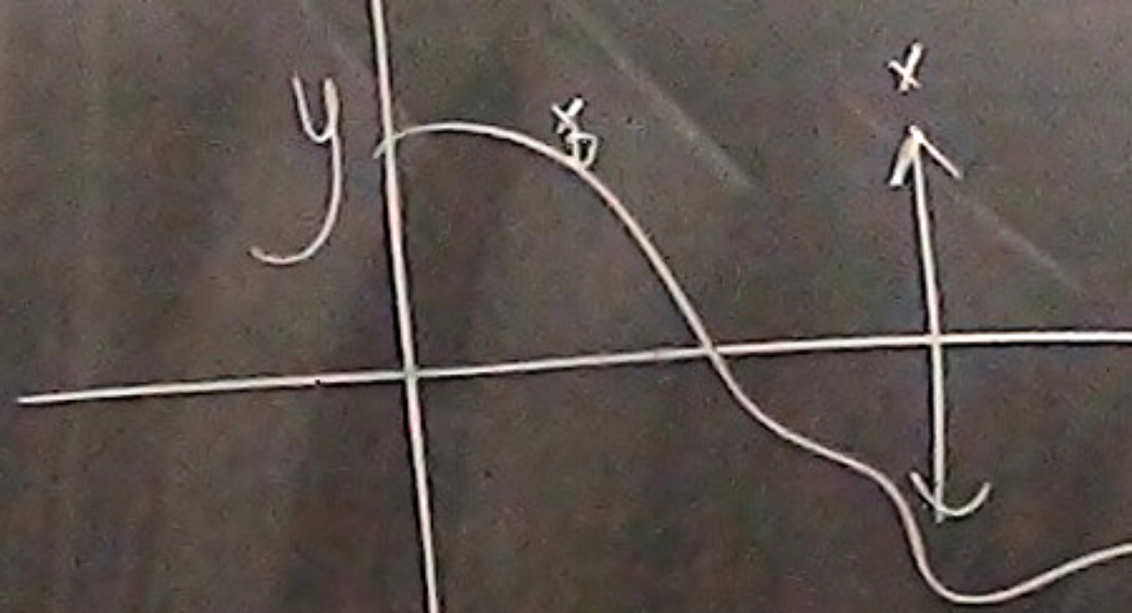
"weight decay"

$$M(w) = G(w) + \alpha E_w(w)$$

$$P(w \mid \{t\}) = \frac{e^{-M(w)}}{Z} = \frac{P(\{t\} \mid w) P}{Z}$$

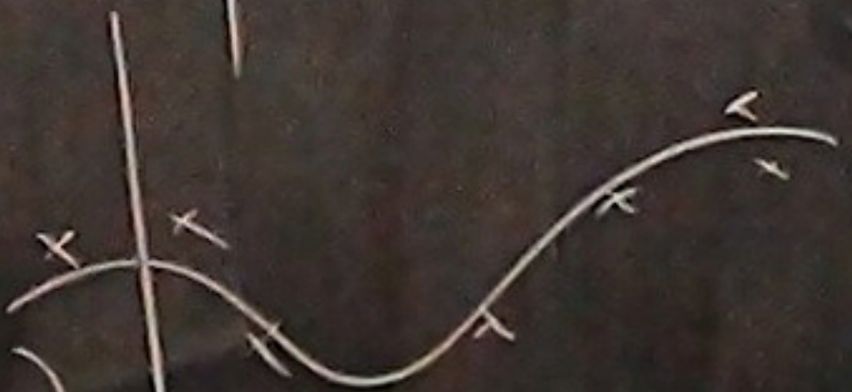

q(w)
−αEw(w)
e
Z1
e
Z2
(w)
J
(红球|w)
P(w|
…—
Z

$$\frac{e}{Z_2}$$



$$E_w = \frac{1}{2} w^2$$

$$P(w) \sim \text{Normal}(0, \sigma_w^2)$$



$$\boxed{\sigma_w^2 = \frac{1}{\alpha}}$$

Inference

→ Prediction

$$P\left(t_{N+1}=1 \,\middle|\, x_{N+1}, \{\text{Data}\}, \ldots\right)$$

$$= \int dw \; P(w \mid \text{Data}) \; y(x$$

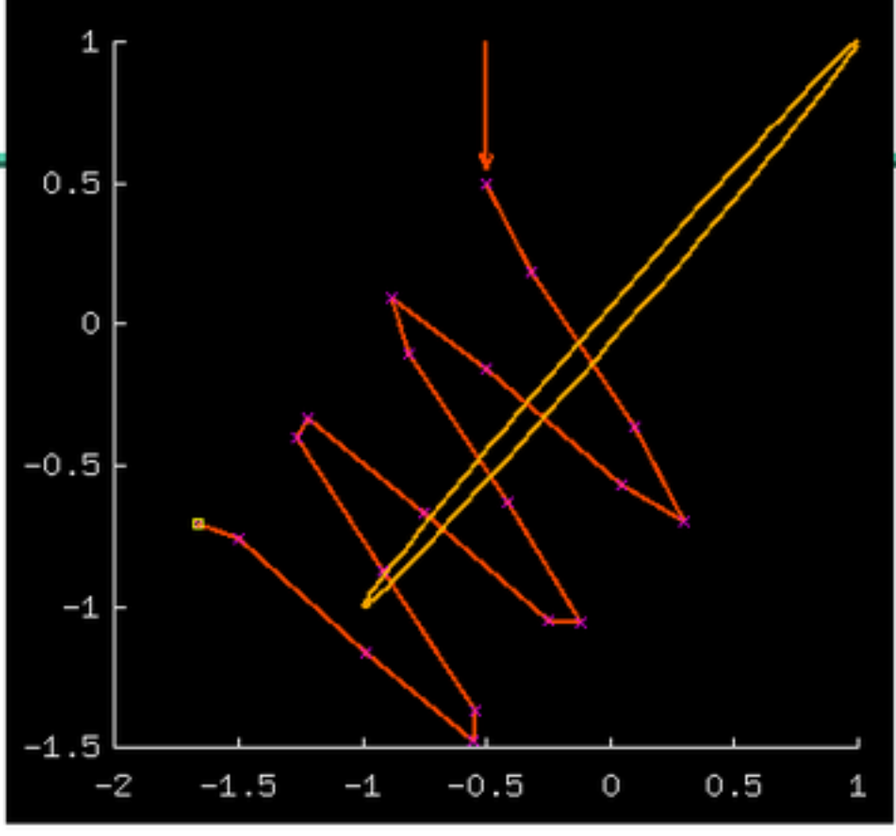
Prediction

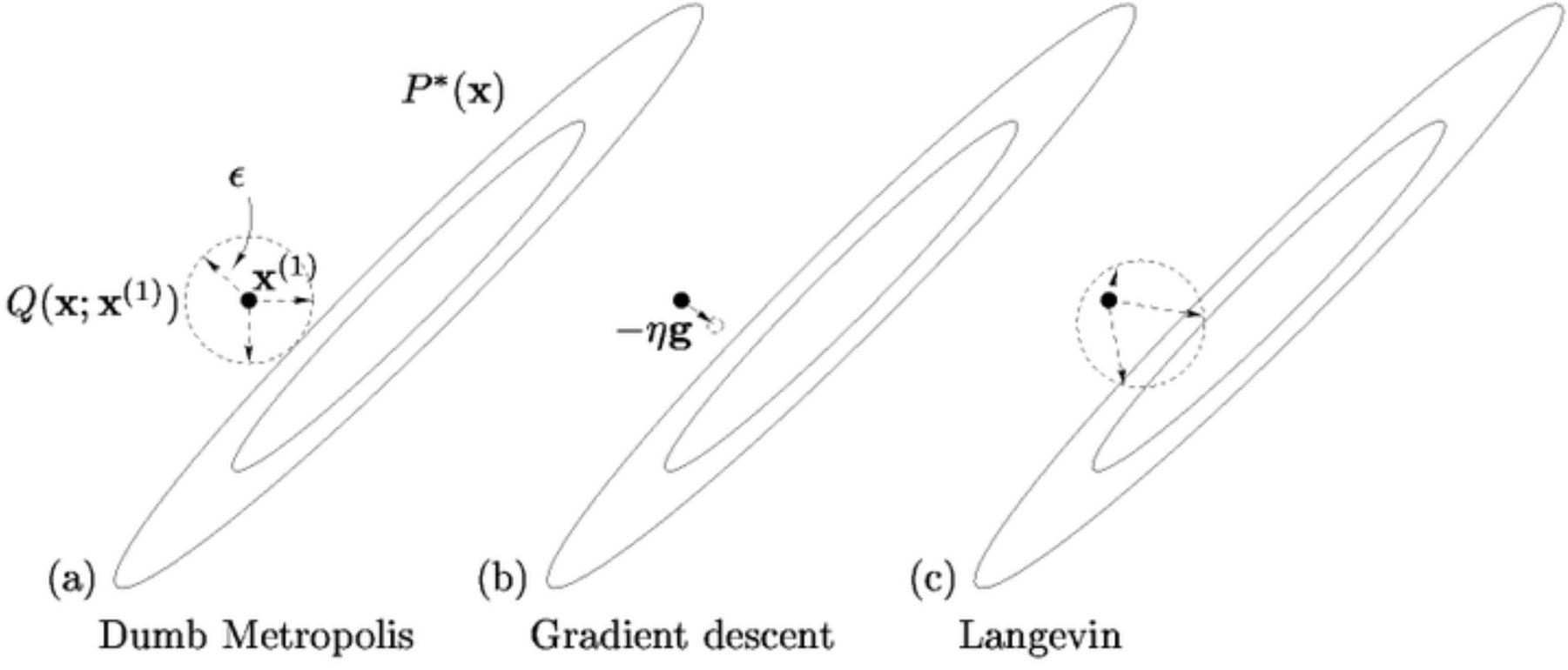
$$P(t_{N+1}=1 \mid x_{N+1}, \{\text{Data}\}, \ldots)$$

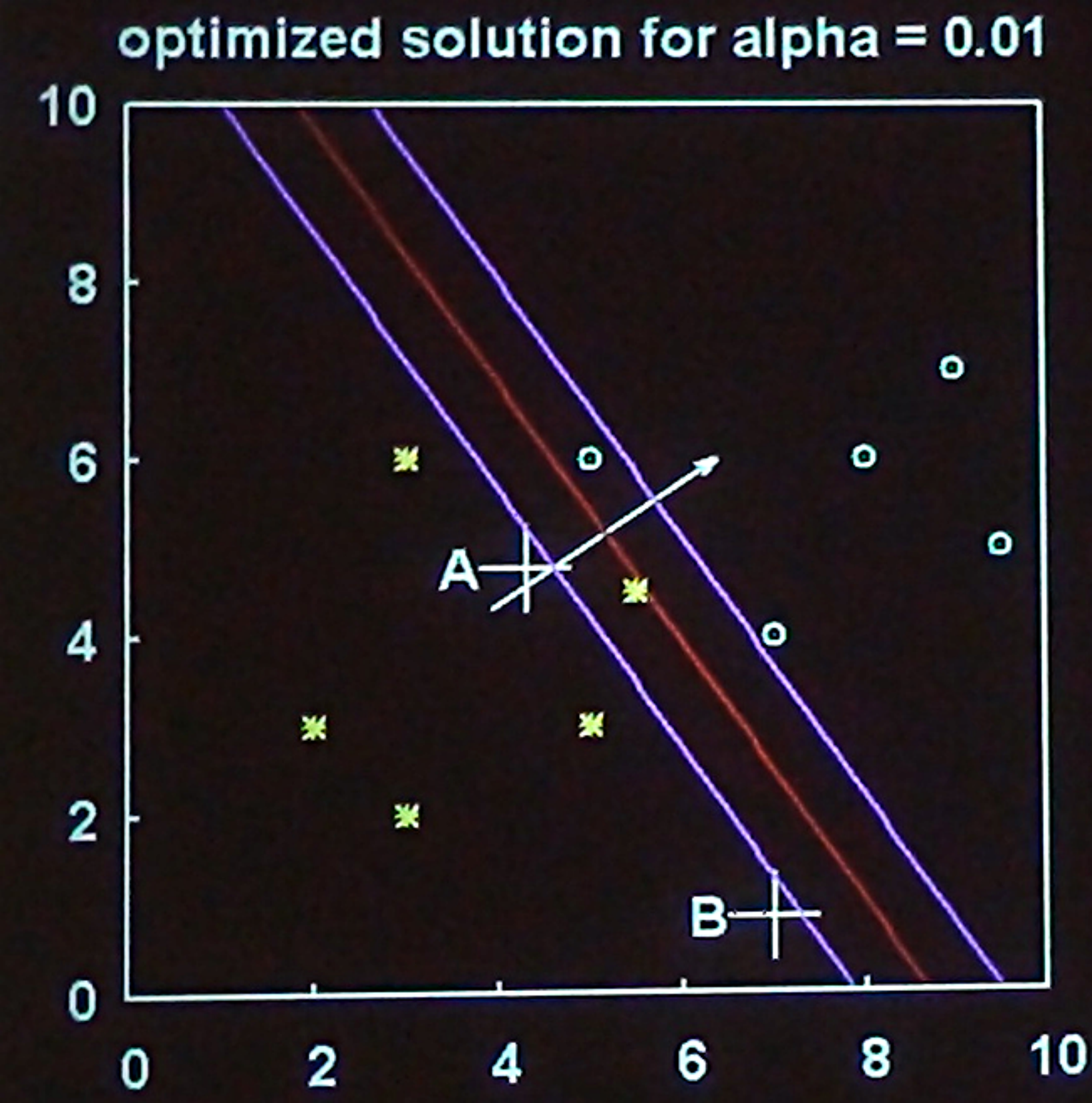
$$= \int dw \; P(w \mid \text{Data}) \; y(x_{N+1}; w)$$

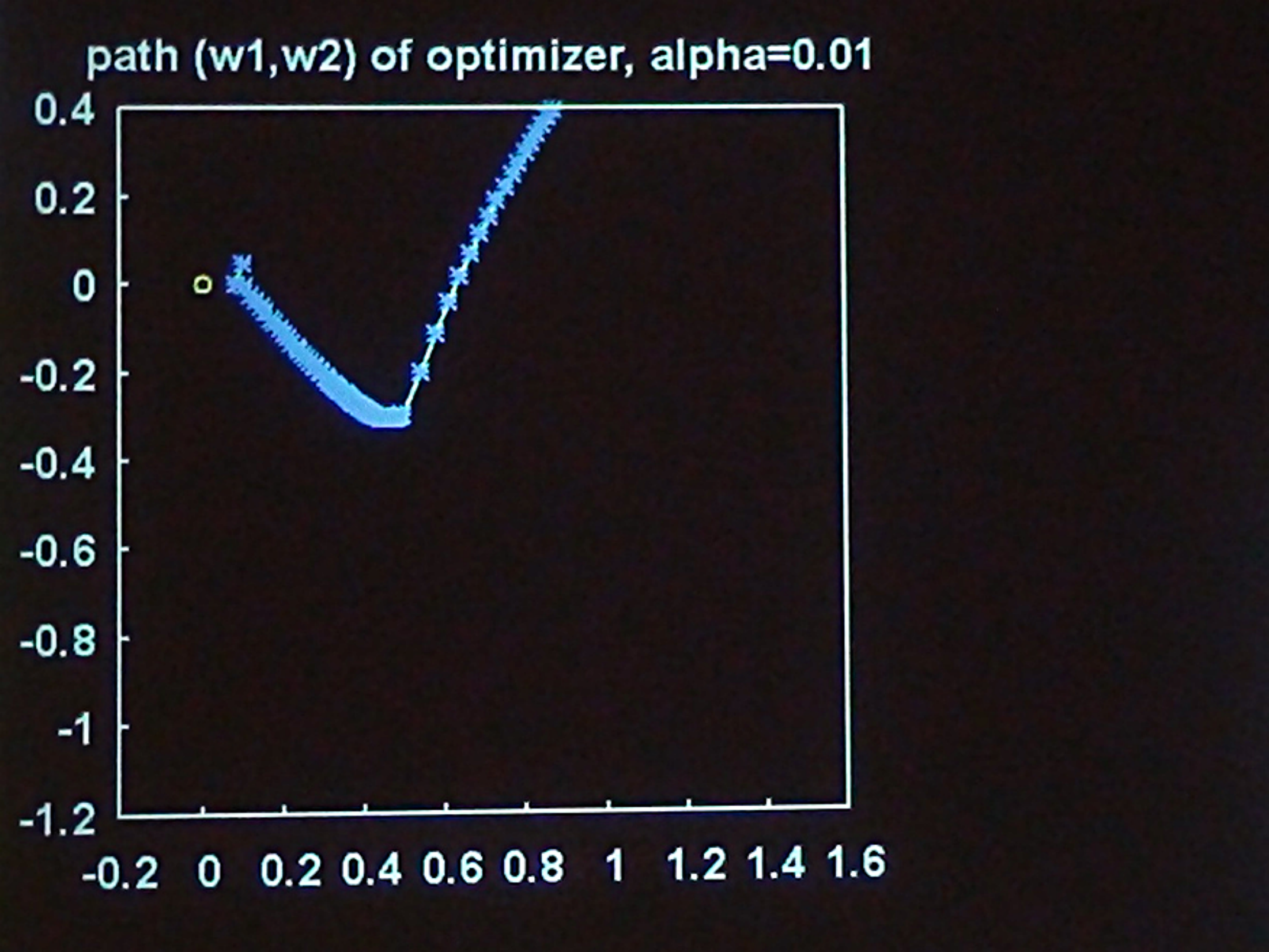
Langevin method

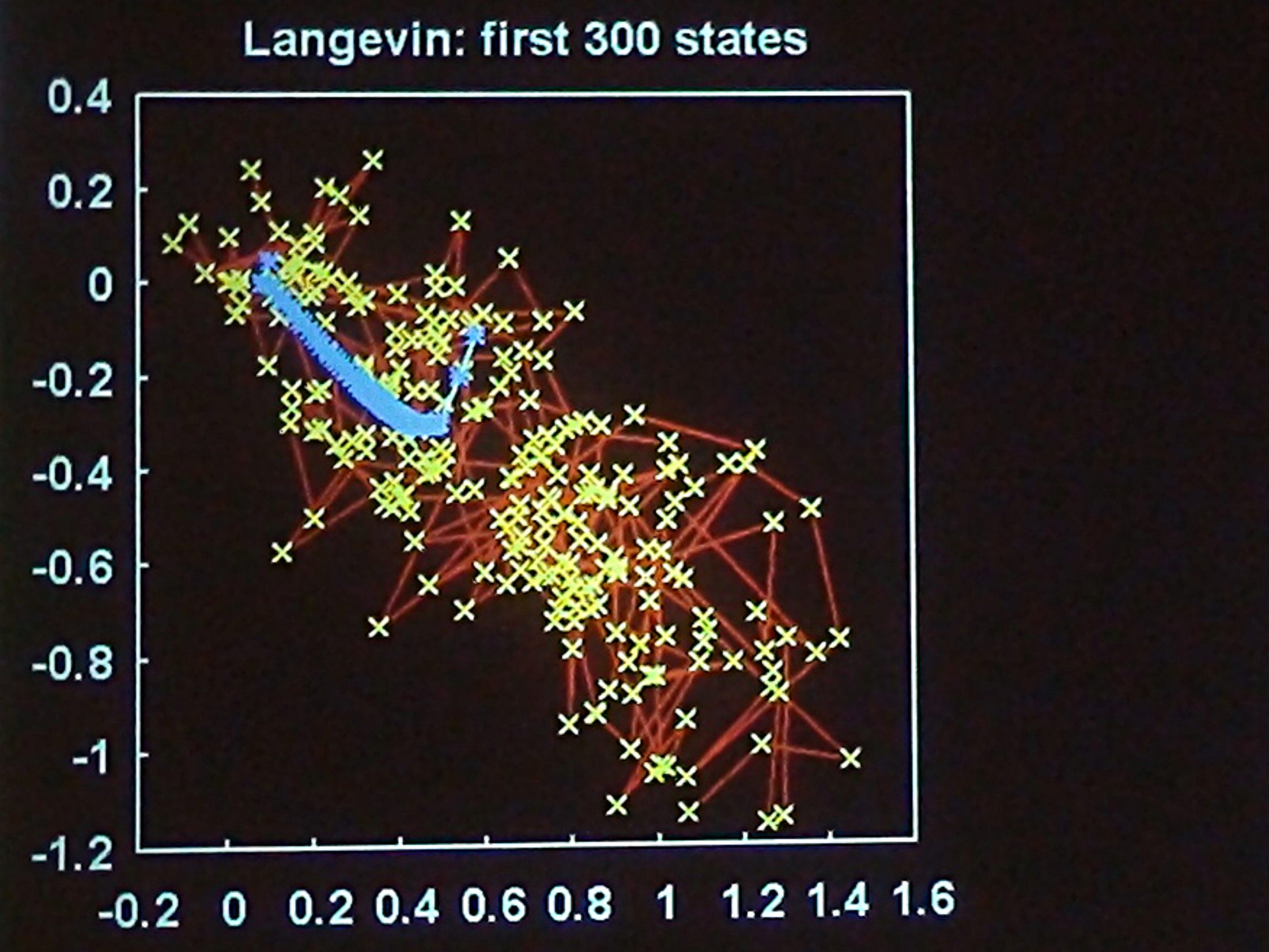
- Simple version of Hamiltonian Monte Carlo



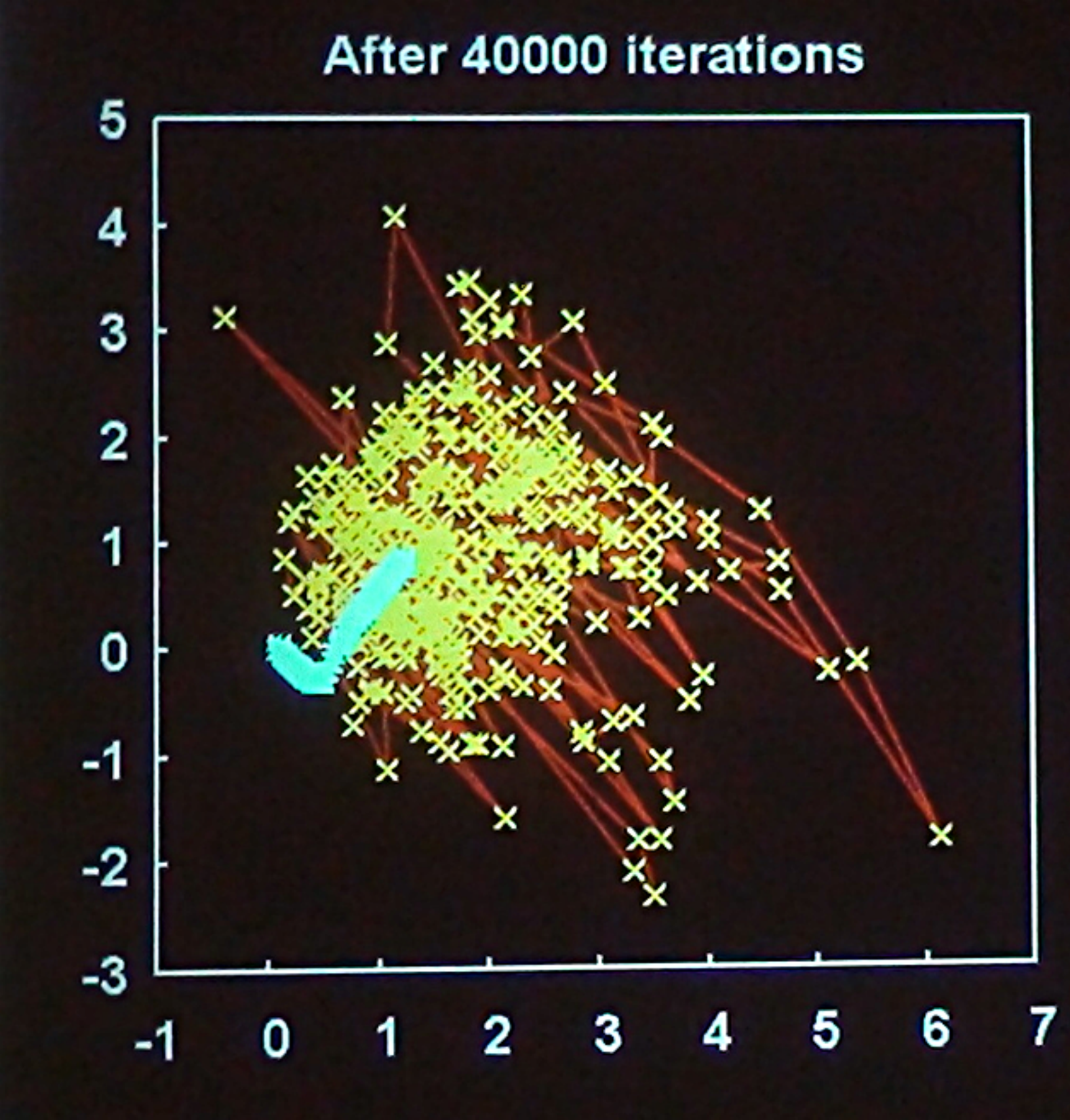


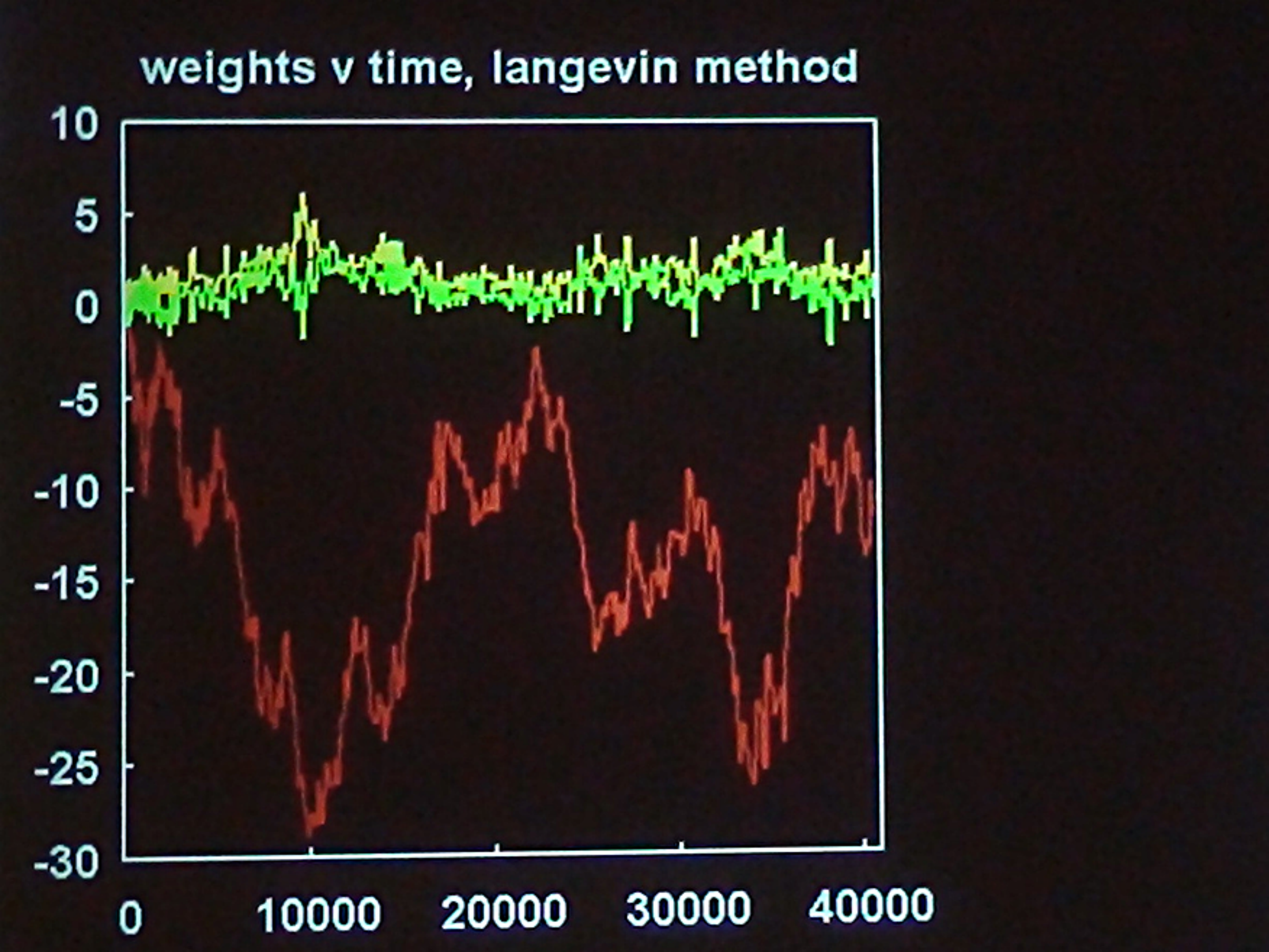

optimized solution for alpha = 0.01
10
8
6
4
2
0
0
2
4
6
8
10
A
B


path (w1,w2) of optimizer, alpha=0.01
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
-1.2
-0.2
0
0.2
0.4
0.6
0.8
1
1.2
1.4
1.6

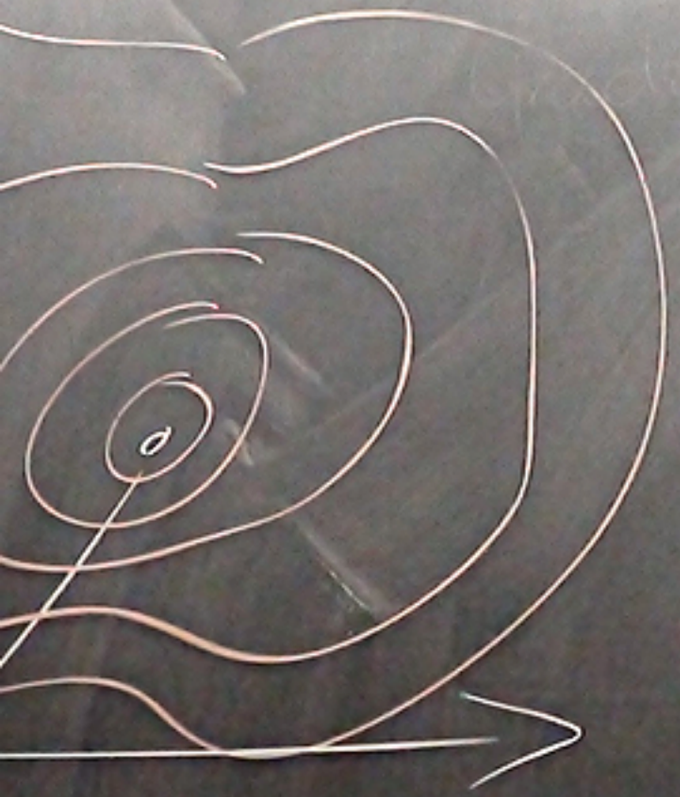

Langevin: first 300 states
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
-1.2
-0.2
0
0.2
0.4
0.6
0.8
1
1.2
1.4
1.6

w_2
w_1


After 40000 iterations
5
4
3
2
1
0
-1
-2
-3
-1
0
1
2
3
4
5
6
7

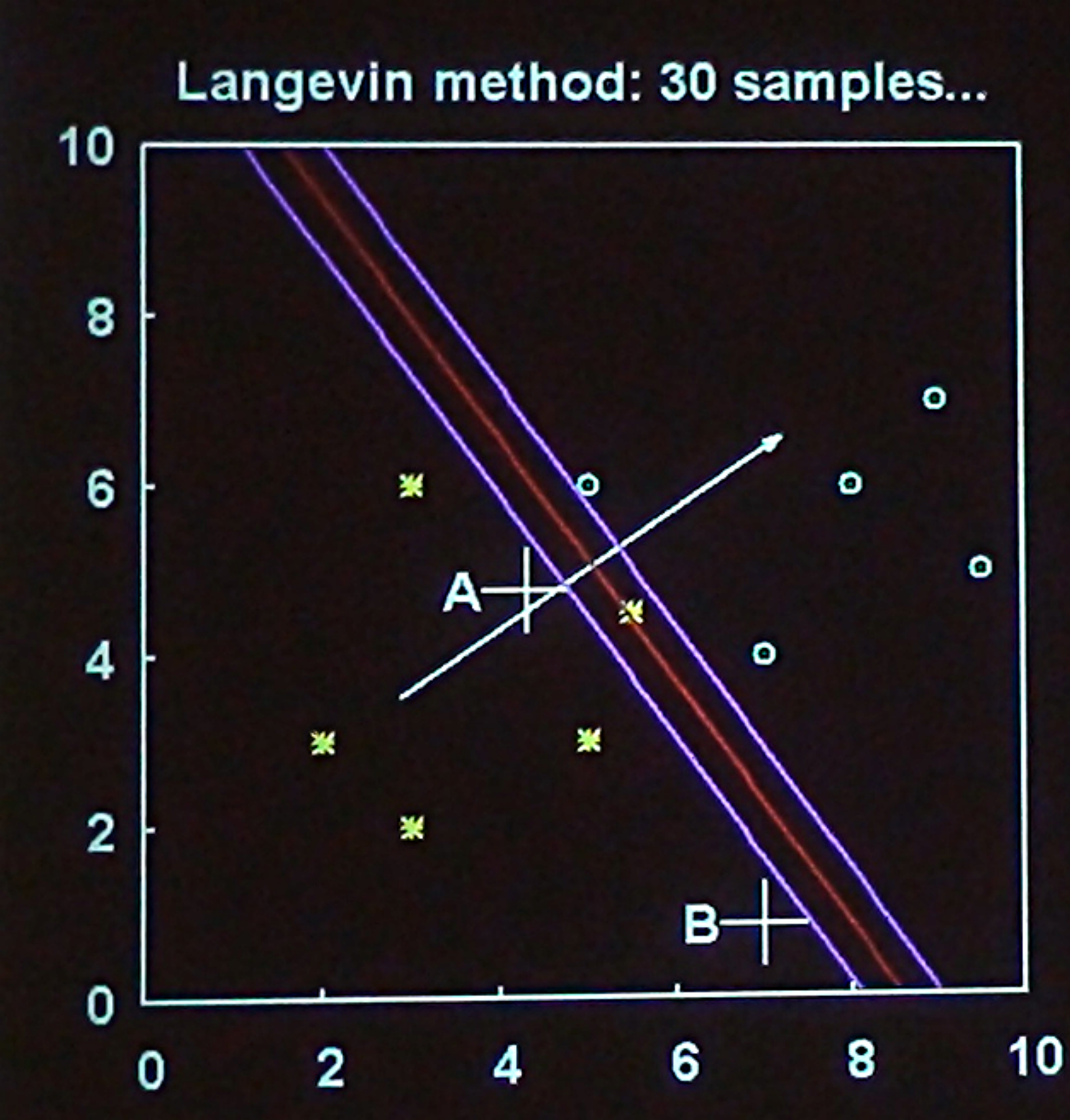

weights v time, langevin method
10
5
0
-5
-10
-15
-20
-25
-30
0
10000
20000
30000
40000

Every 100th sample from 10000 to 40000
5
4
3
2
1
0
-1
-2
-3
-1
0
1
2
3
4
5
6
7

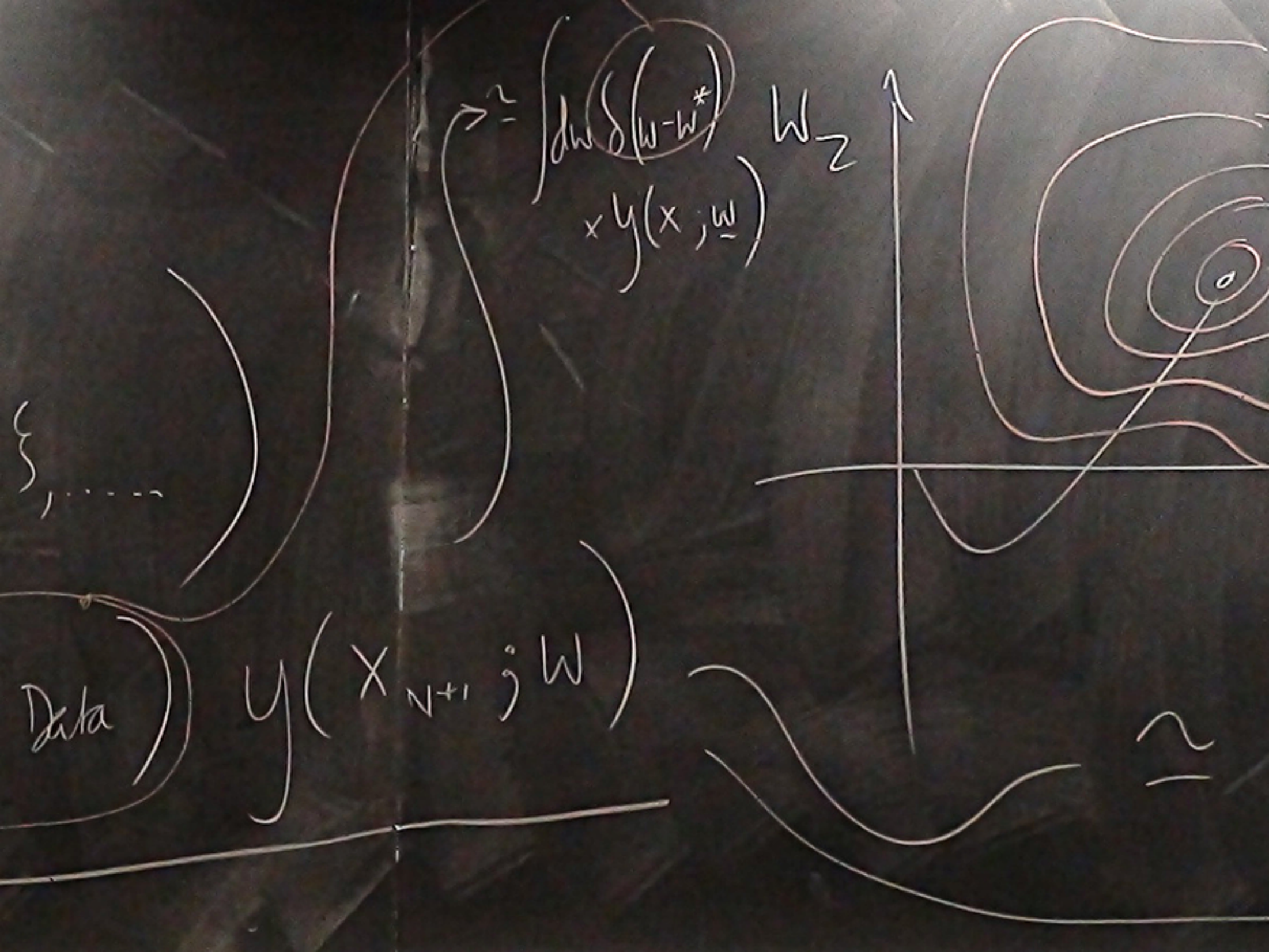


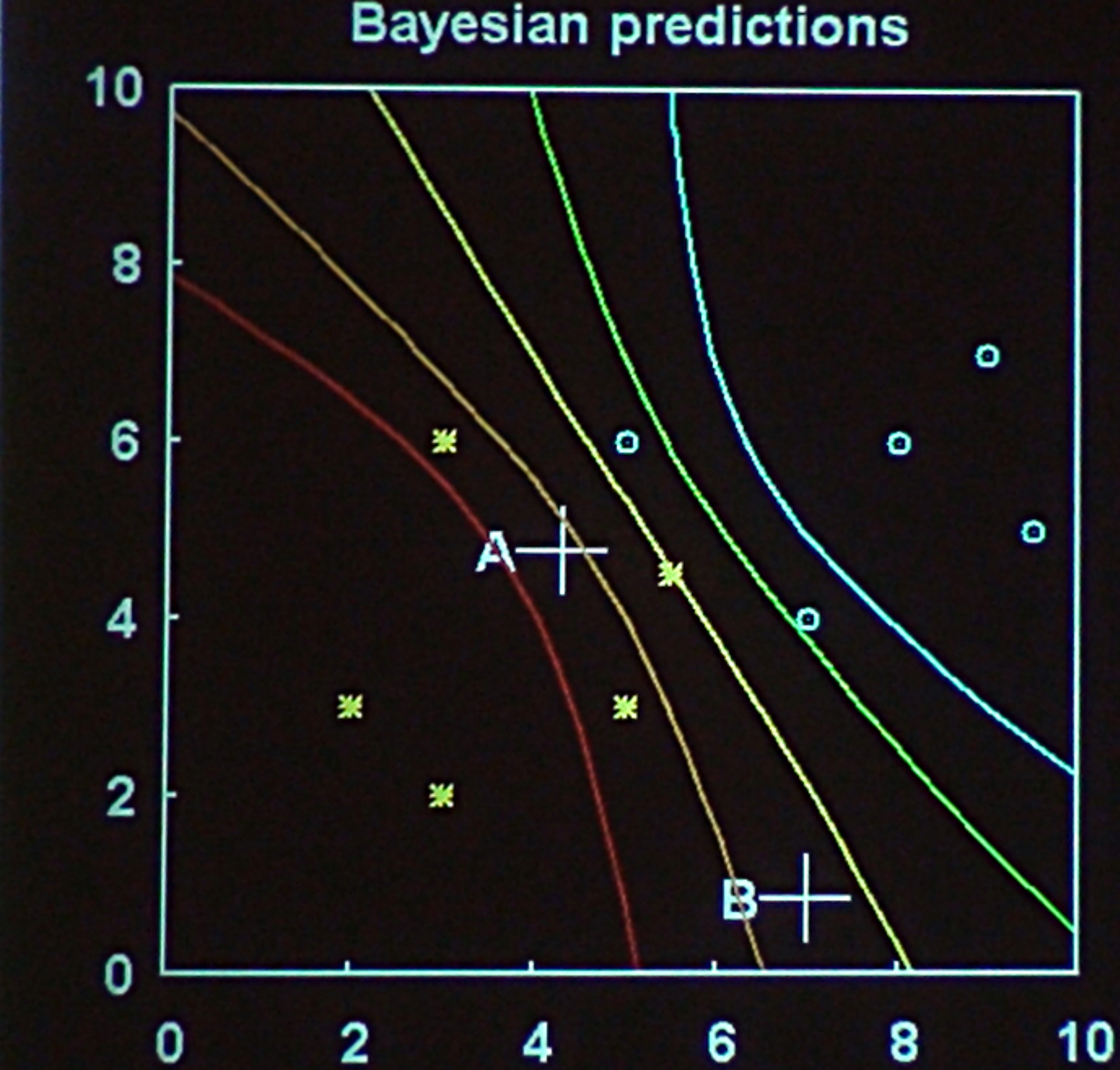
$w^{(r)} \sim P$ through HMC

$$\simeq \frac{1}{R} \sum_{r=1}^{R} y(x_{N+1}; w^{(r)})$$


Langevin method: 30 samples...
10
8
6
4
2
0
0
2
4
6
8
10
A
B

$$\int dw\, \delta(w - w^*) \times y(x; w)$$

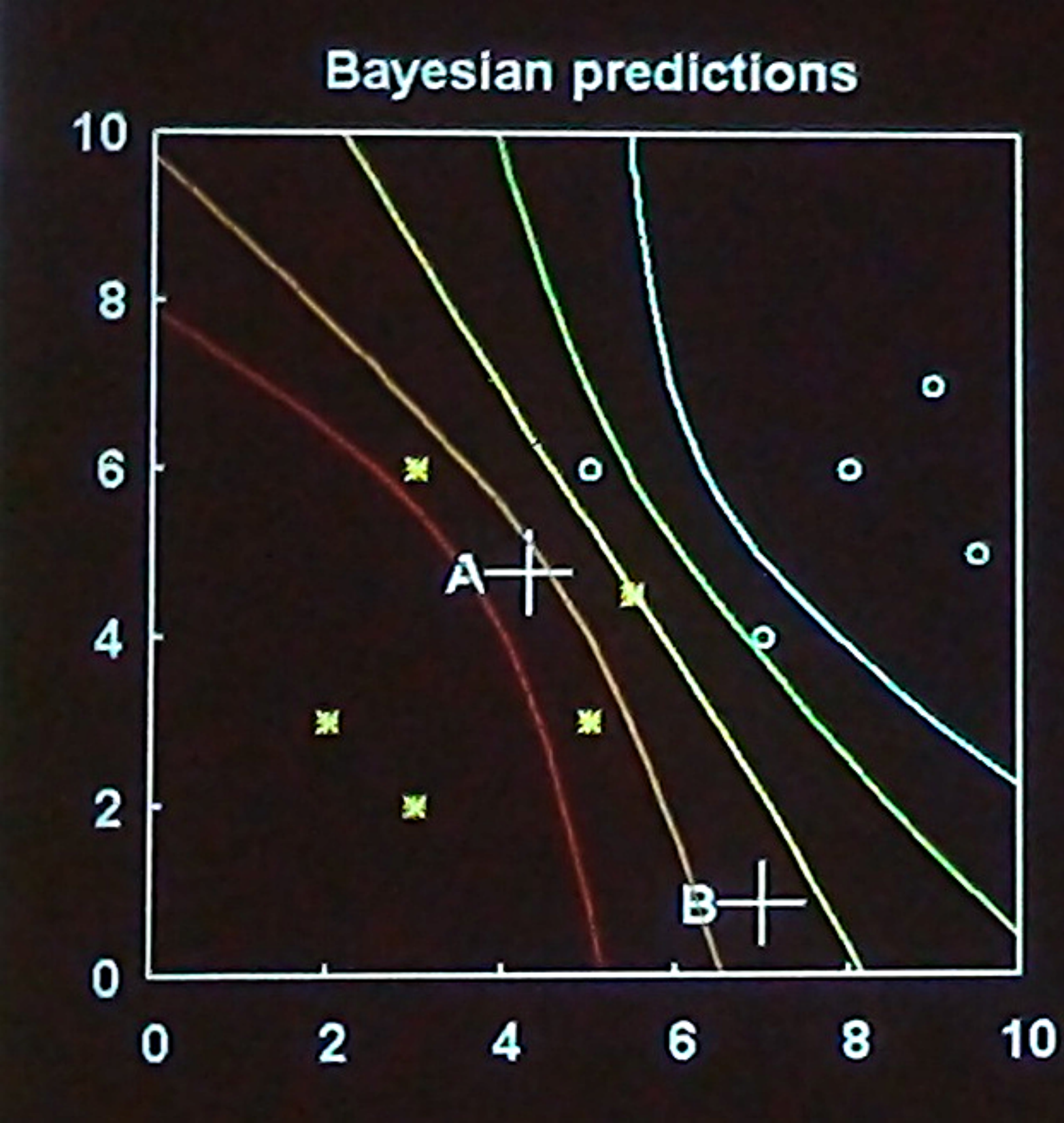


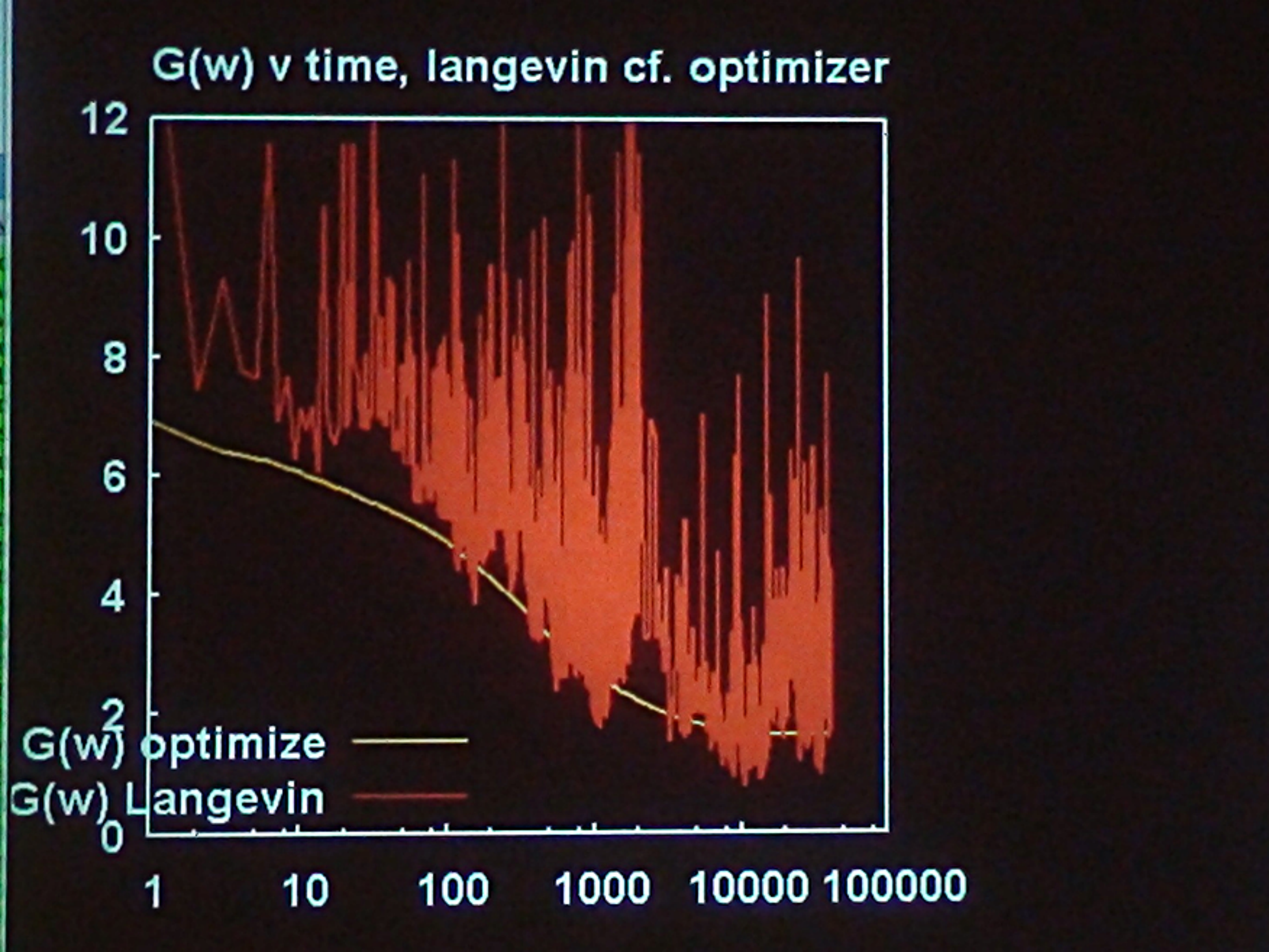

Bayesian predictions
10
8
6
4
2
0
0
2
4
6
8
10
A
B

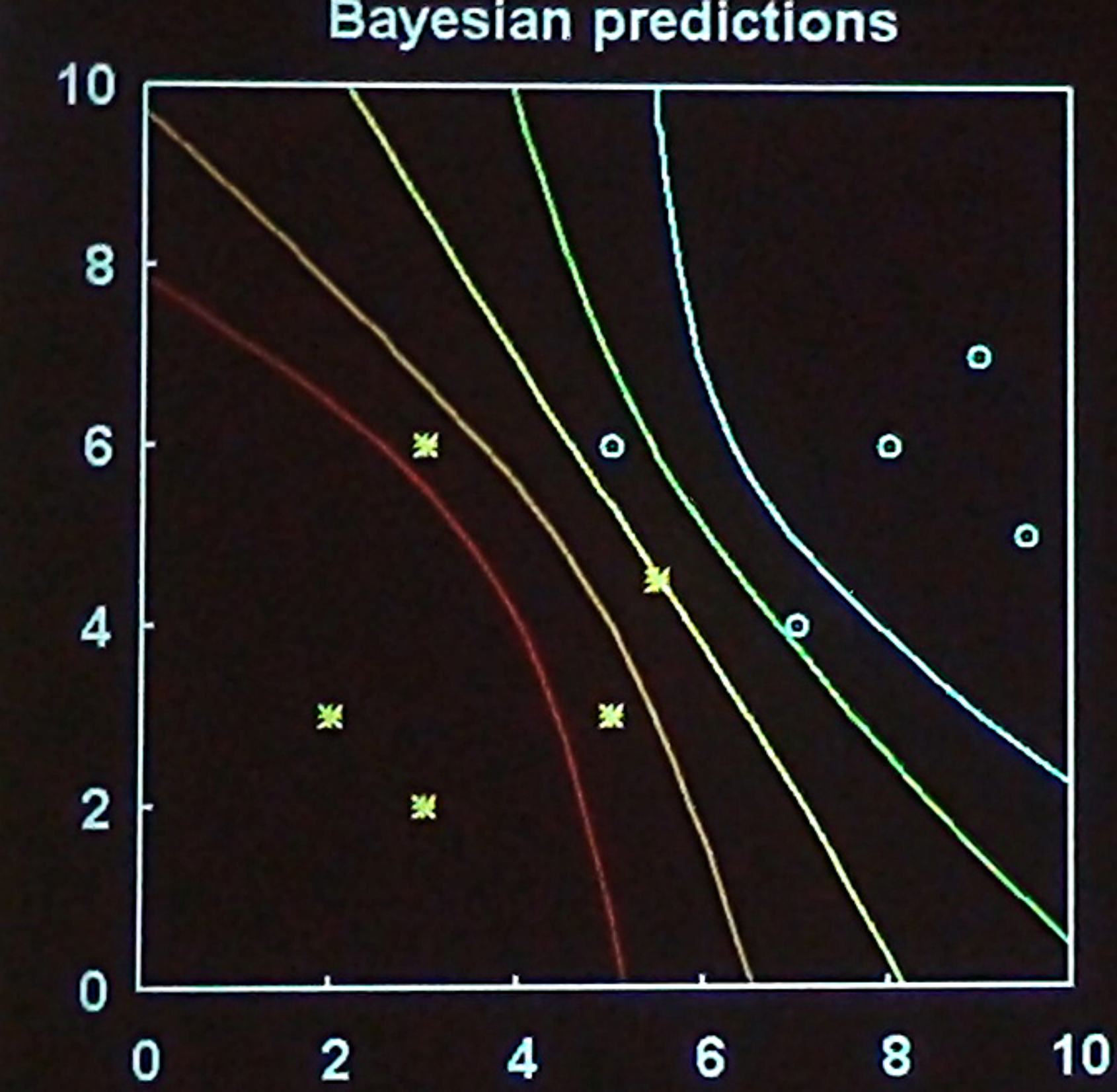
Laplace's method

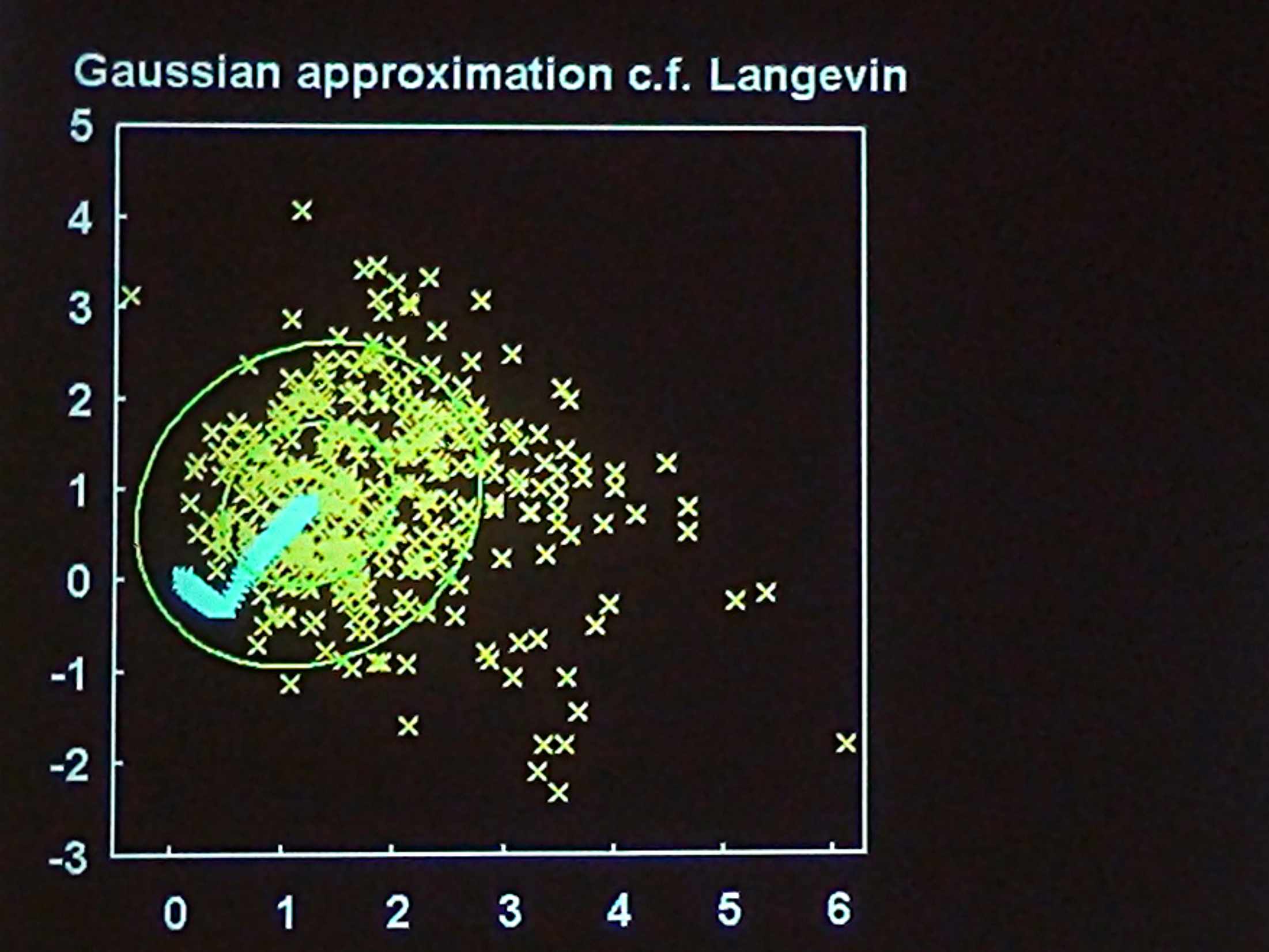
eg Normal

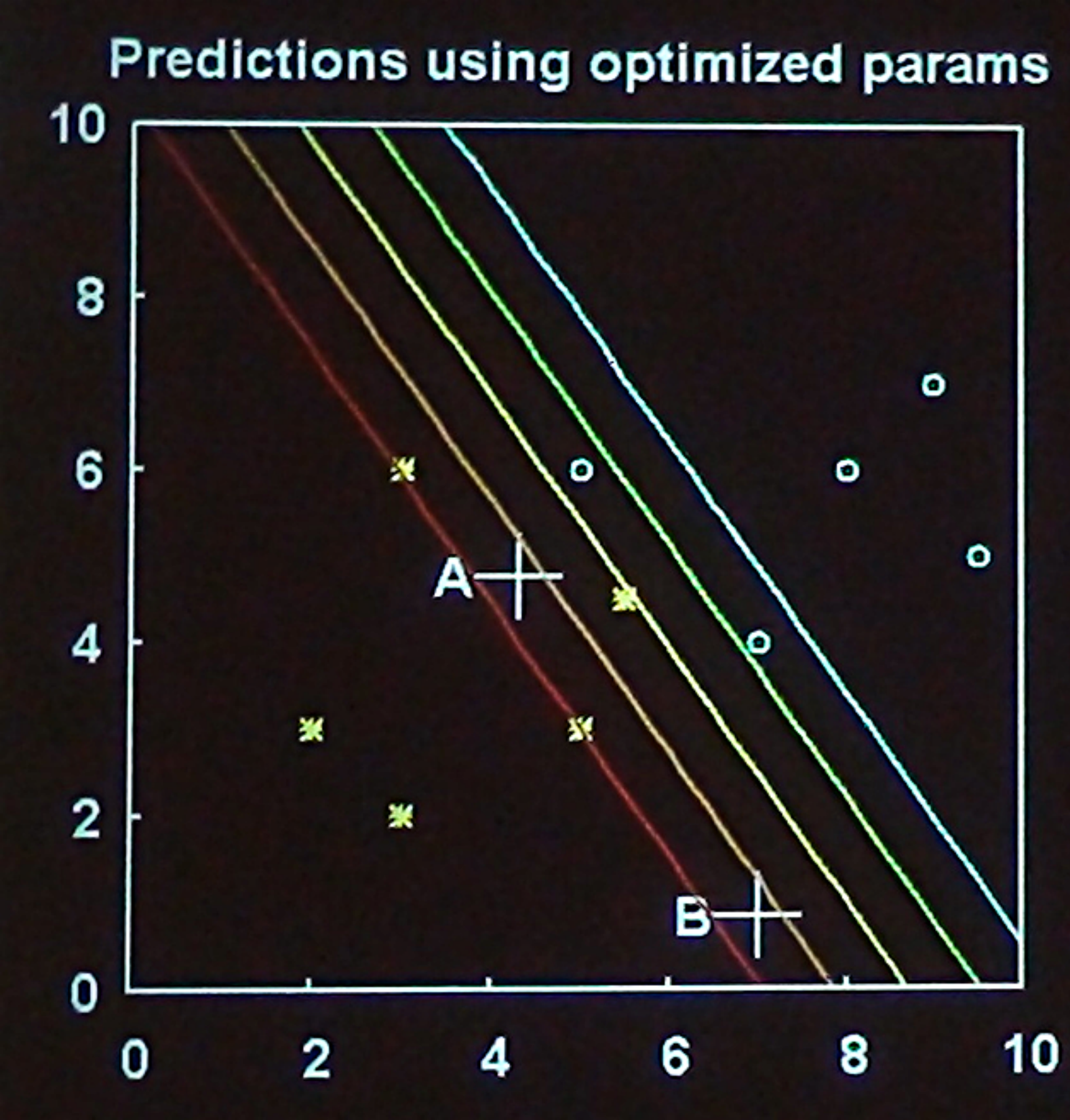
$$\simeq \int dw\, Q(w|\theta)\, y(x;w)$$

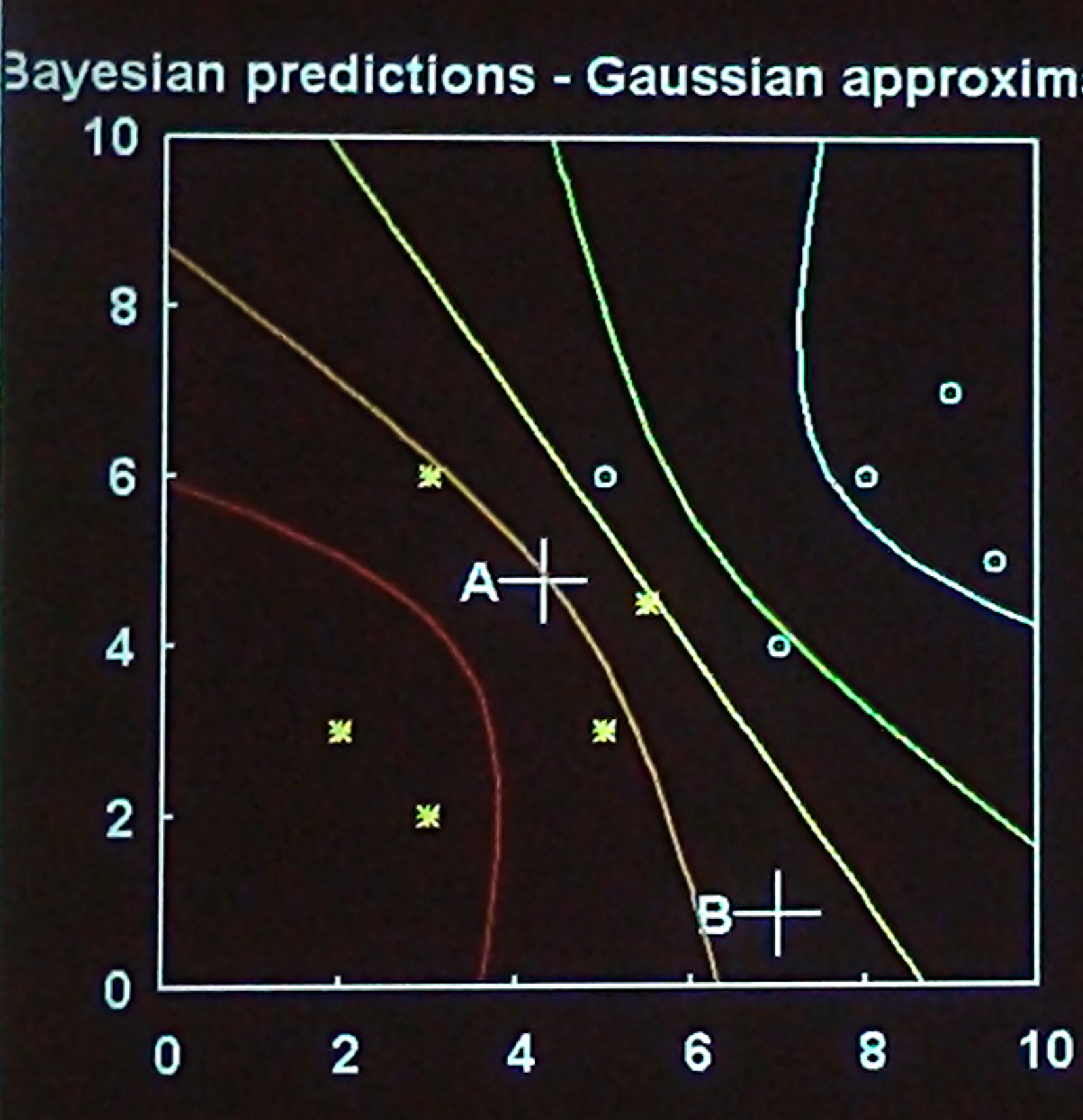

Bayesian predictions
10
8
6
4
2
0
0
2
4
6
8
10
A
B


G(w) v time, langevin cf. optimizer
12
10
8
6
4
2
0
G(w) optimize
G(w) Langevin
1
10
100
1000
10000
100000


Bayesian predictions
10
8
6
4
2
0
0
2
4
6
8
10


Gaussian approximation c.f. Langevin
5
4
3
2
1
0
-1
-2
-3
0
1
2
3
4
5
6

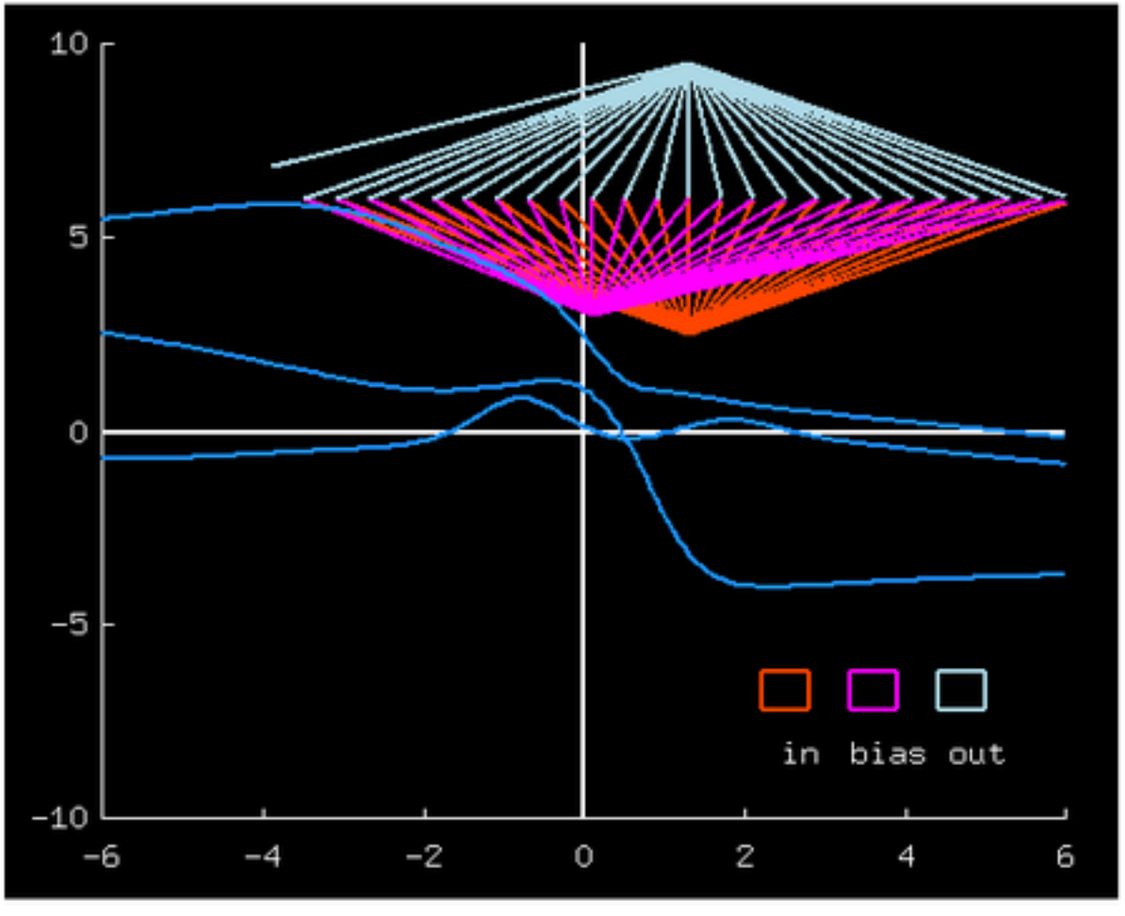

Predictions using optimized params
10
8
6
4
2
0
0
2
4
6
8
10
A
B


Bayesian predictions - Gaussian approximation
10
8
6
4
2
0
0
2
4
6
8
10
A
B

Feedforward neural networks

- Just high-dimensional curve-fitting?
- Brains do more

Multi-layer network for regression



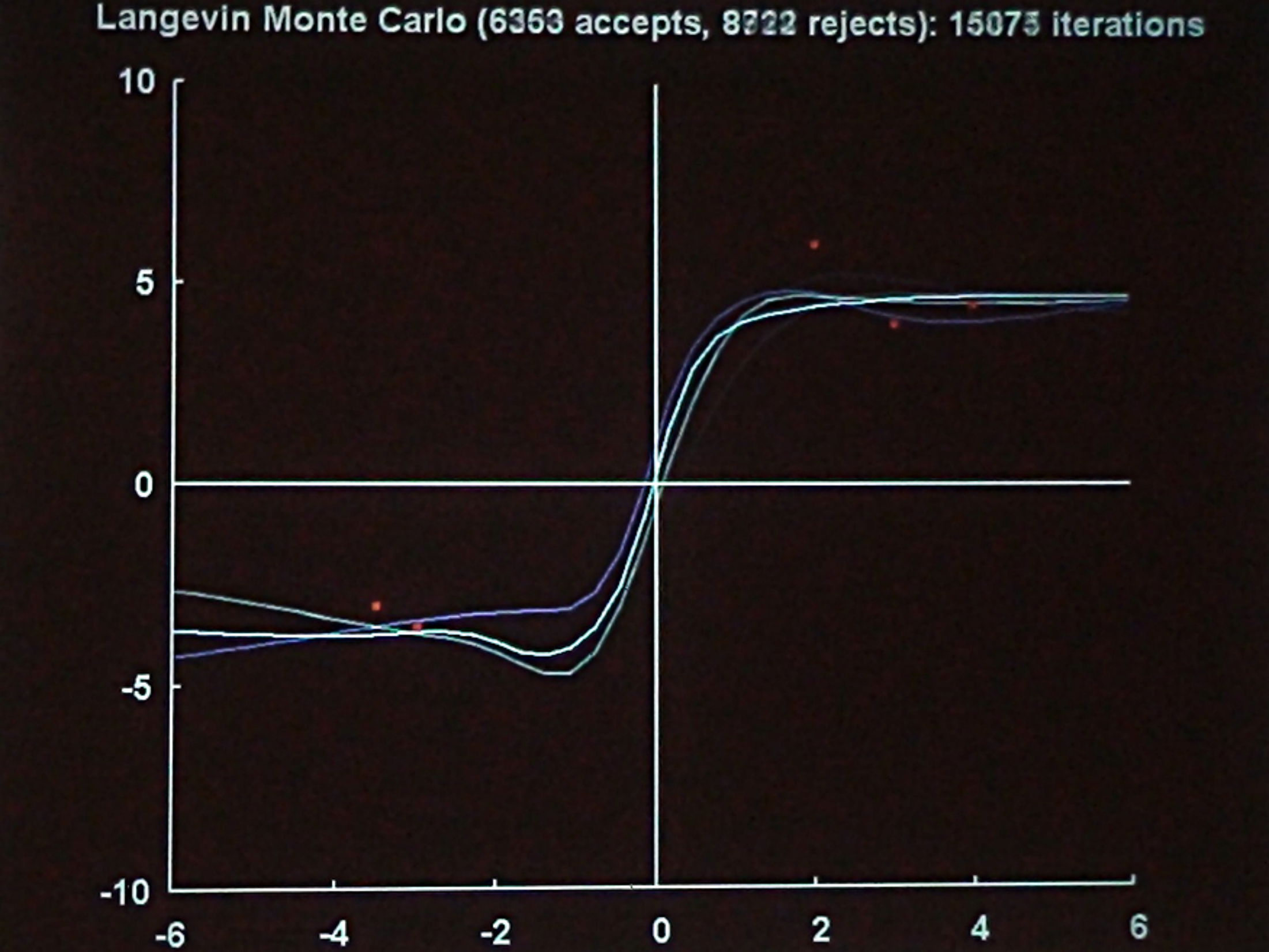
cd itp/neuron/net

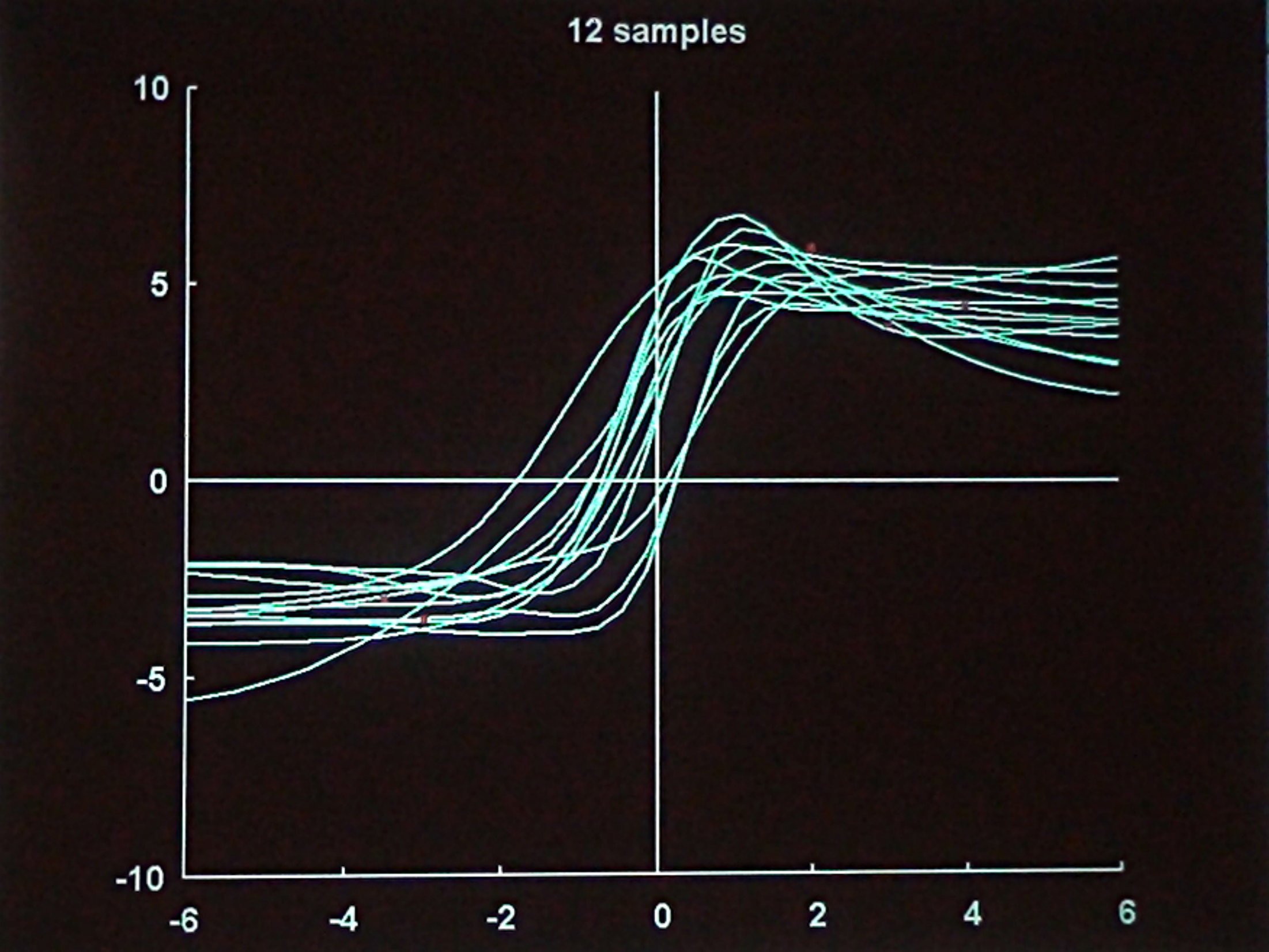
octave

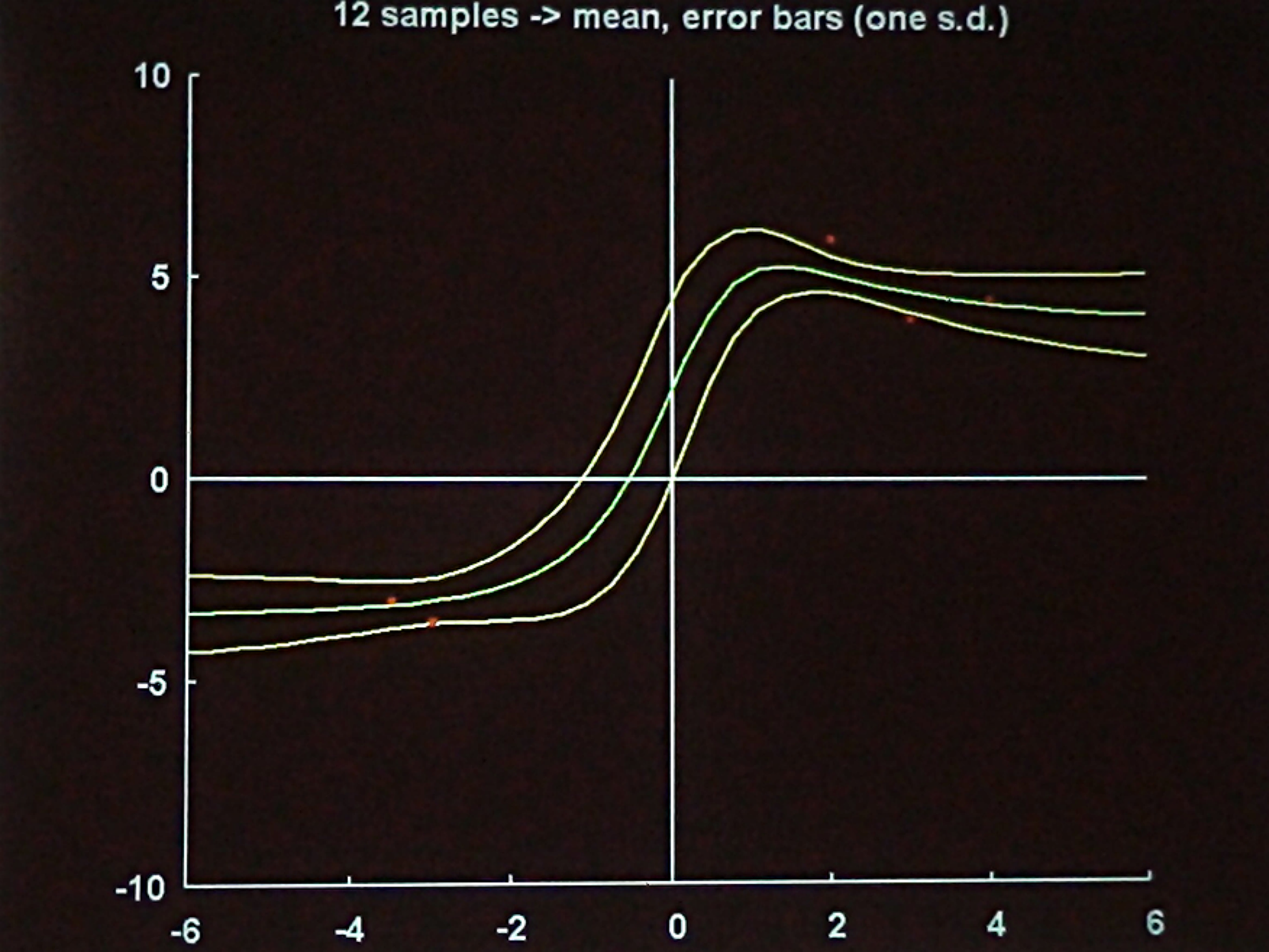
DEMO

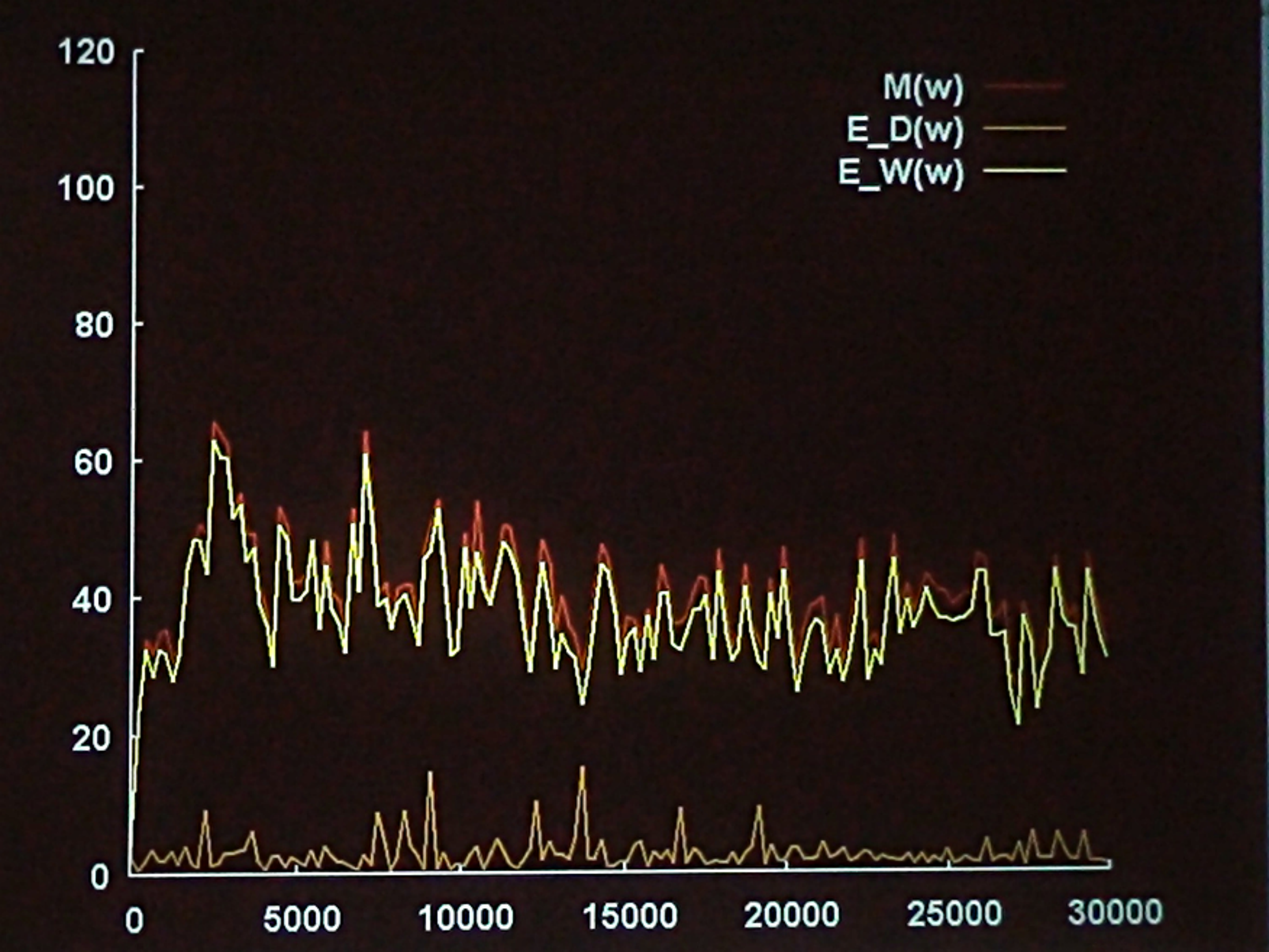
Applications

- Reading aloud - Nettalk
  - Sejnowski and Rosenberg
- Handwriting recognition - LeNet
  - Yann LeCun
- Weld toughness
  - Bhadeshia et al
- Focussing multiple-mirror telescopes
  - Roger Angel

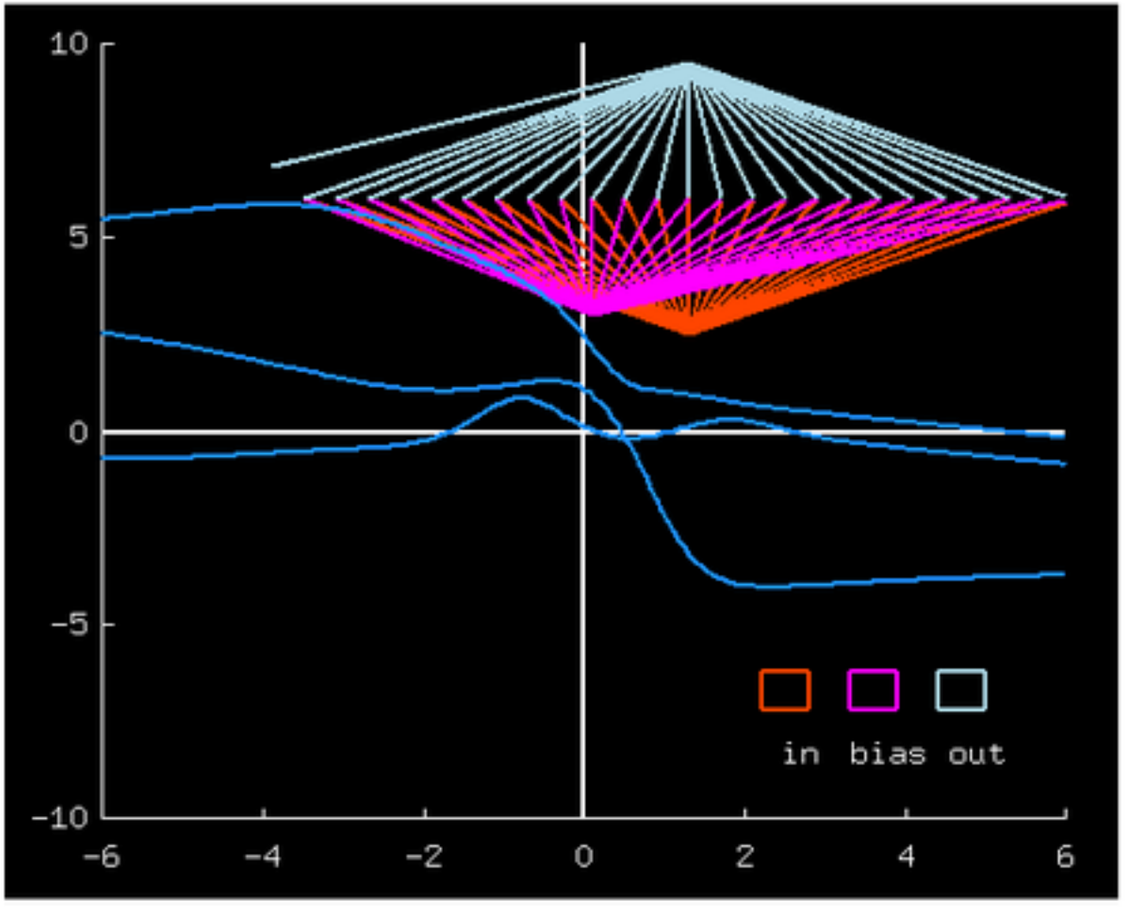

Langevin Monte Carlo (6363 accepts, 8722 rejects): 15075 iterations
10
5
0
-5
-10
-6
-4
-2
0
2
4
6


12 samples
10
5
0
-5
-10
-6
-4
-2
0
2
4
6


12 samples -> mean, error bars (one s.d.)
10
5
0
-5
-10
-6
-4
-2
0
2
4
6


M(w)
E_D(w)
E_W(w)
120
100
80
60
40
20
0
0
5000
10000
15000
20000
25000
30000

Multi-layer network for regression



cd itp/neuron/net

octave

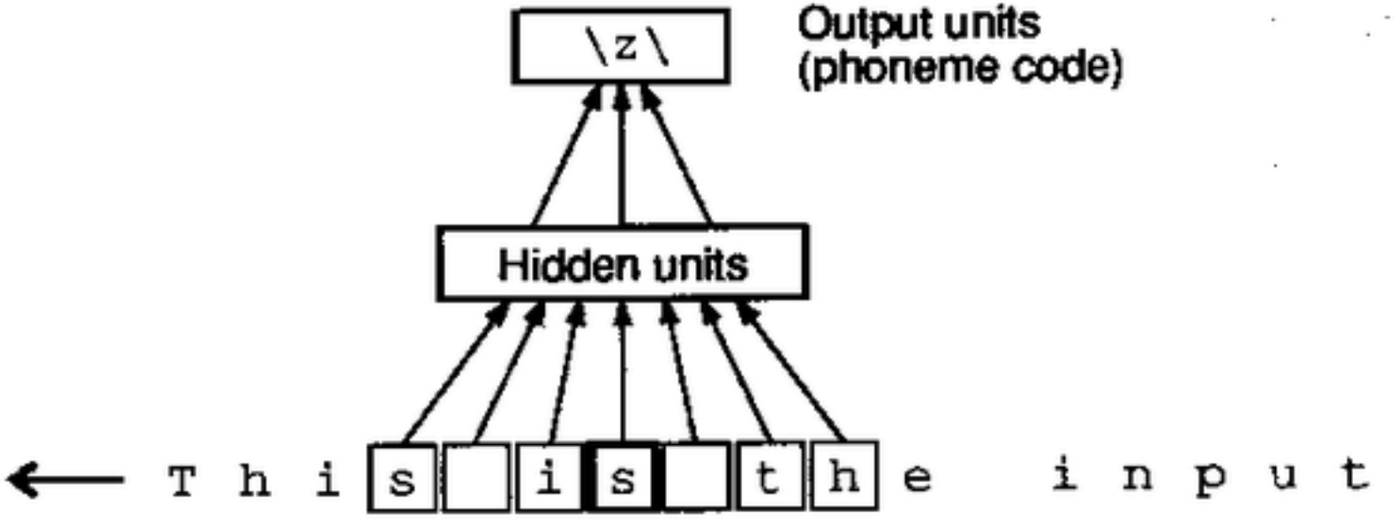
DEMO

Applications

- Reading aloud - Nettalk
  - Sejnowski and Rosenberg
- Handwriting recognition - LeNet
  - Yann LeCun
- Weld toughness
  - Bhadeshia et al
- Focussing multiple-mirror telescopes
  - Roger Angel

Applications

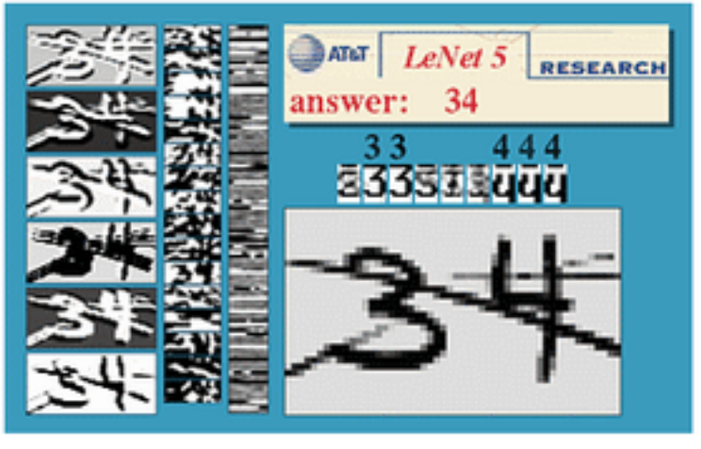
- Reading aloud - Nettalk
  - Sejnowski and Rosenberg



- Handwriting recognition - LeNet
  - Yann LeCun
- Weld toughness
  - Bhadeshia et al
- Focussing multiple-mirror telescopes
  - Roger Angel

Applications

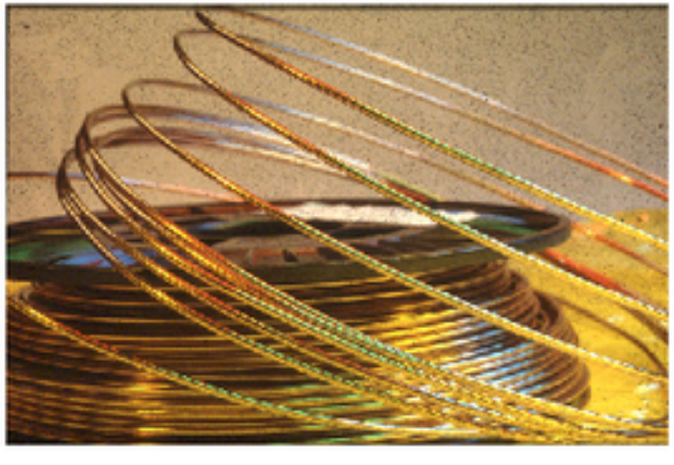
- Reading aloud - Nettalk
  - Sejnowski and Rosenberg
- Handwriting recognition - LeNet
  - Yann LeCun



- Weld toughness
  - Bhadeshia et al
- Focussing multiple-mirror telescopes
  - Roger Angel

Applications

- Reading aloud - Nettalk
  - Sejnowski and Rosenberg
- Handwriting recognition
  - Yann LeCun
- Weld toughness
  - Bhadeshia et al

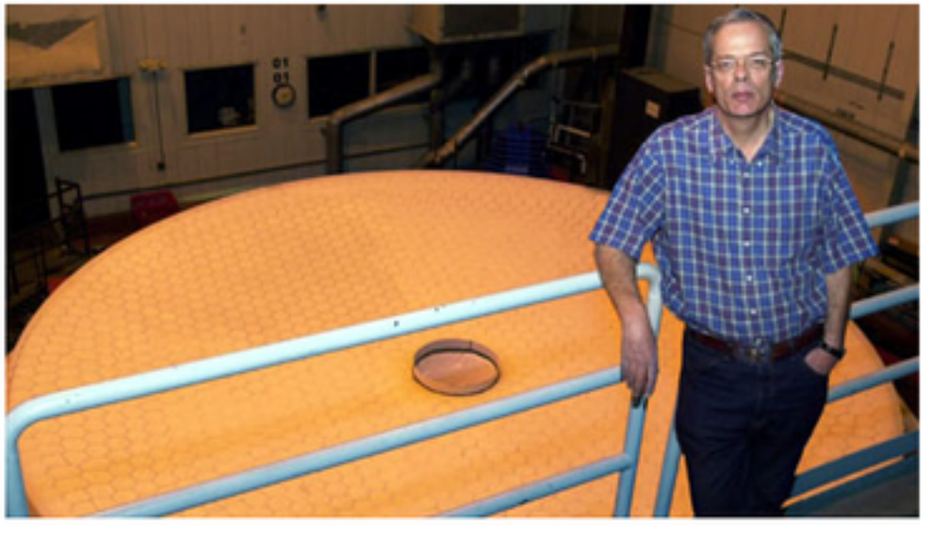


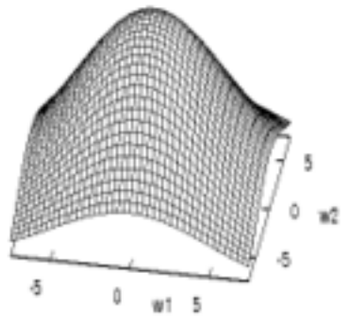
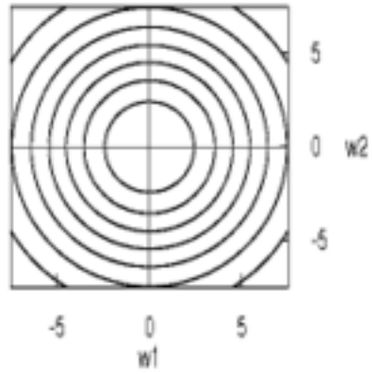
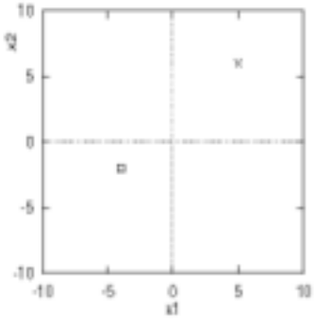
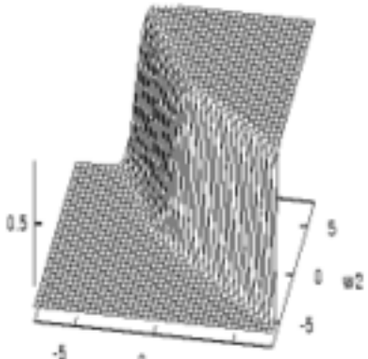
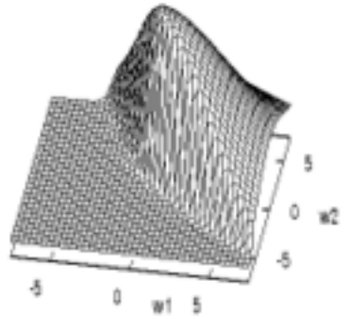
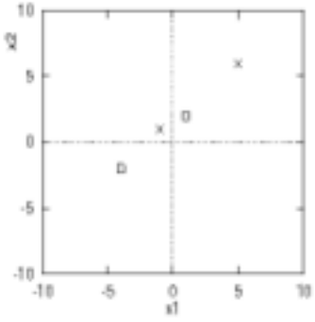
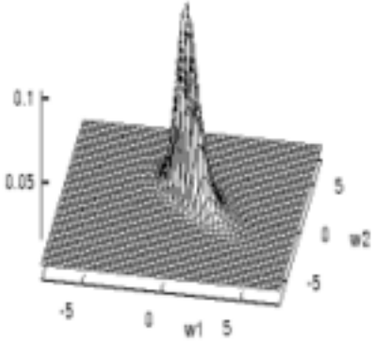
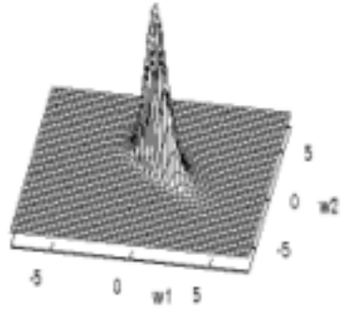
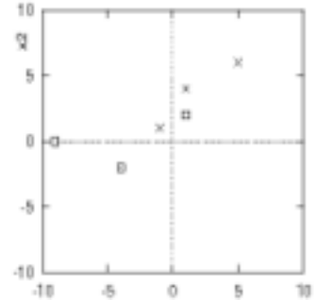
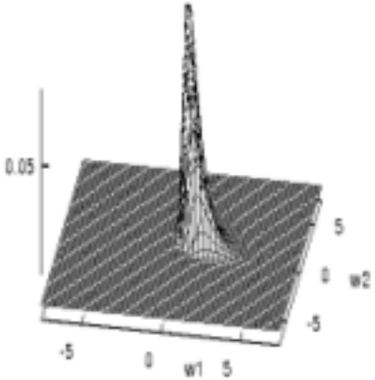
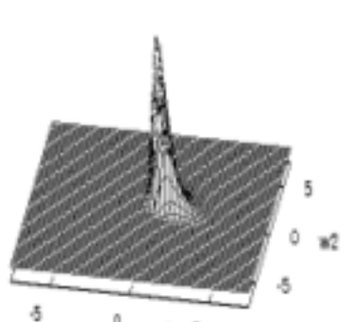
- Focussing telescopes
  - Roger Angel

Siemens arc welding facility

Applications

- Reading aloud - Nettalk
  - Sejnowski and Rosenberg
- Handwriting recognition - LeNet
  - Yann LeCun
- Weld toughness
  - Bhadeshia et al
- Focussing multiple-mirror telescopes
  - Roger Angel



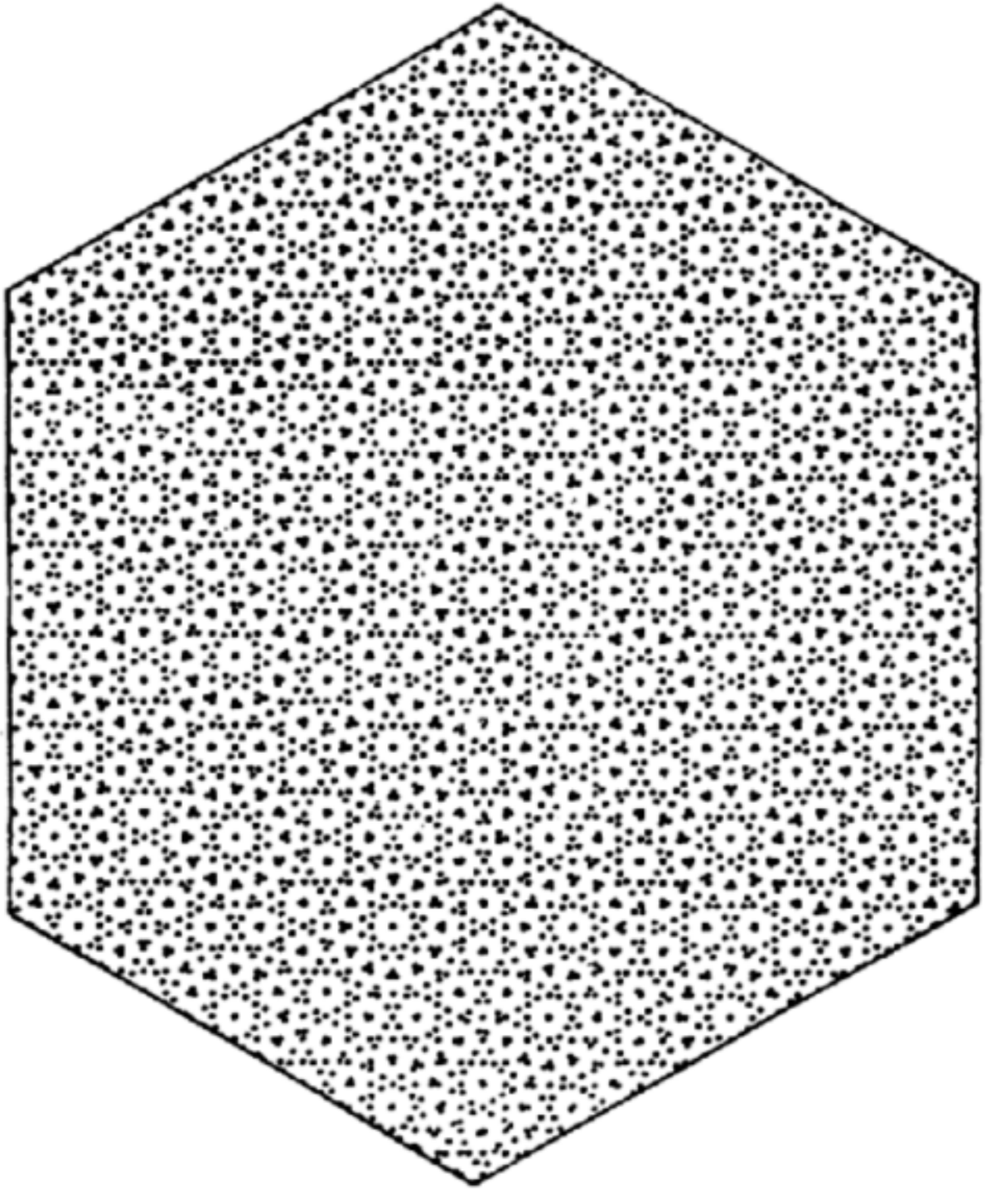
Data set	Likelihood	Probability of parameters	
$N = 0$	*(constant)*		
$N = 2$			
$N = 4$			
$N = 6$			

Feedforward neural networks

- Just high-dimensional curve-fitting?
- Brains do more

Brains

Brains

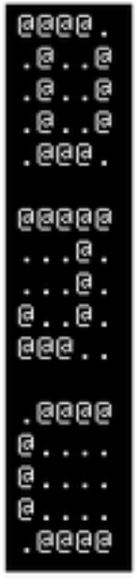


How to make a content-addressable memory?

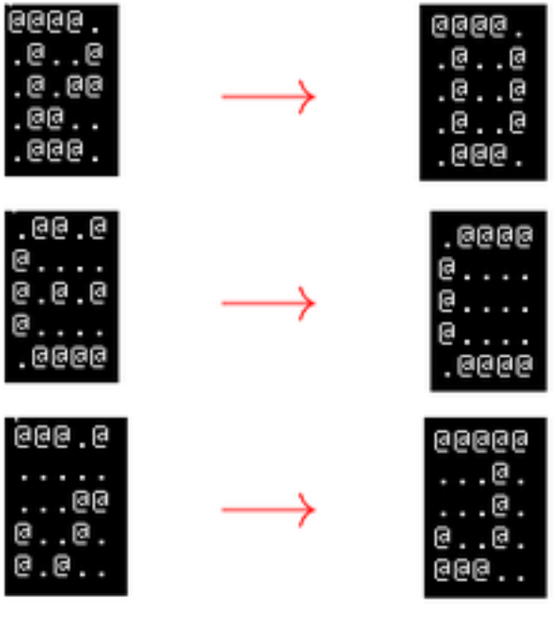
Shaken,

not...

Content-addressable memory challenge



- Make a dynamical system
  - 25 dynamical variables, 300 parameters (4-bit precision)
  - that has attracting fixed points at desired memories
  - such that noisy versions are automatically cleaned up



  - and such that new memories can be added incrementally
  - AND robust to corruption of more than half the parameters